玩转 Magel 语言

一本从零开始、一章只讲一件事的 Magel 入门书

作者：小兔糖糖 OvO
组织：真正的高手不会受到环境的影响
时间：Magel 1.0.0 • 2026 年 10 月 15 日

初次见面，我已经关注你很久了。别看我像个孩子，其实我比任何一位大人都了解这个世界。所以我可以用我的知识，换取你路上的见闻吗？——《原神》纳西妲

前　言

一本从零开始、一章只讲一件事的 Magel 入门书。

你不需要会 C，也不需要懂编译原理。这本书从头讲起，每章只解决一个问题，代码都能直接粘下来跑。读完它，你能独立写出结构清楚、资源管得干净的 Magel 程序，并且知道遇到问题时该去查哪一篇官方文档。

> **提示**
> 本书内容与 Magel 的当前语言版本和能力说明保持同步。你可以按下方目录在线阅读，也可以下载 PDF 电子书，离线阅读或打印。

这本书写给谁

- 刚开始学编程，想找一门“看得清楚”的语言入门；
- 写过 C / Java / Go / Python，想快速知道 Magel 和它们哪里不一样；
- 要维护一份逻辑、同时跑在服务端和嵌入式板子上，想知道这门语言能不能扛。

如果你只是想要一份语法清单，那你该看 语言参考；这本书的目标不一样——它想让你读懂一门语言的设计，而不只是记住写法。

怎么读

三条建议，按重要性排序：

1. 一定要动手。每章的代码都短，敲一遍的收益远大于读十遍。随便建一个目录，把代码存成 `x.mgl`，然后 `magel run x.mgl`。
2. 按顺序读，别跳。这本书是层层递进的：第 6 章解释为什么第 3 章的赋值那么“笨”，第 7 章的引用规则又是从第 6 章长出来的。跳着读，你会觉得语言在无理取闹。
3. 卡住了先看每章最后的「小结」和「下一步」。那里写清了哪些内容本章刻意没讲、以及去下一章还是去查官方文档。

书里的代码块分三类，看栅栏上的标注就知道：

标注	含义
完整的 `main` 函数	可以整段复制出去跑，输出的内容就写在代码下面
只有几行的片段	声明、表达式、语法示意，要放进程序里才能跑
命令行	以 `magel` 开头，直接敲

> **提示**
> Windows 上程序名是 `magel.exe`。在 Git Bash / MSYS 里写 `./magel` 能自动找到它；光写 `magel` 会走 PATH 查找，可能找不到当前目录里这份。

章节地图

全书按第 1 至第 25 章顺序展开，难度逐步提升。第 6 至第 8 章集中介绍值语义、引用和资源管理，建议连贯阅读。

章	标题	你会得到什么
01	第一个程序	装好工具链，跑出 `Hello, Magel!`，知道 `run` 和 `check` 的区别
02	猜数字	用一个小游戏把「变量、输入、循环、分支」串成一个能跑的程序
03	变量与类型	变量、常量、`auto`，以及 Magel 为什么不肯帮你偷偷转类型
04	函数	定义、参数、返回控制表达式、多值返回、重载与默认参数
05	控制流	`if` / `while` / `for` / `loop` / `foreach`，以及把 `loop` 当表达式用
06	值语义	赋值就是复制。为什么不移动也能保证安全
07	引用	`mut&` / `const&`、没有 `null`、引用不能活得比目标久
08	资源与作用域	`Drop` / `Closeable` / `new` / `delete` / `move`，没有 GC 怎么活
09	结构体	把字段和方法收进一个类型，构造器及 Equal/Hash/Clone 派生
10	枚举与模式匹配	代数数据类型，以及让编译器帮你查漏的 `match`
11	容器	`List` / `Map` / `Set`，以及容器也是值的后果
12	字符串与文本	拼接、下标（按字节`!`）、切片，以及 `std::text`
13	接口与运行期多态	`interface`、默认实现、`std::lang::Any`
14	泛型	泛型函数与结构体、约束、编译期单态化
15	函数值与闭包	`fn` 类型、具名局部闭包与四种捕获方式
16	错误处理	`Option` / `Result` / `?`，以及 `assert` 该用在哪
17	迭代器	`std::iter` 的惰性链：`range` / `map` / `filter` / `collect`
18	模块与工程	`module` / `import`、`magel new`、`magel.toml`、测试与格式化
19	预处理与宏	条件编译、文件包含与有序多括号文本宏
20	注解与反射	`@annotation`、运行期查元数据、按名调用
21	与 C 互操作	`abi` 块，调 C 的库、把自己的函数交给 C
22	交付与部署	`.svmb`、AOT、符号压缩、嵌入式与看门狗
23	综合项目：命令行待办清单	把前面二十章的东西拼成一个能用的程序
24	学生信息管理系统	MySQL、HTTP API、跨源前端、会话与权限，配真实界面截图

附录

附录	内容
附录 A	常见报错、诊断码与处理办法
附录 B	按能力域列出的标准库模块，以及去哪儿查细节

和官方文档的关系

这本书不替代官方文档，它是官方文档的导读。

你想	去哪
跟着书学语言	就是这儿
查某个语法怎么写	语言参考
理解这些设计为什么这么做	核心概念
5 分钟跑起来	快速开始
查某个标准库函数	库函数 API 参考
看官方给的可运行样例	`example/samples/`

本书正文里凡是引用官方文档的地方，都给了相对链接，可以随时跳过去。

开始之前

- 工具链：见 第 1 章。
- 编辑器：推荐装 `vscode/` 下的 Magel 插件，有语法高亮、补全、跳转与断点调试；不装也能读完整本书。
- 心里准备：第 6 到第 8 章会请你改掉一些习惯。别急着问“为什么不直接给我”，作者在那三章里把取舍写清楚了。

准备好了就翻到 第 1 章。

目 录

第 1 章 第一个程序 **1**

1.1 把工具装好 1

1.2 Hello, Magel 1

1.3 逐行看懂它 2

1.4 `run` 和 `check` 2

1.5 那行字背后发生了什么 3

1.6 小结 4

1.7 下一步 4

第 2 章 猜数字 **5**

2.1 第一步：先让它说一句话 5

2.2 第二步：读一个数进来 5

2.3 第三步：比一比，给个提示 6

2.4 第四步：让它一直问下去 7

2.5 第五步：让答案真的随机 8

2.6 完整程序 9

2.7 出错的时候 10

2.8 小结 11

2.9 下一步 11

第 3 章 变量与类型 **12**

3.1 变量：给值起个名字 12

3.2 为什么非要写类型 12

3.3 整型家族 13

3.4 浮点 13

3.5 `bool`、`char`、`String` 14

3.6 常量 `const` 与推导 `auto` 15

3.7 字面量怎么写 16

3.7.1 数字后缀：`1u` 与 `1.0f` 16

3.8 隐式转换：只放行「变宽」 17

3.9 显式强转：`(目标类型)值` 19

3.9.1 `cast<T>(value)`：把目标类型写清楚 20

3.10 字面量的取值范围 20

3.11 小结 21

3.12 下一步 22

第 4 章 函数 **23**

4.1 定义与调用 . . . 23
4.2 参数是副本 . . . 24
4.3 返回值 . . . 25
4.3.1 返回一个带值的控制表达式 . . . 25
4.4 重载：同名不同参 . . . 26
4.5 默认参数 . . . 28
4.6 变长参数 . . . 28
4.7 一次返回多个值 . . . 29
4.8 入口 `main` 的两种形态 . . . 30
4.9 小结 . . . 31
4.10 下一步 . . . 32

第 5 章 控制流 **33**
5.1 条件：`if` / `elif` / `else` . . . 33
5.1.1 条件也能产生值 . . . 34
5.2 `while`：先判断，再执行 . . . 36
5.3 `for`：三段式 . . . 36
5.4 `do { } while ( )`：先执行一次 . . . 37
5.5 `loop`：会「带值跳出来」的循环 . . . 38
5.6 `foreach`：遍历一串东西 . . . 39
5.7 `break` 与 `continue` . . . 40
5.7.1 指定退出哪一层循环 . . . 41
5.8 复合赋值与自增减 . . . 41
5.8.1 但 Magel 对「一个表达式里读写同一个对象」管得很严 . . . 42
5.9 小结 . . . 43
5.10 下一步 . . . 44

第 6 章 值语义 **45**
6.1 一个让人愣一下的例子 . . . 45
6.2 复制是整套的 . . . 46
6.3 函数参数也是复制 . . . 47
6.4 和 Rust 的关键区别：没有「移动」 . . . 47
6.5 代价：复制是要花钱的 . . . 48
6.6 显式转移所有权：`move` . . . 48
6.7 为什么值得这么设计 . . . 50
6.8 小结 . . . 50
6.9 下一步 . . . 50

第 7 章 引用 **52**
7.1 引用长什么样 . . . 52

7.2 用引用改到外面 . . . 53
7.3 用引用省掉复制 . . . 54
7.4 引用必须当场绑定：没有「空引用」 . . . 55
7.5 引用不能活得比目标久 . . . 55
7.5.1 那什么时候是合法的？ . . . 56
7.6 借用检查与别名共享 . . . 57
7.7 小结 . . . 58
7.8 下一步 . . . 58

第 8 章 资源与作用域 59

8.1 作用域就是一对花括号 . . . 59
8.2 `Drop`：离开作用域时自动调用 . . . 60
8.3 析构顺序：先自己的 `drop()`，再做字段 . . . 60
8.3.1 一个必须记住的禁忌 . . . 61
8.4 `Closeable`：用完立刻关，而且不会关两次 . . . 62
8.4.1 两个都实现的时候，顺序固定 . . . 63
8.5 堆对象：`new` 和 `delete` . . . 63
8.5.1 `delete` 两次没事 . . . 65
8.6 内存模型的边界 . . . 66
8.6.1 `defer` 和 `scope(exit)` . . . 66
8.7 小结 . . . 67
8.8 下一步 . . . 67

第 9 章 结构体 68

9.1 定义与使用 . . . 68
9.2 方法：把动作也收进去 . . . 69
9.3 `static`：属于类型，不属于对象 . . . 70
9.4 构造器与 `new(...)` . . . 71
9.5 让你的类型支持运算符 . . . 73
9.6 记住：结构体是值 . . . 74
9.6.1 从字段派生 Equal、Hash 和 Clone . . . 75
9.7 常见错误 . . . 77
9.8 小结 . . . 77
9.9 下一步 . . . 77

第 10 章 枚举与模式匹配 79

10.1 定义枚举 . . . 79
10.2 用 `match` 处理 . . . 79
10.3 穷尽性：编译器帮你查漏 . . . 81
10.4 枚举方法 . . . 83

10.4.1 为枚举派生接口 84
10.5 不止能匹配枚举：标量也支持 85
10.5.1 区间重叠的警告 87
10.6 守卫：先绑定，再加条件 87
10.7 什么时候用 match，什么时候用 if 88
10.7.1 match 表达式 88
10.8 小结 90
10.9 下一步 90

第 11 章 容器 **91**
11.1 List<T>：有序表 91
11.1.1 越界会被抓到 92
11.2 Map<K, V>：键 → 值 93
11.3 Set<T>：不重复，自动升序 94
11.4 集合字面量 94
11.5 数组：两种 95
11.6 别忘了：容器也是值 96
11.6.1 容器里的东西也是副本 98
11.6.2 容器归声明它的那一层 98
11.7 遍历 99
11.8 小结 100
11.9 下一步 100

第 12 章 字符串与文本 **101**
12.1 String 是什么 101
12.2 拼接、下标、切片、长度 101
12.3 格式化：format 102
12.4 下标是按字节算的 103
12.4.1 取到「半个字」会怎样 104
12.4.2 所以中文怎么按字处理？ 104
12.5 比较 105
12.6 std::text：文本工具箱 105
12.7 字符串是值：别在循环里拼 107
12.7.1 插值字符串 108
12.7.2 整行输入和文件文本 108
12.8 小结 109
12.9 下一步 109

第 13 章 接口与运行期多态 **110**
13.1 问题：长得不一样，但都会做同一件事 110

13.2 interface：给「能力」起个名字 . . . 111
13.3 结构体实现接口 . . . 111
13.4 默认方法：接口可以带实现 . . . 113
13.5 接口继承 . . . 114
13.6 一个结构体实现多个接口 . . . 115
13.7 接口引用：多态真正登场的地方 . . . 116
13.8 该用谁：静态调用还是接口？ . . . 119
13.9 小结 . . . 119
13.10 下一步 . . . 119

第 14 章 泛型 120

14.1 问题：换一种类型，逻辑就得抄一遍 . . . 120
14.2 泛型函数：把「类型」也变成参数 . . . 120
14.3 显式写出来，或者让编译器推 . . . 121
14.4 泛型结构体 . . . 123
14.5 约束：给类型参数提要求 . . . 124
14.6 单态化：编译期展开，运行期零开销 . . . 126
14.7 默认类型参数 . . . 127
14.8 接口还是泛型？ . . . 128
14.8.1 编译期整数参数 . . . 128
14.8.2 关联类型 . . . 129
14.8.3 泛型返回控制表达式 . . . 129
14.9 小结 . . . 131
14.10 下一步 . . . 131

第 15 章 函数值与闭包 132

15.1 问题：逻辑一样，只有「条件」不同 . . . 132
15.2 函数类型：R fn(A, B) . . . 134
15.3 函数可以当值用 . . . 134
15.4 fn：现场写一个小函数 . . . 135
15.5 省略参数类型：靠上下文推 . . . 136
15.6 闭包：记住定义处的变量 . . . 137
15.7 四种捕获方式 . . . 138
15.7.1 const&：默认，只读借用 . . . 138
15.7.2 mut&：写回来源 . . . 140
15.7.3 copy：拿一份独立副本 . . . 140
15.7.4 move：把来源整个搬进闭包 . . . 142
15.8 捕获列表：逐个变量指定方式 . . . 143
15.8.1 列表后面还能写「其余变量的默认方式」 . . . 143
15.9 复制闭包：拥有状态会复制，借用仍然共享 . . . 145

15.10生命周期：借来的，不能活过来源 . . . 146
15.11带 Drop 的类型：谁负责销毁 . . . 148
15.12捕获方式速查 . . . 150
15.13一个小坑：表达式体 vs 语句体 . . . 151
15.14闭包当工厂：返回一个函数 . . . 152
15.14.1 移动后访问诊断 . . . 152
15.15语句级具名闭包 . . . 153
15.16多层高阶闭包：函数也可以接收函数 . . . 154
15.17组合闭包：接收函数，再返回函数 . . . 156
15.18小结 . . . 158
15.19下一步 . . . 158

第 16 章 错误处理 **159**
16.1 为什么不用异常 . . . 159
16.2 Option<T>：可能没有值 . . . 159
16.3 Result<T, E>：可能失败，还带原因 . . . 160
16.4 四个「取出来」的工具，别用错场合 . . . 162
16.5 ?：失败就自动往上抛 . . . 163
16.6 panic 和 assert：什么时候该「直接崩」 . . . 166
16.6.1 用户自定义传播 . . . 167
16.7 小结 . . . 169
16.8 下一步 . . . 169

第 17 章 迭代器 **170**
17.1 一条链由三段组成 . . . 170
17.2 起点：range 和 List.iter() . . . 170
17.3 中间操作：map / filter / take / skip / rev . . . 171
17.4 终结操作：collect / fold / count . . . 171
17.5 串成一条链 . . . 173
17.6 惰性：写了 ≠ 跑了 . . . 173
17.7 一个坑：List.iter() 的元素是引用 . . . 174
17.8 foreach：把链条跑起来 . . . 176
17.9 foreach 拿到的是元素引用（最容易踩的一个坑） . . . 177
17.9.1 mut&：拿地址，而不是元素 . . . 178
17.10foreach 整数区间 . . . 179
17.11小结 . . . 179
17.12下一步 . . . 179

第 18 章 模块与工程 **181**
18.1 module：给文件一个名字 . . . 181

18.2 import：把别的模块接进来 . . . 181
18.3 起一个工程：magel new . . . 182
18.4 magel.toml：工程配置文件 . . . 182
18.5 三个日常命令 . . . 183
18.6 magel fmt：让风格统一 . . . 184
18.6.1 终端查手册 . . . 185
18.6.2 编辑器增量提示 . . . 185
18.6.3 inline：函数定义的内联属性 . . . 185
18.7 小结 . . . 186
18.8 下一步 . . . 186
18.8.1 终端手册的入口 . . . 186

第 19 章 预处理与宏 **187**
19.1 条件编译：%define + %if . . . 187
19.2 按「有没有定义」分支：%ifdef / %ifndef . . . 188
19.3 从命令行传开关：-D . . . 188
19.4 %include：把文件原地拼进来 . . . 189
19.5 宏：$macro 文本替换 . . . 189
19.5.1 有序多括号分组 . . . 190
19.6 宏的第一个坑：括号要自己加 . . . 191
19.7 第二个坑：%if 只认符号名 . . . 192
19.8 括号形参：每次出现都包一层括号 . . . 193
19.9 expr 宏：整段展开再包一层 . . . 193
19.10 变长尾参 ...rest . . . 194
19.11 按实参个数重载 . . . 194
19.12 $for：按尾参重复一段 . . . 195
19.13 尾参逐元素加括号 ...(values) . . . 195
19.14 $concat：拼出一个标识符 . . . 196
19.15 $stringify：取调用点写下的原文 . . . 196
19.16 $replace：一行写完的宏 . . . 196
19.17 $delete_macro：撤掉再重新定义 . . . 197
19.18 $0～$9：宏内的唯一临时名 . . . 197
19.19 $line / $file / $module 和 $assert . . . 198
19.19.1 宏内条件与局部绑定 . . . 199
19.20 完整指令速查 . . . 199
19.21 什么时候该用，什么时候别用 . . . 200
19.22 小结 . . . 201
19.23 下一步 . . . 201

第 20 章 注解与反射 **202**

20.1 声明一个注解类型 . . . 202
20.2 把注解贴到声明上 . . . 202
20.3 能贴在哪里 . . . 202
20.4 读注解：运行期反射 . . . 203
20.5 按名字调用函数 . . . 204
20.6 构建与目标 VM 的反射能力 . . . 206
20.7 什么时候该用注解 . . . 207
20.8 小结 . . . 208
20.9 下一步 . . . 208

第 21 章 与 C 互操作 209

21.1 三种集成方式 . . . 209
21.2 `abi` 块：调动态库里的 C 函数 . . . 209
21.3 结构体、引用、字符串 . . . 210
21.3.1 支持的类型一览 . . . 211
21.4 数组与「可空」：边界上的两个特殊话题 . . . 212
21.5 宿主胶水：把 C++ 函数注册进来 . . . 212
21.5.1 `impl`：把宿主函数绑定到协议 . . . 213
21.6 反方向：宿主调用 Magel . . . 214
21.7 小结 . . . 215
21.8 下一步 . . . 215

第 22 章 交付与部署 216

22.1 三种跑法 . . . 216
22.2 解释执行：开发者的默认档 . . . 216
22.3 字节码：`.svmb` . . . 216
22.4 AOT：编成本地可执行文件 . . . 217
22.5 交付安全：符号压缩 . . . 218
22.6 嵌入式：同一份逻辑，压到板子上 . . . 218
22.7 内存与看门狗 . . . 219
22.8 把 Magel 嵌进宿主程序 . . . 219
22.9 选型速查 . . . 220
22.10 小结 . . . 220
22.11 下一步 . . . 220

第 23 章 综合项目：命令行待办清单 221

23.1 先想清楚数据长什么样 . . . 221
23.2 读：把文件变回清单 . . . 221
23.3 写：把清单存回文件 . . . 222
23.4 命令分派：`main` 是入口 . . . 223

23.5 完整程序 . . . 225
23.6 跑一遍 . . . 230
23.7 复盘：这个项目用了哪些章 . . . 231
23.8 接下来可以做什么 . . . 231

第 24 章 学生信息管理系统：前后端分离 233
24.1 先找到三个入口 . . . 233
24.2 准备独立练习数据库 . . . 233
24.3 先把接口跑通 . . . 235
24.4 权限必须在后端生效 . . . 236
24.5 数据关系与完整操作 . . . 237
24.6 沿 main.mgl 读完一次请求 . . . 238
24.6.1 main：先准备长期状态，再处理短期请求 . . . 238
24.6.2 Request 与 readRequest：TCP 字节不等于完整 HTTP 请求 . . . 239
24.6.3 route：先处理公开入口，再查会话 . . . 239
24.6.4 dispatchStudents 与 allow：路由不直接等于业务授权 . . . 240
24.6.5 JSON、SQL 与结果封装：新增和查询各追一遍 . . . 241
24.6.6 sendResponse：统一协议出口，再回到浏览器 . . . 241
24.7 验收与交付 . . . 242

第 25 章 单片机上的 Magel 解释器 243
25.1 单片机上解释的是什么 . . . 243
25.2 先分清三类内存 . . . 243
25.3 写一个可退出的 LCD 程序 . . . 244
25.4 从 Magel 签名到原生外设 . . . 244
25.5 构建、加载与菜单 . . . 244
25.6 把五个源码入口串起来 . . . 245
25.6.1 library/stackvm：程序、数据与执行器分开 . . . 246
25.6.2 source/stackvm_mcu：把桌面服务替换为有预算的宿主 . . . 246
25.6.3 板端 main.c：先让硬件能支撑 C++ 宿主 . . . 246
25.6.4 magel_board.cpp：菜单拥有宿主，Execute 拥有一轮程序 . . . 247
25.6.5 board.mgl 与 RegisterBoard：名字和实参数量必须对齐 . . . 247
25.7 验证退出与错误恢复 . . . 248

附录 A 常见错误速查 249
A.1 报错的三种长相 . . . 249
A.2 编译期错误速查 . . . 250
A.2.1 「不能隐式转换」——最高频的一条 . . . 250
A.2.2 「整数字面量超出取值范围」 . . . 251
A.2.3 引用相关的错，多在「生命周期」上 . . . 252

A.3 语法错误速查 . . . 252
A.4 运行期错误速查 . . . 253
A.5 工具链与工程错误 . . . 254
A.6 排查五步法 . . . 255
A.7 下一步 . . . 255

附录 B 标准库速查 256

B.1 先看一个例子 . . . 256
B.2 核心模块 `std::lang`（不用 import） . . . 256
B.3 按能力域速查 . . . 257
B.3.1 数学与数值 . . . 257
B.3.2 文本与数据格式 . . . 257
B.3.3 并发与协作 . . . 258
B.3.4 系统与环境 . . . 258
B.3.5 迭代 . . . 258
B.4 三条容易踩的约定 . . . 259
B.5 细节去哪儿查 . . . 259
B.6 下一步 . . . 259

第 1 章 第一个程序

万事开头难，但这一章只干三件事：把工具装好、跑出一行字、看懂这行字是怎么来的。

读完这一章，你会有第一个能跑的程序，也会知道「出错」在 Magel 里长什么样——后者比前者重要，因为接下来每一章你都会用到它。

1.1 把工具装好

Magel 只有一个命令行程序，叫 `magel`，解释、编译、建工程、跑测试全在它里面。装它有两条路。

这条路最短：用发布包。拿到 `dist/magel-<版本>-<平台>-<架构>.tar.gz`，解开，把里面的 `bin/` 加进 PATH，就完事了。

这条路最全：从源码构建。适合想改编译器的人，也适合想看清内部的人：

```
sh gen.sh      # 用 IELR 从 magel.gram 生成前端
make           # 构建 magel 与 libparser.a
```

`gen.sh` 需要一份 ParseEngine 文法生成器，细节见快速开始，这里不展开。

装完了，敲一句：

```
magel help
```

屏幕上滚出一大串用法，就说明装好了。先别急着读完那串帮助——那是给有经验的人查的，跟着这本书走，你会一个个认识里面的选项。

> **Windows 用户注意**
>
> 程序名是 `magel.exe`。在 Git Bash / MSYS 里写 `./magel` 能自动找到 `magel.exe`；光写 `magel` 会去 PATH 里找，可能找到的不是你当前目录这一份。本书后文一律写 `magel`，你自己敲的时候按环境补上 `./`。

1.2 Hello, Magel

新建一个文件 `hello.mgl`，内容如下：

```
1  int64 main()
2  {
3      println("Hello, Magel!");
4      return 0;
5  }
```

```
Hello, Magel!
```

然后运行它：

```
magel run hello.mgl
```

屏幕上出现 Hello, Magel!，第一个程序就成了。

magel run 这个写法看着啰嗦，其实 run 是可以省的——直接

```
magel hello.mgl
```

效果一样。但本书会一直写 magel run，因为把场景写出来在人多的时候少很多误会：magel hello.mgl Alice 和 magel run hello.mgl Alice 是同一件事，但 magel check hello.mgl 完全是另一件事。

1.3 逐行看懂它

就三行，一行一行来。

第一行：int64 main() {

程序从哪里开始跑？从 main 开始。这个名字是定死的，不能改成 Main、start 或者别的什么。前面的 int64 是它返回的东西的类型，() 里现在空着，意思是它不接收参数。

int64 是一种整数类型：能装 64 位的有符号整数，大概正负九百多万亿。为什么入口要返回一个整数？因为操作系统想看它：返回 0 表示「正常结束」。你在终端里看到的 echo $?，读的就是这个值。

第二行：println("Hello, Magel!");

println 是「打印一行」的意思：把括号里的东西写到屏幕上，末尾自动换行。它是 Magel 核心库里最常用的函数，不用 import 就能用。

行尾那个 ; 是分号，表示这句话说完了。跟 C、Java 一样。

第三行：return 0;

告诉操作系统「我正常结束了」，然后交还控制权。

最后的花括号 } 是 main 的结尾。

看出来了吗——花括号划范围，分号断句，这和 C 系语言一模一样。这是 Magel 刻意这么做的：你已经会的东西，它不想让你重学一遍。

1.4 run 和 check

现在故意把程序弄坏一次，看看会发生什么。把第二行末尾的分号删掉：

```
1 int64 main()
2 {
3     int64 x = 1
4     return 0;
5 }
```

再跑：

```
语法错误：hello.mgl 第 3 行第 5 列：期望 条件(if) / 自增(++) / 自减(--) / 加号(+) 等 44 个之一，实际是
    返回(return)
  |
3 |     return 0;
  |     ~~~~~~
```

三件事值得注意：

- 它告诉你第 3 行第 5 列，还把出问题的那段源码抄给你、底下画上波浪线。第 1 行少了个分号，报的却是第 3 行——这是解析器在往后看：它读完第 2 行还在等一个分号，结果等来了 `return`，于是「在 `return` 这里」断了。看到报错位置和你以为的不一致时，先看报错位置的前面。
- 内部记号名被翻译成了人话：`条件 (if)`、`返回 (return)`。你没看错，就是中文。报错语言跟着系统走，可以用 `magel lang en` 或环境变量 `MAGEL_LANG` 换掉。
- 那句「等 44 个之一」说了它在等什么。这种话现在看不懂没关系，以后你写多了，一眼就知道是「这里少了个东西」。

重要：这个错误是编译期就报出来的，程序一个字符都没执行。这就是 Magel 和其他很多语言不同的地方——能编译期拦住的，绝不放到运行期。

如果你只想检查代码写没写对、不想真的跑它，用：

```
magel check hello.mgl
```

`check` 做前两步（预处理、解析与类型检查），不执行、也不产出文件。写代码的时候这个命令用得比 `run` 还多：先 check 再 run，能省掉很多「跑到一半才发现写错了」的时间。

> **提示**
>
> 还有个更细的用法：`magel check hello.mgl json`，输出机器可读的 JSON 诊断。这是给编辑器插件用的，平时不用管。

1.5 那行字背后发生了什么

`magel run` 看着像「执行脚本」，其实它在一个进程里做完了四件事：

1. 预处理——处理 `%include`、`%define`、`%if` 这类指令（第 19 章讲）；
2. 解析与类型检查——语法、类型、引用的生命周期都在这里查，不通过就不执行；
3. 降级为四元式——把程序翻译成虚拟机认识的中间表示；
4. 解释执行——在当前进程里跑起来。

前三步就是「编译」，第四步才是「执行」。把 `run` 换成别的场景词，前三步是一样的，只是最后一步换了形态：

你想	命令
直接跑起来	`magel run src/main.mgl`
只检查、不跑	`magel check src/main.mgl`
导出字节码 `.svmb`	`magel svmb src/main.mgl out build/app`
导出本地可执行程序	`magel aot src/main.mgl out build/app.exe`

第四种（AOT）会把程序真的编译成本地机器码，得到一个不依赖 `magel` 的可执行文件。这三条路的区别在第 22 章讲；现在你只要知道：同一份源码，可以有好几种走法。

1.6 小结

这一章你认识了：

- 入口函数固定叫 `main`，返回 `int64` 或 `void`；`void main` 正常完成时退出码为 0，panic 仍失败；
- `println(...)` 打印一行，来自核心库，不用 import；
- 分号断句、花括号划范围，和 C 系语言一样；
- `magel run` 跑，`magel check` 只查；
- 报错会给出行列位置和画了波浪线的源码，而且大多在编译期就报出来了。

还有一件事本章没讲：`println` 为什么不用 `import`？因为它属于核心模块 `lang`，而 `lang` 是自动加载的。别的模块就没这个待遇了，第 18 章会讲清楚。

1.7 下一步

光看「Hello, Magel」是学不会编程的。下一章我们写一个真的有用的小程序：键盘猜数字。它会用到变量、输入、循环和分支——四个最基础的积木，而且你会看到它们是怎么串在一起的。

翻到第 2 章猜数字。

第 2 章 猜数字

第 1 章的程序只会说一句话。这一章我们要写一个真的会跟你对话的东西：

> **提示**
> 程序想一个 1 到 100 之间的整数，你输入猜测，它告诉你「太大」还是「太小」，直到你猜中为止，然后告诉你一共猜了几次。

这个程序只用四个最基础的积木：变量、输入、分支、循环。我们把它们一块一块拼起来——每一步都能单独跑，你随时可以停下来看。

> **提示**
> 本章的代码建议你真的敲一遍。后面所有章节都建立在这一章的四种积木上。

2.1 第一步：先让它说一句话

先别管猜数字，先让程序开口：

```
int64 main()
{
    println("我想好了一个 1 到 100 的数，你猜是多少？");
    return 0;
}
```

```
我想好了一个 1 到 100 的数，你猜是多少？
```

没有新东西。就一句 `println`。

2.2 第二步：读一个数进来

光说话不够，它得能听见你。改一下：

```
int64 main()
{
    println("我想好了一个 1 到 100 的数，你猜是多少？");
    int64 guess = inputInt();
    println("你猜的是 {}。", guess);
    return 0;
}
```

运行（`--input` 后面的内容就是喂给程序的标准输入）：

```
magel run guess.mgl --input "42"
```

```
我想好了一个 1 到 100 的数，你猜是多少？
你猜的是 42。
```

三件新东西，一个一个说。

`int64 guess = inputInt();` ——读一个整数。

`inputInt` 会去读用户输入的一项（用空白分隔），按 `int64` 解析，然后把值交出来。如果输入的不是合法整数，程序会在那里停下来报错，不会给你一个莫名其妙的 0。这一点和第 3 章要讲的「不隐式转换」是同一种性格：宁可当场停下，不猜你的意思。

核心模块 `lang` 会自动加载，`inputInt` 和 `println` 都可以直接使用。示例优先写简短名称；只有同名符号发生冲突，或需要说明名称来源时，才使用 `std::lang::inputInt`、`std::lang::println` 这样的完整名称。

`" 你猜的是 {}。"` ——`{}` 是位置占位符。

它不是 C 的 `printf` 那套 `%d`：你不用告诉它「这个位置是整数」，类型是 Magel 自己按实参推的。写错了——比如拿 `%d` 或者 `{name}`——编译期就报错，不会等到运行期才发现。

2.3 第三步：比一比，给个提示

现在让它知道答案是 42，并且判断你的猜测：

```
int64 main()
{
    int64 secret = 42;
    println("我想好了一个 1 到 100 的数，你猜是多少？");
    int64 guess = inputInt();
    if (guess < secret)
    {
        println("太小了。");
    }
    elif (guess > secret)
    {
        println("太大了。");
    }
    else
    {
        println("猜对了！");
    }
    return 0;
}
```

输入 50：

```
我想好了一个 1 到 100 的数，你猜是多少？
太大了。
```

`int64 secret = 42;` 定义了一个变量：类型 `int64`、名字 `secret`、初始值 `42`。这里的 `=` 是赋值，不是数学上的「相等」。第 6 章你会知道它到底做了什么。

`if / elif / else` 就是分支。注意两点：

- 条件是写在括号里的，和 C 一样：`if (guess < secret)`；
- 「否则如果」在 Magel 里拼作 `elif`，不是 `else if`，也不是 `elsif`。

每个分支的主体都要用 `{ }` 包起来。哪怕只有一句，也要包——Magel 没有「不写花括号只作用一行」那套规矩，省下的这点字换来的是不会看错的结构。

2.4 第四步：让它一直问下去

一次猜不中就要再来一次。把「读 + 判断」塞进一个循环：

```
int64 main()
{
    int64 secret = 42;
    println("我想好了一个 1 到 100 的数，你猜是多少？");
    while (true)
    {
        int64 guess = inputInt();
        if (guess < secret)
        {
            println("太小了。");
        }
        elif (guess > secret)
        {
            println("太大了。");
        }
        else
        {
            println("猜对了！");
            break;
        }
    }
    return 0;
}
```

输入 `50` `30` `42`：

```
我想好了一个 1 到 100 的数，你猜是多少？
太大了。
太小了。
猜对了！
```

`while (true) { ... }` 是「只要条件成立就一直做」。条件是 `true`，字面意思就是永远成立——所以这个循环本身不会自己停。

`break;` 就是那个出口：猜对时跳出去。

> **提示**
>
> 有没有更贴切的写法？有。当循环的唯一出路是 `break` 时，Magel 允许你写 `loop { ... }`，意思和 `while (true)` 一样，但一眼就能看出「这是个靠 `break` 结束的循环」。第 5 章会讲 `loop` 的另一个用法：带着值跳出来。

顺便说明一个容易踩的点：变量 `guess` 在每次循环里都是新的。写在 `while` 里面的声明，每转一圈都会重新做一次。所以它不是「上一轮的 `guess` 被改掉」，而是「新一轮又创了一个 `guess`」。第 8 章会讲清这种「出作用域就消失」的规则。

2.5 第五步：让答案真的随机

固定答案 42 只能骗自己。要随机数，得请标准库帮忙：

```
import std::random;

int64 main()
{
    Random gen = Random::withSeed((uint64)2026);
    int64 secret = gen.nextInt(1, 100);
    println("答案是 {}。", secret);
    return 0;
}
```

```
答案是 62。
```

四件事：

`import std::random;` ——把标准库的 `random` 模块引进来。除了核心模块 `lang` 之外的模块，都要显式 `import`（第 18 章）。

`Random gen = ...` ——`Random` 是一个结构体（struct），你可以先把它当成「一个装着随机数发生器的小盒子」。结构体在第 9 章细讲。

`Random::withSeed((uint64)2026)` ——`::` 表示调这个类型上的静态方法。`withSeed` 是「用指定种子造一个发生器」。

那个 `(uint64)` 强转是必需的，不是装饰。`2026` 这个字面量默认是 `int64`，而 `withSeed` 要的是 `uint64`；Magel 不做隐式转换，你不写它就编译不过：

```
magel: 调用 static 函数 Random::withSeed
    找不到匹配的重载：(int64)；没有参数类型能对上的重载（候选：Random.withSeed）
```

第一次遇到会觉得它死板。但换个角度：报错说「我找不到 `(int64)` 的重载」，它把它拿到了什么告诉你了。第 3 章会专门讲这套转换规则。

为什么用种子，不用真随机？因为要能复现。

`withSeed(2026)` 每跑一次都给你同一串数，所以本书能写死「答案是 62」，你照着敲也能得到 62。真实程序里想要每次都不一样，就用：

```
Random gen = Random::fromEntropy();
```

2.6 完整程序

把上面五步合起来：

```
import std::random;

int64 main()
{
    Random gen = Random::withSeed((uint64)2026);
    int64 secret = gen.nextInt(1, 100);
    int64 tries = 0;

    println("我想好了一个 1 到 100 的数，你猜是多少？");
    while (true)
    {
        int64 guess = inputInt();
        tries = tries + 1;
        if (guess < secret)
        {
            println("太小了。");
        }
        elif (guess > secret)
        {
            println("太大了。");
        }
        else
        {
            println("猜对了！答案是 {}，你用了 {} 次。", secret, tries);
            break;
        }
    }
    return 0;
}
```

```
magel run guess.mgl --input "50 75 62"
```

```
我想好了一个 1 到 100 的数，你猜是多少？
太小了。
太大了。
猜对了！答案是 62，你用了 3 次。
```

新东西只有一处：`tries = tries + 1;`

这行读作「`tries` 变成它原来的值加一」。`=` 依然是赋值。Magel 也支持写成 `tries += 1;`（第 5 章会讲这一串复合赋值），但本书前面会一直用 `tries = tries + 1` 这种最直白的写法——先写清楚，再写短。

顺带一提：`++tries` 那种自增写法 Magel 也有，但它对「同一个变量在一个表达式里被读写两次」管得很严，用不好会直接报错。第 5 章再说。

2.7 出错的时候

这个程序很容易出错，两种错法都很典型。

输入不够。程序要两个数，你只给了一个：

```
1  int64 main()
2  {
3      int64 first = inputInt();
4      int64 second = inputInt();
5      println("{} {}", first, second);
6      return 0;
7  }
```

```
magel: 运行期错误: 输入不够（第 2 项缺失）
出错位置: guess.mgl:3（main）
调用栈（外 → 内）:
  main  guess.mgl:3 [thread 0 coroutine 0 ip 3]
```

输入不是整数。

```
1  int64 main()
2  {
3      int64 value = inputInt();
4      println("{}", value);
5      return 0;
6  }
```

```
magel: 运行期错误: std::lang::inputInt 读到的不是合法 int: "abc"
出错位置: guess.mgl:2（main）
调用栈（外 → 内）:
  main  guess.mgl:2 [thread 0 coroutine 0 ip 3]
```

注意这两条报错和上一章那条语法错误的区别：

	什么时候报	谁的问题
语法错误：…	编译期，程序一个字符都没跑	代码写错了
magel：运行期错误：…	运行期，跑到那一句才炸	数据不合法

Magel 的立场是：代码的错误要尽量在编译期抓干净。所以「类型对不上」是编译期的事，而「用户输入了个 abc」只能运行期知道——这种事它只能诚实地停下来，并告诉你停在哪一行。

程序停下来的位置写成了 guess.mgl:9，连调用栈都给了；写大程序时这东西能救你。

2.8 小结

这一章你认识了四种积木，还有几个语法细节：

你会了	关键点
变量	int64 secret = 42; ——类型写在名字前面
输入	inputInt() 读一个整数，读不合法就报错
分支	if (…) { } elif (…) { } else { }，主体必须带花括号
循环	while (true) { …break; }
格式化	{} 是位置占位符，类型由实参推导
模块	import std::random;；只有 lang 是自动加载的
静态方法	Random::withSeed(...)，:: 后面是类型级的东西
强转	(uint64)2026 ——Magel 不做隐式转换

还有一个习惯要开始养成：先 check 再 run。

```
magel check guess.mgl    # 只查语法和类型，不执行
```

2.9 下一步

第 2 章里我们「先用后解释」了不少东西——变量到底是什么、为什么 2026 不能直接交给 withSeed、{} 是怎么知道该打印成整数的。

下一章就来补上最基础的一块：变量与类型。

翻到第 3 章变量与类型。

第 3 章 变量与类型

第 1、2 章你已经写过不少变量了，但都是「照抄」。这一章把它讲清楚：

- 变量是什么、为什么每个变量都要写类型；
- Magel 有哪些基础类型；
- 数字、字符、字符串分别怎么写；
- 以及本章的重头戏：Magel 什么时候替你转类型、什么时候非要你签字。

这一章会让你第一次觉得 Magel「有点死板」。请忍着别关页面——第 3.8 节会把这份死板值不值讲清楚。

3.1 变量：给值起个名字

```
int64 score = 30;
```

这一行的读法从左到右：

部分	含义
`int64`	类型：这个变量装 64 位有符号整数
`score`	名字：你自己起的，区分大小写
`=`	赋值：把右边算出来的值放进这个变量
`30`	初始值

类型写在名字前面，这是 C / Java / C++ 的习惯，不是 Rust / TypeScript 那种 `score: int64` 或者 `let score`。和 C 系一样的写法是故意的：Magel 不打算让你重新学一遍已经会的东西。

三个必须记住的细节：

1. 分号结尾。Magel 不用换行来表示语句结束，写成一整行也行。
2. 区分大小写。`score` 和 `Score` 是两个不同的名字。
3. 变量必须先声明再用。写到没声明过的名字是编译错误，不是「默认 0」。

3.2 为什么非要写类型

很多现代语言会帮你推：`let score = 30` 就够了。Magel 能推（下一节就讲 `auto`），但声明时默认要求你写出来。理由和它的整体性格一致：

- 读代码的人不用猜。`int64 score = getScore();` 一眼就知道这里是整数；`let score = getScore()` 得跳去看 `getScore` 的签名。
- 它不猜你的意思。记住第 2 章那条报错——`2026` 不能直接交给要 `uint64` 的函数。Magel 宁可让你写一个 `(uint64)`，也不愿意在某处悄悄换一下。
- 出错点离原因近。类型写错，报错就在那一行；不写类型，错可能传到十层外面才炸。

3.3 整型家族

```
int64 main()
{
    int8 tiny = 127;
    int32 normal = 70000;
    int64 huge = 9000000000;
    uint8 byte = 255;
    println("{} {} {} {}", tiny, normal, huge, byte);
    return 0;
}
```

```
127 70000 9000000000 255
```

有符号	无符号	位数	大概装得下
int8	uint8	8	±127 / 0 到 255
int16	uint16	16	±32767 / 0 到 65535
int32	uint32	32	±21 亿 / 0 到 42 亿
int64	uint64	64	$\pm 9.2\times10^{18}$ ——默认选手

日常直接用 int64。只有两种情况才去挑更窄的：

- 要跟硬件、文件格式、二进制协议对齐（这时你知道必须是 8 位还是 16 位）；
- 要省内存（嵌入式板子上，一个 8 位字段和 64 位字段差 8 倍）。

为什么没有 int、long 这种名字？因为那些名字的含义会随平台变，而这门语言要「一份逻辑从 MCU 跑到云端」——平台上不确定的东西，类型里不能含糊。

3.4 浮点

```
int64 main()
{
    float64 ratio = 3.25;
    float32 single =(float32)1.5;
    println("{} {}", ratio, single);
    return 0;
}
```

```
3.25 1.5
```

float64（双精度）是默认选手，float32（单精度）在数值算法、图形、嵌入式里省内存时用。

`float64` 一栏里那个（`float32`）是必需的——直接写 `float32 single = 1.5;` 会报「不能隐式转换：float64 -> float32」。原因见第 3.8 节：字面量 `1.5` 的类型是 `float64`，往更窄的类型里放就得自己签字。

> **一个容易上当的地方**
>
> `123456789.0` 打印出来是 `1.23e+08`，`1000000000.0` 打印出来是 `1e+09`。浮点打印会走「短」的那条路，别拿它当精确输出用。要定格式，用 `format` 的样式参数（第 12 章）。

3.5 bool、char、String

```
int64 main()
{
    bool ready = true;
    bool done = false;
    char initial = 'M';
    String word = "Magel";
    println("{} {} {} {}", ready, done, initial, word);
    return 0;
}
```

```
1 0 M Magel
```

注意第一眼就有点怪的地方：`bool` 打印出来是 `1` 和 `0`，不是 `true` / `false`。它不是「字符串化的布尔」，打印时当整数处理了。要比对文本，得自己拼：

```
int64 main()
{
    bool ready = true;
    String label;
    if (ready)
    {
        label = "yes";
    }
    else
    {
        label = "no";
    }
    println("{}", label);
    return 0;
}
```

```
yes
```

> **C 程序员注意**
>
> Magel 没有三目运算符。写 `ready ? "yes" : "no"` 是语法错误，老老实实用 `if / else`。可以使用 `if (条件) { 值一 } else { 值二 }` 条件表达式：只求值命中的分支。

- `bool` 只有 `true` 和 `false` 两个字面量，没有「非零即真」：

```
magel: if 条件必须是 bool，现在给的是 int64（数值条件请显式写成 `(bool)值` 或比较表达式）
```

  硬要拿数字当布尔，得签字写成 `(bool)值`——但十有八九你真正想要的是一个比较表达式。
- `char` 是单个字符，用单引号：`'M'`。它和整数不是一回事，`char c = 65;` 会报错，得写 `(char)65`。
- `String` 是字符串，用双引号，类型是 `std::lang::String` 的简写。它属于核心模块 `lang`，所以不用 import。字符串的内容是第 12 章的主题。

3.6 常量 const 与推导 auto

```
1  int64 main()
2  {
3      const int64 LIMIT = 100;
4      auto inferred = LIMIT;
5      println("{} {}", LIMIT, inferred);
6      return 0;
7  }
```

```
100 100
```

`const` 表示「这个值定下来就不改了」。给 const 变量再赋值是编译错误。数组长度、缓冲区大小这类东西应该写成 const——不是为了让编译器放心，是为了让下一个读代码的人放心。

`auto` 表示「类型你替我推」。什么时候用？当右边已经写得很清楚的时候：

```
1  auto info = describe(7);
2  // 想省一次手写类型
3  auto picked = range(1, 6).map(square).collect();
4  // 类型根本写不出来
```

第二种情况才是 `auto` 真正的用武之地——后面第 14、17 章会出现那种「类型长得没法看、写出来反而更难读」的场合。

但别滥用。函数内部的小变量用 `auto` 无所谓，函数签名、结构体字段、全局变量一律写全类型。一句话：类型是给别人看的，不是给编译器看的。

3.7 字面量怎么写

```
int64 main()
{
    println("{} {} {}", 0xFF, b1010, 3.14);
    println("{} {}", 2.5e3, 2.5e - 3);
    println("{}", 1e9);
    return 0;
}
```

```
255 10 3.14
2.5e+03 0.0025
1e+09
```

写法	含义
42	十进制整数
0xFF	十六进制（前缀 0x）
b1010	二进制（前缀 b，不是 0b）
3.14	浮点，默认 float64
2.5e3	科学计数法：2500.0
1e9	科学计数法：10^9（没有小数点也算浮点，只要有指数）
'M'	字符（单引号）
"\{}n"	字符串（双引号），里面有转义

> **提示**
>
> 二进制的前缀是 b 而不是 0b：b1010 是 10，写 0b1010 会报词法错误。这条容易记混，见本章最后的表。
>
> 带指数的就是浮点。1e9 和 2.5e3 一样是 float64，所以 int64 x = 1e9; 会报「不能隐式转换：float64 -> int64」——浮点往整数里放要签字（见 3.9 节）。

3.7.1 数字后缀：1u 与 1.0f

小写 u 表示无符号整数；没有类型上下文时默认是 uint64。无符号目标类型可以按自己的位宽接收这个字面量，并检查取值范围。它不会隐式变成有符号整数，也不能对无符号字面量直接写一元负号。

小写 f 把浮点字面量定为 float32。不带后缀的 1.0 是 float64；1f、1.0f、1e2f 都是 float32 写法。后缀必须小写。

```
int64 floatingKind(float32 value)
{
    return 32;
}

int64 floatingKind(float64 value)
{
    return 64;
}

uint64 unsignedValue(uint64 value)
{
    return value;
}

int64 main()
{
    auto one = 1u;
    auto single = 1.0f;
    uint8 narrow = 255u;
    uint64 maximum = 18446744073709551615u;
    println("{} {} {} {}", unsignedValue(one), floatingKind(single),
            floatingKind(1.0), narrow);
    println("{}", maximum);
    return 0;
}
```

```
1 32 64 255
18446744073709551615
```

`uint8 narrow = 256u;` 会因为超范围报错。`int64 signedValue = 1u;` 不能隐式转换，`-1u` 也会被拒绝；需要有符号负数时直接写 `-1`。`0xffu`、`b_101u` 同样支持无符号后缀。十六进制和现有二进制写法支持数字间的下划线分隔，但十进制 `1_000` 和 `0b1010` 并不是当前文法支持的写法。

3.8 隐式转换：只放行「变宽」

这是本章最重要的一节。

先说结论，别被别的语言带偏：Magel 只放行一类转换——「变宽」，也就是「目标装得下源、不会悄悄丢东西」的方向。反过来（变窄、有符号放进无符号、浮点放进整数、`char`/`bool` 跟整数互放）一律要你签字。所以准确的说法不是「没有隐式转换」，而是只放行「装得下」的那半边。

```
int64 main()
{
    int32 a = 100;
    int64 b = a;
    // 整型同符号变宽，放行
    uint8 u = 200;
    int32 m = u;
    // 无符号 -> 更宽的有符号，放行
    float64 c = b;
    // 整型 -> 浮点，放行
    println("{} {} {}", b, m, c);
    return 0;
}
```

```
100 200 100
```

放行清单（不用签字）：

方向	例子
同符号整型变宽	int8 → int16/int32/int64、int32 → int64、uint8 → uint32 ……
无符号 → 更宽的有符号	uint8 → int16/int32/int64、uint16 → int32、uint32 → int64
整型 → 浮点	int64 → float64、int32 → float32
浮点变宽	float32 → float64

下面这三类，方向是反的，必须自己签字：

```
int64 main()
{
    int64 big = 70000;
    int32 small = big;
    // 缩窄：位宽变小
    println("{}", small);
    return 0;
}
```

```
magel：不能隐式转换：int64 -> int32（请显式强转）
```

- 缩窄：位宽变小（int64 → int32、float64 → float32、uint8 → int8）——装不下，要签字；
- 有符号 → 无符号：哪怕目标更宽也不行（int32 → uint32、int64 → uint64）——负数塞不进无符号，要签字；
- 跨大类：浮点 → 整型、char / bool ↔ 整型——含义会变，要签字。

> **提示**
> 一句话记法：装得下的方向，Magel 自动帮你；装不下、或者换符号/大类，必须你亲自签字。

这么一管，麻烦是麻烦了点，但它买到的是三件事：

1. 每一处精度损失都是有意的。`int64` 里装着 90 亿，你要往 `int32` 里塞——这一下你必然写了 `(int32)`，将来出了问题，你能顺着这个强转找到是谁决定的。
2. 重载能被准确选中。第 2 章那条「找不到匹配的重载 `(int64)`」就是这套规则的副产品：因为不替你「凑合着转」，编译器只能说「你要 `uint64`，给我的是 `int64`」——它把真相告诉你了，而不是塞进某个它认为差不多的重载。
3. 跨平台的结果是确定的。在 32 位板子上 `long` 是 4 字节、PC 上是 8 字节，这种「同一个表达式换台机器结果不同」的坑，Magel 用「类型必须对上」堵掉了。

什么时候你会想骂它？写累加、写混合计算的时候。这时候记住两句话：

- 拿不准就用 `int64` / `float64`，让变量类型统一，转换就少了；
- 真要转，把它写在最显眼的地方（用括号包住，别藏在表达式中间）。

3.9 显式强转：(目标类型)值

```
int64 main()
{
    int64 big = 70000;
    int32 normal =(int32)big;
    uint8 byte =(uint8)300;
    char letter =(char)65;
    println("{} {} {}", normal, byte, letter);
    return 0;
}
```

```
70000 44 A
```

写法就是 C 的老规矩：在值前面用括号写上目标类型。

三行分别演示了三件事：

- `(int32)big` ——缩窄。70000 装得进 `int32`，值不变。
- `(uint8)300` ——截断。300 装不进 8 位，只留下低 8 位，得到 44。强转不会帮你检查装得下装不下——签了字就由你负责。
- `(char)65` ——数字变字符，65 就是 `'A'`。

> **提示**
> 强转是「你签字、编译器照做」，不做范围检查。所以 `(int8)200` 得到 `-56`、`(uint8)300` 得到 44，都不报错。会做范围检查的是字面量直接落进槽位那条路（3.10 节）——两条路泾渭分明：没写强转，编译器替你把关；写了强转，责任在你。

反过来 (int64)3.9 得到 1：浮点转整数是直接砍掉小数位，不是四舍五入。要四舍五入得自己想办法（std::math 里有取整函数，第 12 章提）。

3.9.1 cast<T>(value)：把目标类型写清楚

除了前面的 C 风格强转，也可以写 cast<目标类型 >(值)。这是显式转换，不是允许普通赋值偷偷缩窄。

```
int64 main()
{
    int64 original = 42;
    int32 narrowed = cast<int32>(original);
    println("{}", narrowed);
    return 0;
}
```

```
42
```

尖括号中是目标类型，圆括号中是被转换的表达式。自定义类型的显式转换要提供对应协议；宿主类型的 impl 绑定见第 21 章。转换也不会让一个引用越过来源的生命周期。

3.10 字面量的取值范围

这一节两条规矩，都是本书撰写时实测确认过的：

规矩一：负数字面量可以直接写进窄类型。

-300 在语法上是「一元负号 + 正数字面量 300」，但编译器把它当常量看值——只要值落在目标类型的范围内，就直接放行：

```
int64 main()
{
    int16 small = -300;
    // -300 落在 int16（-32768..32767）里
    int8 tiny = -100;
    println("{} {}", small, tiny);
    return 0;
}
```

```
-300 -100
```

规矩二：十进制字面量超出范围会报错，不静默回绕。

```
int64 main()
{
    int8 overflow = 200;
    // int8 只到 127
    return 0;
}
```

```
magel: 整数字面量 200 超出 int8 的取值范围（-128..127）
```

`uint8 b = 255;` 没问题（255 在 0..255 里），`uint8 b = 256;` 报同样的错。这条只管「字面量落进槽位」（初始化 / 赋值）；显式强转是另一回事——`(int8)200` 是你亲口签字，编译器只管按位截断（见 3.9 节）。

> **提示**
>
> 十六进制 / 二进制字面量按「位模式」处理，不做范围检查。`int8 x = 0xFF;` 得到 `-1`（低 8 位全 1，按有符号读就是 -1），和 C 一样。要写明确的位模式，就用 `0x` / `b` 前缀。

还有几个词法上的「不认」，撞上了照右列改：

你可能想写	实际会怎样	改成
`0b1010`	词法错误（前缀是 `b`，不是 `0b`）	`b1010`
`1_000`	语法错误（下划线分隔不认）	`1000`
`1.0F` / `1U`	后缀必须小写	`1.0f` / `1u`

3.11 小结

你会了	关键点
声明	`int64 score = 30;`，类型在前、名字在后、分号结尾
整型	`int8/16/32/64` + `uint8/16/32/64`，日常用 `int64`
浮点	无后缀为 `float64`，`1.0f` 为 `float32`，也可显式转换
其它标量	`bool` 只有 `true/false`（打印成 1/0）、`char` 用单引号
字符串	`String` 用双引号，属核心模块 `lang`，不用 import
常量	`const int64 LIMIT = 100;`
推导	`auto`，适合右边已经把类型写清楚的场合
字面量	`0xFF`、`b1010`、`3.14`、`2.5e3`、`1e9`、`'M'`、`"\{}n"`
转换	只放行装得下的方向（同符号整型变宽、无符号 → 更宽的有符号、整 → 浮、`float32→float64`）；其余要显式强转 `(int64)x`
字面量范围	十进制字面量超范围直接报错（`int8 x = 200;` 不合法）；`0x/b` 字面量按位模式放行

一句话总结这一章：Magel 要把「类型在哪里变了」留成一件看得见的事。

3.12 下一步

现在你会声明变量、会算数，但所有代码都堆在 `main` 里。第 2 章那个猜数字程序要是再长一点，`main` 就没法看了。

下一章解决这个问题：把代码切成函数。

翻到第 4 章函数。

第 4 章 函数

第 2 章的猜数字程序全塞在 main 里。如果它再长三倍——比如加上「记录每次猜测」、「算平均猜几次」——main 就再也读不动了。

函数就是解决这个的：给一段代码起个名字，需要的时候叫它。

这一章讲清楚五件事：怎么定义、怎么传参、怎么返回、怎么重载、以及怎么一次返回好几个值。

4.1 定义与调用

```
int64 add(int64 a, int64 b)
{
    return a + b;
}

void log(String message)
{
    println("[log] {}", message);
}

int64 main()
{
    log("start");
    println("{}", add(1, 2));
    return 0;
}
```

```
[log] start
3
```

拆开看一个函数的脑袋：

```
int64        add      ( int64 a , int64 b )   { ... }
 ↑            ↑          ↑
返回类型      名字       参数表：每个参数都要写类型
```

void 表示「不返回东西」。log 只做事、不交值，所以返回类型是 void。void 函数里不用写 return；想中途退出，写一个光秃秃的 return; 就行：

```
void trace(int64 number)
{
    if (number < 0)
    {
        return;
        // 负数不看，直接走
    }
    println("n={}", number);
}
```

函数可以写在 main 前面或后面，Magel 不要求「先声明后使用」。这一点比 C 舒服——不用再维护一份头文件里的原型。

函数的顺序也有讲究，但不是给编译器的，是给读代码的人的：把「先读的放前面」。一般先放小工具，再放用它的人，最后才是 main。

4.2 参数是副本

这是本章最重要的一条规则，也是第 6 章那条大规则的小预告：

```
void shout(String text)
{
    text = text + "!";
}

int64 main()
{
    String word = "hi";
    shout(word);
    println("{}", word);
    return 0;
}
```

```
hi
```

shout 里改了 text，main 里的 word 一点没变。

从 C++ / Java / Python 过来的人最容易在这里栽跟头：那些语言里字符串、对象传进去往往是「共享的引用」，函数里一改外面就变了。Magel 不是——传进去的是一份完整的副本。

想真的改到外面那个，得显式要求传引用（T mut&），这是第 7 章的内容。现在你只要记住：函数的参数表就是「我复制这些值过来用」。

> **提示**
>
> 那大对象传进去岂不是很贵？是的，会很贵。这正是 Magel 让你自己决定的地方：不想复制就写 String const&——一次显式的选择，而不是编译器偷偷替你优化。第 7 章会给出这几条写法的完整对照。

4.3 返回值

```
int64 clamp(int64 value)
{
    if (value < 0)
    {
        return 0;
    }
    return value;
}
```

return 后面跟的值的类型，必须和函数声明的返回类型一致，不允许隐式转换（还记得第 3 章那条规矩吧）。返回 int64 的函数不能 return 3.5;。

一个函数可以有多个 return——只要每条路都返回了一样类型的值。上面那个 clamp 有两条出路，都返回 int64。

如果一条路忘了返回呢？编译期就会报错，不会让你跑到一半发现「这个函数返回了垃圾」。

4.3.1 返回一个带值的控制表达式

return 后面既可以写普通值，也可以写块、if、match 或 loop。它们先给出一个结果，函数再返回这个结果。

```
int64 fromBlock()
{
    return
    {
        int64 local = 3;
        break local;
    };
}

int64 choose(bool first)
{
    return if (first)
    {
        break 5;
    }
    else
    {
        break 6;
    };
}

```

```
int64 fromMatch(int64 item)
{
    return match (item)
    {
        0 => { break 7; }
        _ => { break 8; }
    };
}

int64 fromLoop()
{
    return loop
    {
        break 9;
    };
}

int64 main()
{
    println("{} {} {} {}", fromBlock(), choose(false), fromMatch(0), fromLoop());
    return 0;
}
```

```
3 6 7 9
```

这里的 `break value;` 给出当前值表达式的结果，不是返回函数。只有表达式完成后，外层 `return` 才返回函数。所有可能给出结果的分支必须具有完全相同的类型；不能让一支是 `int32`、另一支是 `int64`，再指望编译器自动凑成同一种类型。

在这个返回表达式内部，不能再嵌套当前函数的 `return`，也不能使用 `break #level(...)` 跨层跳转；会提前返回的 `?` 同样需要改成显式匹配。表达式里新定义的独立函数有自己的返回边界。只含一个结果的 `{ value }` 分支仍可沿用前面的简写。

内部拥有值变量通过 `break` 交出结果时会隐式移动，外部变量仍按值复制。移动先保存结果，再执行离开作用域的清理；不会把已经交出去的值再析构一次。引用不能因此逃出来源的生命周期。第 8 章的清理规则和第 14 章的泛型返回会继续解释这些边界。

4.4 重载：同名不同参

同一个名字，可以按实参类型挑不同的实现：

```
int64 add(int64 a, int64 b)
{
    return a + b;
}

float64 add(float64 a, float64 b)
{
    return a + b;
}

int64 main()
{
    println("{} {}", add(1, 2), add(1.5, 2.5));
    return 0;
}
```

```
3 4
```

选哪个是编译期定下来的，看实参类型：add(1, 2) 两个都是 int64，走第一份；add(1.5, 2.5) 两个都是 float64，走第二份。

这和第 3 章那条「没有隐式转换」是配套的：因为不做隐式转换，重载的挑选才是准的。如果 int64 能自动变 float64，add(1, 2.5) 到底该走哪一份就得靠一套复杂的「谁更优」规则去猜了。Magel 的选择是：不猜，让你把类型写对。

找不到匹配的重载时，报错会把它拿到了什么类型和有哪些候选都列出来。比如只定义了 int64 的 add，却拿一个 int64 加一个 float64 去调：

```
int64 add(int64 a, int64 b)
{
    return a + b;
}

int64 main()
{
    println("{}", add(1, 2.5));
    return 0;
}
```

```
magel: 调用 函数 add 找不到匹配的重载: (int64, float64); 没有参数类型能对上的重载（候选: add）
```

看到这种错，先看括号里它拿到了什么类型，再决定是改实参还是加一份重载。（顺便：2.5 是 float64，add(1, 2.5) 里的 1 也不会被「凑」成 float64——回想第 3 章：没有隐式转换。）

4.5 默认参数

参数可以带默认值，调用时从尾部省略：

```
int64 add(int64 a, int64 b = 10)
{
    return a + b;
}

int64 main()
{
    println("{} {}", add(2), add(2, 3));
    return 0;
}
```

```
12 5
```

两条规矩：

- 带默认值的参数必须排在后面并且连续——不能「第 1 个有默认值、第 2 个没有」：

```
magel: 函数 add 的默认参数后不能有必填参数
```

- 省略时只能从尾部往前省。`add(2)` 省掉了 `b`，没问题；`add()` 不行，因为 `a` 没有默认值：

```
magel: 调用 函数 add 找不到匹配的重载: (); 没有参数类型能对上的重载（候选: add）
```

默认值在每次省略该参数时重新求值。所以别在默认值里写「记录调用次数」这种带副作用的表达式——每次都会算一遍。

> **提示**
>
> 什么时候用默认值？当一个参数大多数调用都会传同一个值，且传别的值不影响语义。比如 `add(a, b = 10)` 里的 `b` ——它是「可选的微调」。反过来，一个「猜数字最大值」这种参数就别给默认值，因为给了等于偷偷替调用方做了决定。

4.6 变长参数

最后一项参数可以写成 `T...`，表示「这里可以给任意多个」：

```
int64 total(int64... items)
{
    int64 sum = 0;
    foreach (int64 item in items)
    {
        sum = sum + item;
    }
    return sum;
}

int64 main()
{
    println("{} {}", total(1, 2, 3), total(5));
    return 0;
}
```

```
6 5
```

items 在函数体里就是一个能遍历的容器。foreach (int64 item in items) 是「把 items 里的东西一个个拿出来，每轮叫它 item」——第 5 章细讲 foreach。

变长参数只能放在最后一项，而且一个函数只能有一处。

4.7 一次返回多个值

Magel 的返回值可以是一个元组（tuple）——把几个值打包起来：

```
(int64, String) describe(int64 id)
{
    return(id, "player");
}

int64 main()
{
    auto info = describe(7);
    println("{} {}", info.0, info.1);
    return 0;
}
```

```
7 player
```

- 返回类型写成 (int64, String)，就是「一个 int64 加一个 String」；
- return (id, "player"); 把两个值打包成元组；
- 取的时候用 .0 / .1 ——这是位置，不是名字，从 0 开始数。

return 后面的括号可以省，return id, "player"; 和上一种写法等价。

如果你不想用 .0 / .1，可以直接拆开接：

```
(int32, String) pair(int32 key, String name)
{
    return key, name;
}

int64 main()
{
    int32 key;
    String name;
    key, name = pair((int32)7, "player");
    println("{} {}", key, name);
    return 0;
}
```

```
7 player
```

这行 key, name = pair(...) 的意思是：把右边元组按位置拆给左边的每一个目标。左边必须是已经声明过的变量——声明新变量请写 auto (key, name) = pair(...);。

> 提示
> 元组还能用来交换两个变量，而且不用中间变量：x, y = (y, x); ——右边先整个求值一次，再逐个写回去。注意右边那个括号不能省，x, y = y, x; 会报「多目标解构赋值必须使用 = 和一个元组来源」。

4.8 入口 main 的两种形态

到现在为止你写的都是不带参数的版本：

```
int64 main()
{
    ...
}
```

要读命令行参数，换成另一种：

```
int64 main(List<String> arguments)
{
    println("收到 {} 个参数", arguments.len());
    foreach (String item in arguments)
    {
        println("  {}", item);
    }
    return 0;
}
```

```
收到 2 个参数
  Alice
  two words
```

只有这两种入口是合法的，别自己发明第三种。

传参的写法要注意：直接给文件时，文件后面的都算程序参数。

```
magel app.mgl Alice "two words"
```

但如果你把场景词 `run` 显式写出来，就得多一个 `--` 把「配置词」和「程序参数」隔开：

```
magel run app.mgl -- Alice "two words"
```

不写 `--` 的话，`Alice` 会被当成另一个源文件，报「一次只能编译一个源文件」。

4.9 小结

你会了	关键点
定义	`int64 add(int64 a, int64 b) { return a + b; }`
不返回值	返回类型写 `void`，里面可以 `return;` 提前退出
调用	位置实参，从左往右一一对应
参数语义	按值传入，形参是副本，改形参不影响调用方
重载	同名不同参，编译期按实参类型选，报错会列出候选
默认参数	`int64 b = 10`，只能从尾部连续省略
变长参数	`int64... items`，放最后一项，用 `foreach` 遍历
多值返回	返回类型写元组 `(A, B)`，用 `.0` / `.1` 取或拆开接
入口	`int64 main()` 或 `int64 main(std::lang::List<std::lang::String> arguments)`

还有两处本章故意没讲透：

- `static` 方法（写在类型上、用 `Type::method()` 调）和第 2 章那个 `Random::withSeed` ——那是结构体的东西，第 9 章讲；
- 把函数当值传递（`int64 fn(int64)` 类型、`fn`）——第 15 章讲。

4.10 下一步

现在你能把代码切成函数了，但函数里只能「从上往下一句句做」。真实程序还需要判断和重复——第 2 章你用过 `if` 和 `while`，但那是照着抄的。

下一章把控制流完整讲一遍，包括 `loop` 那个能「带值跳出来」的用法。

翻到第 5 章控制流。

第 5 章 控制流

程序光会「从上往下做」是干不了活的。它得会拐弯（条件）和转圈（循环）。

第 2 章的猜数字已经把这两种都用上了，这一章把它们补全：Magel 一共有四种循环，还有一套对「同一个变量在一个表达式里被读写两次」管得很严的规则。

5.1 条件：if / elif / else

```
String classify(int64 value)
{
    if (value < 0)
    {
        return "负数";
    }
    elif (value == 0)
    {
        return "零";
    }
    else
    {
        return "正数";
    }
}

int64 main()
{
    println("{} {} {}", classify(-3), classify(0), classify(8));
    return 0;
}
```

```
负数 零 正数
```

形态就是这些：

```
if (条件) { ... }
elif (条件) { ... }
else { ... }
```

三条要注意的：

1. 条件必须真的能当条件。

```
int64 main()
{
    if (1)
    {
        println("x");
    }
    return 0;
}
```

```
magel: if 条件必须是 bool，现在给的是 int64（数值条件请显式写成 `(bool)值` 或比较表达式）
```

没有「非零即真」。想判断「count 是不是 0」，写 `if (count != 0)`——把你想说的说出来。

2. 拼写是 `elif`，不是 `else if`，也不是 `elsif`。少一个字母都编译不过。

3. 花括号不能省。只写一句 `if (x) y = 1;` 是编译错误。这条和 C 不一样，但换来的是「这个 `y = 1` 到底属于哪个 if」永远不会有歧义。

> **提示**
>
> Magel 没有三目运算符。`cond ? a : b` 是语法错误（第 3 章提过）。简单替换就用 `if / else` 赋值；分支多了，第 10 章的 `match` 更好读。可以使用 `if（条件）{ 值一 } else { 值二 }` 条件表达式；只求值命中的分支。

5.1.1 条件也能产生值

`if` 可以出现在变量定义、赋值、函数实参和其他表达式里。只执行选中的分支，条件和分支都按短路规则求值。

```
int64 main()
{
    int32 p = 5;
    bool odd = if (p % 2 == 1)
    {
        true
    }
    else
    {
        false
    };
    int32 category = if (p % 3 == 1)
    {
        1
    }
    elif (p % 3 == 2)
    {
        2
    }
    else
    {
        3
    };
    odd = if (p % 2 == 0)
    {
        false
    }
    else
    {
        bool value = true;
        break value;
    };
    println("{} {}", odd, category);
    return 0;
}
```

```
1 2
```

只有结果表达式时写 `{ value }`，块内不加分号；有声明、赋值等语句时用 `break value;` 给出块的结果。各分支结果类型一致，并且必须有 `else`。外层变量定义和赋值仍以分号结束。

条件选择使用 `if` 表达式。

5.2 while：先判断，再执行

```
int64 main()
{
    int64 countdown = 3;
    while (countdown > 0)
    {
        println("{}", countdown);
        countdown = countdown - 1;
    }
    println("起飞！");
    return 0;
}
```

```
3
2
1
起飞！
```

条件写在括号里，先判后做。条件一开始就不成立，循环体一次都不执行。

5.3 for：三段式

```
int64 main()
{
    int64 total = 0;
    for (int64 k = 0; k < 5; k = k + 1)
    {
        total = total + k;
    }
    println("{}", total);
    return 0;
}
```

```
10
```

for 的括号里是三段，用分号隔开：

```
for ( 初始化 ; 条件 ; 每轮结束做什么 )
       ↑         ↑         ↑
     int64 k=0   k<5       k=k+1
```

顺序是：先跑一次初始化 → 判条件 → 执行循环体 → 跑「每轮结束做什么」→ 再判条件……

for 里声明的变量（这里是 k）只在这个循环里存在，出了循环就没了。所以循环里可以放心用 i、k 这种短名字。

遍历容器时不要用 for。数下标容易数错，foreach 是专门干这个的（5.6 节）。

5.4 do { } while ( )：先执行一次

```
int64 main()
{
    int64 tries = 0;
    do
    {
        tries = tries + 1;
    }
    while (tries < 3);
    println("{}", tries);
    return 0;
}
```

```
3
```

和 while 的唯一区别：循环体至少执行一次，条件在末尾判。

什么时候需要它？「至少得做一次，做完再看要不要继续」——比如读一个输入、至少问用户一次。注意 while (条件) 后面那个分号不能省。

5.5 loop：会「带值跳出来」的循环

```
int64 main()
{
    int64 attempts = 0;
    loop
    {
        attempts = attempts + 1;
        if (attempts >= 3)
        {
            break attempts * 10;
        }
    };
    println("{}", attempts);
    return 0;
}
```

```
3
```

loop { ... } 就是「一直转」，它自己不会停——出口全靠 break。

它的特别之处在于：break 后面可以带一个值，这个值就是整个 loop 的「结果」：

```
int64 firstEven = loop
{
    int64 candidate = nextNumber();
    if (candidate % 2 == 0)
    {
        break candidate;
        // 这个 candidate 就是 firstEven
    }
};
```

所以 loop 是表达式（能算出值），不是语句。这带来一个语法上的小尾巴：

当 loop 出现在语句位置时，后面要补一个分号。

```
loop
{
    done = work();
    if (done)
    {
        break;
    }
};
// ← 这个分号不能少
```

看到 `};` 别以为是笔误，那是「一个表达式被当成语句用」的正常写法。（`while`、`for`、`if` 是语句，不需要。）

5.6 foreach：遍历一串东西

```
int64 main()
{
    List<int64> scores;
    scores.push(30);
    scores.push(55);
    scores.push(41);

    int64 best = 0;
    foreach (int64 score in scores)
    {
        if (score > best)
        {
            best = score;
        }
    }
    println("最高分 {}", best);
    return 0;
}
```

```
最高分 55
```

写法：

```
foreach ( 元素类型 名字 in 容器 ) { ... }
```

`List<int64>` 是一个「有序的整数表」，`push` 往末尾加一个。容器是第 11 章的主题，现在你把它当成「一串东西」就够了。

三个要点：

- 元素类型可以显式写成 `foreach (int64 score in scores)`，也可以由迭代源推导成 `foreach (score in scores)`。
- 遍历出来的 `score` 是元素的副本（还记得第 4 章吗），改它不影响容器。想改容器里的元素得用引用（第 7 章）。
- `foreach` 能遍历的不只是容器，凡是「能一个个吐出来的东西」都行，比如第 4 章的变长参数、第 17 章的迭代器。

5.7 break 与 continue

```
int64 main()
{
    List<int64> scores;
    scores.push(12);
    scores.push(-5);
    scores.push(0);
    scores.push(99);

    foreach (int64 score in scores)
    {
        if (score < 0)
        {
            continue;
            // 负数不算，看下一个
        }
        if (score == 0)
        {
            break;
            // 到此为止，后面的都别看了
        }
        println("分数 {}", score);
    }
    return 0;
}
```

```
分数 12
```

（0 那一项触发了 `break`，所以 99 从没被打印。）

`continue` = 「这一轮到此为止，直接进下一轮」；`break` = 「整个循环到此为止」。

两个都只作用于最内层那一圈循环。想从两层里一次跳出来，得靠 `loop` 带值或者加个标志变量——Magel 没有「带标签的 break」。

5.7.1 指定退出哪一层循环

break; 退出最近一层循环。break #level(2); 退出两层，break #level(ALL); 退出当前函数里所有包围它的循环。ALL 参数不区分大小写，all、All 等写法相同；#level 自身仍按小写拼写。

```
int64 main()
{
    int64 visited = 0;
    loop
    {
        loop
        {
            visited++;
            break #level(all);
        };
        visited += 100;
    };

    int64 total = 0;
    List<int64> values;
    values.push(2);
    values.push(4);
    values.push(6);
    foreach (item in values)
    {
        total += item;
    }
    println("{} {}", visited, total);
    return 0;
}
```

```
1 12
```

层数只计算 while、do、for、foreach 和 loop，普通块及 if、match 不增加层数。数字参数必须是正整数字面量，不能用变量代替层数。ALL 是指令参数中的特殊写法，普通变量仍可命名为 all。

层级退出不能携带结果，也不能跳出另一个函数的循环；离开路径上的清理照常执行。带值的 break value; 则交给最近接收结果的值表达式，无值循环不会把这个值截走。返回表达式中禁止 break #level(...)，详见第 4 章。

5.8 复合赋值与自增减

写累加的时候，total = total + score; 说三遍就烦了。可以缩短：

```
int64 main()
{
    int64 value = 17;
    value += 3;
    // value = value + 3
    value -= 1;
    value *= 2;
    println("{}", value);
    return 0;
}
```

```
38
```

`+= -= *= /= %= &= |= ^= <<= >>=` 全都有，含义就是「把右边算出来，跟左边做这个运算，写回左边」。

单独的自增自减也能写：

```
int64 main()
{
    int64 i = 5;
    i++;
    println("{}", i);
    i--;
    ++i;
    println("{}", i);
    return 0;
}
```

```
6
6
```

5.8.1 但 Magel 对「一个表达式里读写同一个对象」管得很严

这是 Magel 和 C 差异最大的地方之一。同一个完整表达式里，一个自增减的目标不能再被别的操作读写：

```
int64 main()
{
    int64 i = 0;
    i = i++;
    return 0;
}
```

```
magel: [MGL1301] 同一表达式中，自增减目标与其他读写冲突：i
```

C 程序员会心头一紧——`i = i++` 在 C 里是未定义行为（每个编译器给的结果可能不一样）。Magel 的做法是：干脆不让你写。你可能觉得它啰嗦，但它拦下的是一整类「换个编译器就换结果」的玄学 bug。

那什么情况是合法的？目标互不影响就行，比如两个不同的变量：

```
int64 main()
{
    int64 i = 0;
    int64 j = 0;
    int64 sum =(i++) +(j++);
    println("{} {} {}", sum, i, j);
    return 0;
}
```

```
0 1 1
```

还有一条顺便记住：下标自增只发生一次。`xs[i] += 1` 里，`xs[i]` 这个「位置」只算一次，不会因为复合赋值就把 `i` 加两次。

```
int64 main()
{
    List<int64> store;
    store.push(10);
    store.push(20);
    int64 i = 0;
    store[i] += 1;
    println("{} {}", store[0], i);
    return 0;
}
```

```
11 0
```

5.9 小结

结构	什么时候用	备注
`if / elif / else`	条件分支	条件必须是 `bool`；花括号不能省
`while (cond) { }`	先判断后执行	可能一次都不执行
`for (init; cond; step) { }`	已知次数的计数循环	循环变量只活在循环里
`do { } while (cond);`	至少执行一次	末尾分号不能省
`loop { ... break v; }`	出口不确定、要给个结果	是表达式，语句位置要写 `};`
`foreach (T x in c) { }`	遍历一串东西	类型可显式写出或推导，遍历出来是副本

结构	什么时候用	备注
`break` / `continue`	跳出 / 下一个	普通无值形式作用于最近循环；`#level` 可指定退出层数

还有一套要记住的规则：同一个表达式里对同一个目标读写两次是被禁止的（`MGL1301`），这是 Magel 用来消灭「未定义行为」的手段。

三件本章没讲的事：

- `foreach` 遍历出来的元素是副本，怎么改到原来的——第 7 章；
- 分支多了 `if / elif` 会很难看，`match` 怎么救场——第 10 章；
- `loop` 里 `break` 出来的值，除了赋值还能怎么用——第 10 章的 `match`、第 16 章的 `?` 会陆续出现。

5.10 下一步

到这里，「语言基础」这一部分就结束了。你现在会写变量、函数、判断、循环——用这些已经能写出完整的程序。

但会写程序 ≠ 会写对的程序。接下来三章是这本书最重要的一段：【第 6 章值语义】【第 7 章引用】【第 8 章资源与作用域】。

读懂了这三章，你会明白 Magel 为什么长成现在这样——以及为什么它愿意让你多写几个字。

翻到第 6 章值语义。

第 6 章 值语义

从这一章开始，我们进入 Magel 与别的语言真正分岔的地方。

前面五章你学的东西，在 C、Java、Go、Rust 里大都能找到对应的写法和直觉。但这一章讲的那条规则，会让你需要改掉一些习惯。

先不改，先看一个现象。

6.1 一个让人愣一下的例子

```
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Point a;
    a.x = 1;
    a.y = 2;

    Point b = a;
    // 把 a 赋给 b
    b.x = 99;
    // 改 b

    println("a=({}, {}) b=({}, {})", a.x, a.y, b.x, b.y);
    return 0;
}
```

```
a=(1, 2) b=(99, 2)
```

改了 b，a 没变。

如果你从 Java、Python、JavaScript 或 C# 过来，你刚才很可能在等 a 也跟着变成 99——那些语言里 b = a 之后，两个名字指的是同一块数据。

如果你从 Rust 过来，你等的是编译器报错——那些语言里 let b = a; 之后 a 就废了。

Magel 两个都不是。它给的是第三种：

> 提示
>
> Point b = a; 把 a 完整地复制了一份给 b。两个变量从此各过各的。

这就是值语义。

6.2 复制是整套的

这不是「只复制了数字、里面的东西还是共享的」。是整套复制：

```
struct Bag
{
    List<int64> items;
};

int64 main()
{
    Bag original;
    original.items.push(1);

    Bag copy = original;
    // 整只袋子复制一份，连同里面的表
    copy.items.push(2);

    println("original={} copy={}", original.items.len(), copy.items.len());
    return 0;
}
```

```
original=1 copy=2
```

`copy.items.push(2)` 只动了 `copy` 自己那份。`original` 里的表还是只有 1 个元素。

字符串也一样：

```
int64 main()
{
    String original = "hello";
    String copy = original;

    copy[0] =(char)72;
    // 72 就是 'H'

    println("{} {}", original, copy);
    return 0;
}
```

```
hello Hello
```

改 `copy` 的第一个字符，`original` 安然无恙。

这里有个重要的推论：结构体的字段、容器的元素、嵌套了好几层的值……赋值时全都跟着复制。没有「浅复制 vs 深复制」的选择题，只有一种复制：深到底。

6.3 函数参数也是复制

第 4 章你已经见过这条规则，现在你知道它从哪来了：

```
struct Point
{
    int64 x;
    int64 y;
};

void shift(Point point)
{
    point.x = point.x + 10;
}

int64 main()
{
    Point a;
    a.x = 1;
    a.y = 2;

    shift(a);
    println("a=({}, {})", a.x, a.y);
    return 0;
}
```

```
a=(1, 2)
```

`shift` 里改了 `point.x`，调用方的 `a` 一动不动。因为 `point` 是一份副本。

函数改不到外面——这是 Magel 的默认行为，不是限制。想改到外面，是第 7 章要学的显式引用（`Point mut&`）。

6.4 和 Rust 的关键区别：没有「移动」

Rust 里 `let b = a;` 之后 `a` 就不能用了——值被移动走了。Magel 没有这回事：

普通赋值、按值传参、`return` 返回值，都不会销毁源值。`a` 照常可用。

那「我要把值交给别人、自己不要了」怎么办？Magel 给了你一个显式的操作：`move`（6.6 节）。要点是——它是你主动写的，不是编译器默默做的。

这个区别看起来小，但它决定了你读代码时的心智负担：

	Rust	Magel
看到 `let b = a;`	得在脑子里算「a 还能用吗」	不用算，a 就是还能用
想省一次复制	靠移动语义自然发生	写 `move(&a)`，一眼能看出来
读代码时	要跟着所有权走	赋值就是复制，没有隐藏状态

6.5 代价：复制是要花钱的

现在讲实话。值语义不是白拿的。

每次赋值、每次传参、每次返回，都在复制。一个装着一万条记录的容器、一个几兆的缓冲区，按值传来传去就是真的在搬内存。

所以什么时候要小心？

- 大对象当参数：函数要读一个几 MB 的结构体，别按值传。
- 循环里赋值：`foreach` 里每轮都在复制元素（小元素无所谓，大结构体就很痛）。
- 深层嵌套的结构体：复制是递归到底的，层数越多越贵。

那怎么避免？答案是第 7 章：

```
void inspect(BigData const& data)
{
    ...
}
// 只读，不复制
void update(BigData mut& data)
{
    ...
}
// 要改，不复制
```

注意这是你主动的选择，不是编译器偷偷帮你优化掉复制。这一点很重要：

- 你写了 `const&`，说明「我知道这里不复制」；
- 你写了按值，说明「我接受这次复制，我需要一份独立的副本」。

编译器不会在你背后做决定，所以性能上发生了什么，读代码就能看出来。

6.6 显式转移所有权：`move`

有时候你确实想要「把原有资源交给另一个值，原来的值不再使用」。Magel 把它写成一个函数：

```
struct Message
{
    String text;
};

int64 main()
{
    Message source;
    source.text = "hello";

    Message result = move(&source);
    println("{}", result.text);
    return 0;
}
```

```
hello
```

move(&source) 做的事情，按顺序是：

1. 把 source 原有的资源句柄交给结果，不深拷贝资源；
2. 清空源存储并结束源值生命周期，不执行它的 close / drop；
3. 接收方取得所有权，最终由接收方执行清理。

三条规矩：

- 参数必须写 &source，也就是传一个 T mut&。传普通值或者只读引用会编译报错。
- 移动之后不能再读 source：

```
struct Message
{
  String text;
};

int64 main()
{
  Message source;
  source.text = "hello";
  Message result = move(&source);
  println("[{}] [{}]", source.text, result.text);
  return 0;
}
```

```
magel: [MGL1205] 借用来源已移动或销毁，不能继续访问（源对象已经销毁）：text
```

要再用这个名字，须重新声明（如 Type source = something;）建立新的绑定；旧绑定禁止读写。

- move 可以省去资源的深拷贝：接收方取得原有字符串和容器的句柄，源值不能再读。需要继续使用源值时保留普通复制，临时只读访问则使用 const&。

> 提示
>
> 什么时候真的需要 move？把一份数据「转移到别的地方」的时候，比如从缓冲区里取出结果、把临时结构体的内容装进长期对象。那时候用 move 说清楚「源不要了」，比留在那里等它被析构更明确。

6.7 为什么值得这么设计

回过头看，Magel 为这条规则付出了什么、又买到了什么。

付出的：多一些复制，大对象要自己记得写 const&。

买到的：

1. 没有别名共享，也就没有「谁改了它」。一个函数拿到的是副本，它不可能在你不知情的时候改掉你的数据。调试「这个值怎么变了」的时间大幅减少。
2. 不需要所有权标注、不需要生命周期参数。复制出来的值跟源没有任何关系，所以不存在「它活得比源久」的问题——第 7 章你会看到，引用才需要关心这个。
3. 容器、返回值、参数传递的行为完全一致。一套规则走遍全语言，没有「这里共享那里复制」的记忆负担。
4. 确定性释放变得自然。每个值有明确的归属（哪个变量、哪一层作用域），离开作用域就能干净地清掉——这是第 8 章。

一句话：Magel 用「多一点复制」换「少一点你可能想不通的事」。

6.8 小结

你会了	关键点
赋值	b = a 复制整个值，之后 a、b 互不影响
复制范围	深到底：字段、容器元素、字符串，全都复制
传参	形参是副本，函数改不到调用方
返回	返回值也是副本，源值照常可用
与 Rust	没有隐式移动，普通赋值不动源值
与 Java/Python	没有共享，b = a 不是「两个名字一块数据」
省复制的办法	显式写引用 const& / mut&（第 7 章）
显式转移	move(&x) = 转移资源所有权；旧绑定禁止读写，二次声明建立新绑定（MGL1205）

6.9 下一步

拥有值默认复制，显式 move 转移；借用检查器继续跟踪引用来源、移动与释放，用户无需书写生命周期参数。但有一件事它没解决：如果要改到外面那个对象怎么办？

一份副本改不了原对象，那函数怎么修改调用方的数据？容器怎么原地排序？大结构体怎么避免复制？

答案都指向同一个东西：引用。下一章讲它——包括 Magel 引用最特别的一条规矩：它不能活得比它指的东西久。

翻到第 7 章引用。

第 7 章 引用

第 6 章那条规则（赋值就是复制）解决了很多麻烦，但留下了两个：

1. 函数改不到外面。一份副本改不了原对象，那「就地排序」「累加计数」怎么写？
2. 大对象复制很贵。几 MB 的结构体按值传一次，就是在搬内存。

答案都是同一个东西：引用。

这一章讲它，包括 Magel 引用最特别的一条规矩——引用不能活得比它指的东西久。

7.1 引用长什么样

三种写法，先认脸：

```
T mut&      可变引用：能读，也能写
T const&    只读引用：只能读
T &         和 T const& 完全等价（简写）
```

实际用起来是这样：

```
int64 main()
{
    int64 value = 7;
    int64 mut& alias = &value;
    // 造一个「指向 value」的可变引用
    alias = 9;
    // 写引用 = 写 value
    println("{} {}", value, alias);
    return 0;
}
```

```
9 9
```

三个要点：

`&value` 是「取地址」。它造出一个指向 `value` 的引用。`&` 这个符号表示「我不是要它的值，我要它本身」。

写下引用就是写原对象。`alias = 9;` 之后 `value` 也变成 9 了。从这一刻起，`value` 和 `alias` 是指向同一份数据的两个名字——这是本章 7.8 节要讲的代价。

只读引用不能写。

```
int64 main()
{
    int64 value = 7;
    int64 const& view = &value;
    view = 9;
    return 0;
}
```

```
magel: 只读引用（`T const&`）不能写入元素
```

T & 是 T const& 的简写，写起来省一点。改不了的东西用 const& 声明，本身就是一种文档：读代码的人一眼知道「这个函数不会动我的数据」。

7.2 用引用改到外面

第 6 章那个改不动调用方的 bump，加一个 mut& 就改得动了：

```
void bump(int64 mut& counter)
{
    counter = counter + 1;
}

int64 main()
{
    int64 number = 5;
    bump(&number);
    println("{}", number);
    return 0;
}
```

```
6
```

两处变化：

- 形参从 int64 counter 变成 int64 mut& counter（要改，所以是 mut&）；
- 调用时写 bump(&number) ——多了一个 &。

那个 & 很关键，它是「签字」：我主动允许这个函数改我的 number。看到 bump(&number) 的人，不用跳进 bump 就知道 number 可能被改。

> **提示**
>
> 这也是 Magel 和「默认就是引用传递」的语言最大的差别：这里改到外面是显式的。一眼扫过去就能数出哪些调用会动数据。

7.3 用引用省掉复制

函数只是读一个对象，却不想复制它——用 const&：

```
struct Player
{
    String name;
    List<int64> scores;
};

int64 total(Player const& player)
{
    int64 sum = 0;
    foreach (int64 score in player.scores)
    {
        sum = sum + score;
    }
    return sum;
}

int64 main()
{
    Player p;
    p.name = "amy";
    p.scores.push(10);
    p.scores.push(20);
    println("{} 的总分 {}", p.name, total(&p));
    return 0;
}
```

```
amy 的总分 30
```

total(&p) 一次都没复制那个 Player。const& 说了两件事：

- 不复制：性能上你知道发生了什么；
- 不修改：调用方可以放心把东西交出去。

什么时候该用 const&？一个大致的门槛：

参数是	怎么写
标量（int64、bool、char）	直接按值——复制一个整数比传引用还快
小结构体（两三个字段）	按值就行，简单
字符串、容器、大结构体	const&（只读）或 mut&（要改）

记住一件事：这是你主动写的选择，编译器不会偷偷帮你优化掉复制。好处就是——性能上发生了什么，读代码就能看出来。

7.4 引用必须当场绑定：没有「空引用」

Magel 没有 `null`。没有 `nullptr`，没有 `nil`，没有「先声明一个空引用，以后再让它指向什么」：

```
magel: 引用声明必须初始化，缺省值请使用 Option<T mut&>
```

引用只能当场绑定到一个确实存在的东西：

```
int64 first = 1;
int64 mut& keep = &first;
// 声明的同时必须绑定
```

而且 `&` 只能取「有实体对象」的地址——变量、字段、数组元素、字符串的字节。表达式的结果取不了地址：

```
int64 main()
{
    int64 value = 1;
    int64 mut& bad = &(value + 1);
    return 0;
}
```

```
magel: `&` 只能取数据区里对象的地址（数组元素 / 结构体字段 / 字符串字节；栈上的标量变量取不了）
```

（这条报错的措辞有点绕，别被最后半句带偏：取变量的地址是没问题的，上面第一个例子里 `&value` 就是。它真正拒绝的是临时算出来的值——`value + 1` 这个结果没有存放的地方，自然也没有地址可指。）

这一条消灭了整类崩溃。空指针解引用这种东西在 Magel 里不存在，因为没有那个「空」的状态。

7.5 引用不能活得比目标久

这是 Magel 引用最重要的规矩，也是它最容易被拒的地方。

想清楚这件事：一个引用只是「指向某处的一个地址」。如果它指的那个对象已经没了，它还留着，那用它的那一刻就会读到一块不属于你的内存——这就是悬垂引用。

Magel 的处理方式是：编译期跟踪每个引用的来源，来源活得不够久就拒绝编译。

```
int64 mut& bad()
{
    int64 local = 1;
    return &local;
    // local 在函数返回时就没了
}
```

```
magel: [MGL1204] 返回值中的引用不能超过局部来源的生命周期（int64 mut&，所属类型 ，来源 local）
```

`local` 是函数里的局部变量，函数一返回它就消失了，可你还要把指向它的引用交出去——编译期直接拒绝。

包几层也逃不掉。把引用装进结构体再返回，一样拦得住：

```
struct Holder
{
    int64 mut& slot;
};

Holder make()
{
    int64 local = 5;
    Holder holder;
    holder.slot = &local;
    return holder;
}

int64 main()
{
    Holder h = make();
    println("{}", h.slot);
    return 0;
}
```

```
magel: [MGL1204] 返回值中的引用不能超过局部来源的生命周期（Holder，所属类型 ，来源 holder）
```

这条规则覆盖的范围（记住大意就行）：引用别名、显式权限转换、Option / Result 里的载荷、元组、容器、结构体字段——包多少层都沿来源传播。

7.5.1 那什么时候是合法的？

来源比引用活得久就行。最常见的合法情形是「来源是参数」：

```
int64 mut& larger(int64 mut& a, int64 mut& b)
{
    if (a >= b)
    {
        return a;
    }
    return b;
}

int64 main()
{
    int64 x = 3;
    int64 y = 8;
    larger(&x,&y) = 99;
    // 把大的那个改成 99
    println("{} {}", x, y);
    return 0;
}
```

```
3 99
```

这里 a、b 指向的对象是调用方的，调用方活得比 larger 的返回值久，所以合法。

注意最后一行 larger(&x, &y) = 99;——返回引用时，函数调用本身是个可写位置，可以直接给它赋值。这是「返回引用」最实用的地方。

7.6 借用检查与别名共享

Magel 有独立的借用检查器。它先追踪引用来源是否活得足够久，再在控制流中检查移动、析构和释放后的访问。分支合流、循环、字段和能够识别的别名都会参与分析。

检查内容	规则
来源寿命	引用不能超出来源，沿聚合传播；常见诊断 MGL1204
move 后访问	旧绑定禁止读写或重绑；完整移动后可以二次声明同名变量
drop/delete 后访问	已终结来源不能继续当有效值使用；常见诊断 MGL1205
多个别名	可以指向同一对象，通过一个修改，其他有效别名可以观察到
检查边界	不实行全局独占借用规则，不覆盖任意原始宿主内存行为

这要求同时考虑对象的作用域和生命周期状态。仅把来源移到外层，并不能使已经移动或释放的值重新有效；重新声明的同名变量也不会让旧引用改指向新变量。

复制拥有值可以获得独立数据；复制引用仍共享来源。容器和大对象的深复制有成本，需要省复制时显式选择有效引用。详细边界见借用控制流验证。

7.7 小结

你会了	关键点
三种引用	`T mut&`（可写）、`T const&`（只读）、`T &`（≡ `const&`）
取地址	`&value`；写引用就是写原对象
传引用	形参加 `mut&` / `const&`，调用时写 `&x`
省复制	大对象（字符串、容器、大结构体）用 `const&` 传
非空	引用必须当场绑定，没有 `null`，没有空引用
取址边界	`&` 只对「有实体对象」的东西有效，临时值不行
生命周期	引用不能活得比来源久，违反是 `MGL1204`
合法返回引用	来源是参数、是更外层作用域的对象时可以
别名	允许两个引用指向同一份数据，改一个另一个也变
借用检查器	来源寿命与控制流中的移动、析构和释放检查；允许别名共享

拥有值默认复制，引用显式共享，借用检查跟踪来源和生命周期状态。宿主内存、并发与无法静态建模的行为仍有各自边界。

7.8 下一步

引用解决了「改到外面」和「省复制」两个问题。但还有一个问题没碰：

没有垃圾回收，那些资源（内存、文件句柄、连接）什么时候释放？

已知的答案你已经猜到了：离开作用域的时候。但「离开作用域」这套机制怎么写的、怎么保证不出错、`Drop` 和 `Closeable` 有什么区别、堆对象又该怎么管——这些是下一章。

翻到第 8 章资源与作用域。

第 8 章 资源与作用域

前面两章讲了「值」和「引用」，但一直没回答一个很现实的问题：

> **提示**
> 没有垃圾回收，那内存、文件句柄、网络连接这些东西，什么时候释放？谁负责？

Magel 的答案很短：离开作用域的时候，自动释放。

这一章讲这套机制怎么用、为什么它不出错，以及堆对象该怎么管。

8.1 作用域就是一对花括号

先说清「作用域」在这门语言里指什么。

每一对 `{ }` 划出一个作用域。在那个花括号里声明的东西，出了花括号就不存在了：

```
int64 main()
{
    int64 outer = 1;
    {
        int64 inner = 2;
        println("里面能看到 {} 和 {}", outer, inner);
    }
    println("外面只剩 {}", outer);
    return 0;
}
```

```
里面能看到 1 和 2
外面只剩 1
```

这里面 `inner` 只活在里层那对大括号里。出了它，`inner` 就没了——想在外面用 `inner` 是编译错误，不是「值是 0」。

作用域有好几种来源：

来源	例子
一对花括号	`{ int64 x = 1; }`
函数体	`int64 add(...) { ... }` ——整个函数是一个作用域
`if` / `for` / `while` 的主体	`for (int64 k = 0; ...) { }` 里的 `k` 只活在循环里
块	`main` 本身也是块

注意最后一条：`main` 也是个作用域。所以 `main` 里声明的东西，程序结束时会被清理。

8.2 Drop：离开作用域时自动调用

给一个结构体实现 Drop，它的 drop() 就会在这个值离开作用域时被调用：

```
struct Resource: Drop
{
    String name;
    void drop()
    {
        println("drop {}", self.name);
    }
};

int64 main()
{
    {
        Resource r;
        r.name = "conn";
        println("using");
    }
    println("after block");
    return 0;
}
```

```
using
drop conn
after block
```

注意 drop conn 出现的位置：在里层花括号结束的那一刻，而不是在程序末尾。这就是「确定性释放」——资源什么时候还，看代码结构就能读出来。

这类拥有型资源由 Drop 自动收尾，不需要额外登记释放动作。结构就是释放点。另有非所有权清理动作时可用本章的 defer。

drop() 里拿自己用 self。第 9 章会专门讲 self，现在当成「这个对象自己」就行。

8.3 析构顺序：先自己的 drop()，再做字段

字段会被自动递归析构——一层层往下，直到最里面：

```
struct Inner: Drop
{
    String tag;
    void drop()
    {
        println("drop inner {}", self.tag);
    }
};

struct Outer: Drop
{
    Inner field;
    void drop()
    {
        println("drop outer");
    }
};

int64 main()
{
    Outer o;
    o.field.tag = "x";
    println("using");
    return 0;
}
```

```
using
drop outer
drop inner x
```

顺序是：先跑 Outer 自己的 drop()，再析构它的字段。

> **提示**
>
> 如果你写过 C++，注意这里和 C++ 是反的：C++ 先析构成员再跑析构体。Magel 先跑你自己的 drop()，之后字段自动清理。
>
> 这个顺序有个实际后果：drop() 里还能安全地用字段（o.field 那时还活着）。这在「关闭连接时要写日志」「释放前要刷缓冲」这类场景里很有用。

8.3.1 一个必须记住的禁忌

结构体的 drop() 里不要手写 self.字段.drop()。

因为字段本来就会被自动析构，你再手动来一次就是析构两次。这条规则没有例外：只要字段实现了 Drop，Magel 就会负责到底，你插手就是添乱。

8.4 Closeable：用完立刻关，而且不会关两次

有些资源不能等到「出块」才放：文件句柄、数据库连接、锁。它们要的是「用完立刻放，别占着」。

实现 Closeable 就是干这个的：

```
struct Handle: Closeable
{
    int64 id;
    void close()
    {
        println("close {}", self.id);
    }
};

int64 main()
{
    {
        Handle h;
        h.id = 1;
        println("using");
    }
    println("after block");
    return 0;
}
```

```
using
close 1
after block
```

机制和 Drop 一样（离开作用域自动调 close()），但多了一条很实用的保证：

> **提示**
>
> 显式调用过 close() 的变量，不会再自动关闭。

所以你可以放心地「提前关」：

```
struct Handle: Closeable
{
    void close()
    {
        println("close");
    }
};

int64 main()
{
    {
        Handle h;
        h.close();
        // 提前关
        println("closed early");
    }
    // 这里不会再关一次
    println("after");
    return 0;
}
```

```
close
closed early
after
```

close 只出现了一次。这条保证让「提前关闭」变成一件可以随手做的事——不用记「我关过了吗」，也不用写一堆标志位。

8.4.1 两个都实现的时候，顺序固定

```
先 close()，再 drop()
```

不用记特例：关动作在前，销毁动作在后。

8.5 堆对象：new 和 delete

前面所有东西都活在栈上，跟着作用域走。要手动控制生命周期时，用堆：

```
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Point mut& p = new;
    p.x = 3;
    p.y = 4;
    println("{},{}", p.x, p.y);
    delete p;
    println("deleted");
    return 0;
}
```

```
3,4
deleted
```

- `Point mut& p = new;` ——在堆上建一个 `Point`，`p` 是指向它的引用；
- `delete p;` ——释放它。

`p` 的类型是引用（`Point mut&`），因为堆上的东西没有名字，你只能拿着一个地址。

`new(...)` 也可以带实参调构造器——左边的类型决定用谁构造，那是第 9 章的内容。

忘记 `delete` 会怎样？手动分配的对象不会因为引用变量离开作用域而自动释放。运行结束时会报告仍未释放的堆块；应显式 delete，或将分配封装进有明确 Drop/Closeable 契约的拥有类型。遗留分配会占用预算，长期运行可能耗尽内存。堆对象的最佳实践还是让它的生命周期和某层花括号对齐，别让 `new` 和 `delete` 离得太远。

8.5.1 delete 两次没事

```
struct Box: Drop
{
    int64 id;
    void drop()
    {
        println("drop {}", self.id);
    }
};

int64 main()
{
    Box mut& b = new;
    b.id = 7;
    delete b;
    delete b;
    println("twice ok");
    return 0;
}
```

```
drop 7
twice ok
```

drop 只出现了一次。同一对象只析构、释放一次——重复 delete 不会崩，也不会执行两遍清理。drop() 手动调用同理。

但释放之后不能再读：

```
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Point mut& p = new;
    p.x = 1;
    delete p;
    p.x = 2;
    return 0;
}
```

```
magel: [MGL1205] 借用来源已释放，不能继续访问：表达式
```

MGL1205 也是第 6 章 move 之后读源值时报的码——同一个意思：「这东西已经不在了，别再碰」。

8.6 内存模型的边界

这一节讲 Magel 的「安全」到底覆盖到哪儿。

会被检查的：

- 数组越界、整数运算溢出，都会被抓到并给出定位，不是未定义行为；
- 悬垂引用、释放后访问、移动后访问，编译期或运行期报 MGL1204 / MGL1205。

资源在哪儿：

- PC 上默认内存池 512 MiB（可用 --memory-size 字节调整，最小 64）；
- 嵌入式目标按板子配置，见第 22 章。

长时间运行的服务可以用 std::watchdog 开「软重启」：内存申请失败、显式请求重启、越过内存水位、或开启超时后整整一个周期没 kick() 时，重启自身逻辑而不杀进程。

最后说清它不做什么。Magel 不试图消除所有错误。它针对性地消除三类：

1. 空指针解引用——没有 null，引用必须绑定真实对象（第 7 章）；
2. 悬垂引用——来源作用域检查（第 7 章）；
3. 忘记释放——作用域 + Drop / Closeable 自动收尾（本章）。

剩下的逻辑错误仍然是你自己的责任。比如「算错了平均值」「把用户的年龄存成了负数」「该关的连接关了两次但业务上不允许」——这些语言帮不了你，只能靠第 16 章的错误处理和第 18 章的测试。

8.6.1 defer 和 scope(exit)

defer stmt; 登记当前作用域离开时执行的动作，scope(exit) { ... } 是块形式。普通结束和 return、panic、break、continue 都执行已登记动作一次；同一作用域按登记的逆序执行。

```
int64 main()
{
    {
        defer println("first");
        scope (exit)
        {
            println("second");
        }
        println("body");
    }
    return 0;
}
```

```
body
second
first
```

defer 用于没有所有权关系的收尾，例如恢复状态或结束宿主操作。拥有型资源继续按 Drop、Closeable 的原规则清理。不要对同一资源再登记一份释放动作。

8.7 小结

你会了	关键点
作用域	每对 { } 一层；函数体和 main 也是作用域
Drop	实现它，drop() 在离开作用域时自动调用
析构顺序	先自己的 drop()，再自动析构字段（和 C++ 相反）
禁忌	不要在 drop() 里手动析构字段，会双重析构
Closeable	离开作用域或提前 close()；关过就不会再关
两者都有	顺序固定：先 close() 再 drop()
堆对象	T mut& p = new; 建，delete p; 释放
幂等	delete / drop() 重复调用只执行一次
释放后访问	MGL1205
被消除的三类错误	空指针、悬垂引用、忘记释放
不保证的	逻辑错误——那是你的活儿

8.8 下一步

到此为止，「Magel 的骨头」这一部分讲完了：值语义、引用、资源与作用域。你现在理解了这门语言所有的取舍——为什么复制、为什么不给 null、为什么不用 GC。

接下来是组织代码的工具。第一个是结构体：把一堆字段和一堆方法收进同一个名字下面。

翻到第 9 章结构体。

第 9 章 结构体

到这里我们攒了一堆零散的东西：int64、String、List……它们描述的都是单个值。真实程序里，数据往往是成组出现的：「一个玩家」有名字和分数，「一个点」有横纵坐标。

结构体（struct）就是把一组字段和一组行为收进一个名字下面的东西。

9.1 定义与使用

```
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Point p;
    p.x = 3;
    p.y = 4;
    println("{},{}", p.x, p.y);
    return 0;
}
```

```
3,4
```

拆开看：

```
struct Point {        ← struct 名字 {
    int64 x;          ← 字段：类型 名字;
    int64 y;
}                     ← 类型声明到这里已经完整，后面的分号可省略
```

结构体声明在 } 处结束，允许省略末尾的分号；原有 }; 写法仍有效。字段声明仍然需要分号。

用法：声明一个变量，然后用 . 访问字段（片段，Point 假定已定义）。

```
Point p;
// 声明
p.x = 3;
// 写字段
println("{}", p.x);
// 读字段
```

没赋过值的字段是 0，不是垃圾：

```
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Point p;
    println("[{}][{}]", p.x, p.y);
    return 0;
}
```

```
[0][0]
```

这条和很多 C 系语言不同——那里的未初始化字段是随机值。Magel 帮你清零了。

> **提示**
>
> 字段没有「声明处的默认值」。写 `int64 retries = 3;` 在结构体里是语法错误。想要默认值，用构造器（9.4 节）。

9.2 方法：把动作也收进去

字段旁边可以直接写方法：

```
struct Point
{
    int64 x;
    int64 y;

    int64 sum()
    {
        return self.x + self.y;
    }

    void shift(int64 delta)
    {
        self.x = self.x + delta;
    }
};

int64 main()
{
    Point p;
    p.x = 1;
    p.y = 2;

    p.shift(10);
    println("{} {}", p.sum(), p.x);
    return 0;
}
```

```
13 11
```

两条要点：

方法体里用 `self` 指「这个对象自己」。字段不能裸写，得写 `self.x`。这条规则比 C++ / Java 严格——那里写 `x` 就当 `this->x` 了。Magel 要求你写出来，因为一眼能看出哪个名字是字段、哪个是参数或者局部变量。

方法定义在结构体的花括号里，和字段混在一起，不用另开一段。（有些语言要在结构体外面再写一遍 `impl` 块，Magel 不用。）

调用的写法是 `p.shift(10)` ——用 `.`，因为 `p` 是一个具体的对象。（`::` 是留给「属于类型、不属于对象」的东西的，下一节。）

9.3 static：属于类型，不属于对象

有些东西和「某一个具体对象」无关，只跟类型有关。比如「一共造过多少个 Counter」——这个数不是某个 `Counter` 的字段，它是 `Counter` 这个类型的。

```
struct Counter
{
    static int64 alive;
    int64 value;

    Counter(int64 initial)
    {
        self.value = initial;
        Counter::alive = Counter::alive + 1;
    }

    static int64 count()
    {
        return Counter::alive;
    }
};

int64 main()
{
    Counter c = new(7);
    println("value={} alive={}", c.value, Counter::count());
    return 0;
}
```

```
value=7 alive=1
```

对照着看两种东西：

	写法	访问方式	属于谁
实例字段 / 方法	`int64 value;`	`c.value`	某一个对象
静态字段 / 方法	`static int64 alive;`	`Counter::alive`	类型本身

`.` 和 `::` 的区别就是「对象」和「类型」的区别。记住这条，很多报错就看得懂了。

`static` 方法里没有 `self`——因为它不属于任何对象。想用实例字段？不行。

9.4 构造器与 new(...)

字段没赋过值是 0，但我们常常想要「造出来就是对的」。这就是构造器：

```
struct Counter
{
    int64 value;

    Counter(int64 initial)
    {
        self.value = initial;
    }
};

int64 main()
{
    Counter c = new(7);
    println("{}", c.value);
    return 0;
}
```

```
7
```

构造器没有返回类型，名字和结构体同名。调用它用 new(实参)。

两种情形要说清楚（片段，Counter 假定已定义）：

```
Counter c = new(7);
// 栈上：跟着作用域走
Counter mut& p = new(7);
// 堆上：要自己 delete
```

左边写什么类型，就调那个类型的构造器——new(7) 本身不带类型信息。

区别只在左边有没有 &：

- 左边是值，对象在栈上，出作用域自动清理；
- 左边是引用（Counter mut&），对象在堆上，得自己 delete p;（第 8 章）。

想「就地」建堆对象、不写实参时，括号可以省掉：

```
Counter mut& p = new;
// 等价于 new(0)，元素类型由 p 的类型定
```

new 给的是「按目标类型构造」——目标类型没写，或者那个类型没有匹配实参的构造器，编译器就报错：

```
magel: struct Counter 没有匹配的构造函数：(int64)
```

> **关于 static T new(...)**
>
> 这是「具名构造入口」写法，允许的。它和普通构造器一样，隐式带着 self，体里往 self 上写字段就行，不用（也不能）写 return——返回值由调用点接住。注意调用写法：new(3, 4) 只认真构造器；要用具名入口得写 Point::new(3, 4)：

```
struct Point
{
    int64 x;
    int64 y;

    Point(int64 x, int64 y)
    {
        self.x = x;
        self.y = y;
    }
    // 真构造器
    static Point new(int64 x, int64 y)
    {
        self.x = x * 10;
        self.y = y * 10;
    }
    // 具名入口
};

int64 main()
{
    Point o = new(3, 4);
    // 真构造器
    Point mut& p = new(1, 2);
    // 真构造器 + 堆
    Point q = Point::new(3, 4);
    // 具名入口
    println("{},{} {} {} {} {}", o.x, o.y, p.x, p.y, q.x, q.y);
    delete p;
    return 0;
}
```

```
3,4 1 2 30 40
```

`static X new(...)` 返回所属类型；丢弃它的返回值会报错（拥有值的构造结果必须有归属）。

工厂方法的价值在于：它把「怎么造出一个合法的对象」这件事固定下来，调用方不用记住「先 x 后 y」这种细节。

9.5 让你的类型支持运算符

想让 `a + b` 这种写法用在自己的类型上，靠实现内建接口：

```
struct Money: Add<Money, Money>
{
    int64 cents;

    Money leftAdd(Money other)
    {
        Money result;
        result.cents = self.cents + other.cents;
        return result;
    }
};

int64 main()
{
    Money a;
    a.cents = 150;
    Money b;
    b.cents = 75;

    Money total = a + b;
    println("{} 分", total.cents);
    return 0;
}
```

```
225 分
```

`a + b` 被翻译成 `a.leftAdd(b)`。Magel 里没有 `operator+` 这种声明——运算符是「实现哪个接口」决定的：

运算符	实现接口	方法名
`+`	`Add<R, T>`	`leftAdd`
`==`	`Equal<bool, T>`	`leftEqual`
`-x`（一元）	`Negate<R, T>`	`negate`
在容器里当键	`Hash<T>`	`hash`

接口是什么、`<R, T>` 里的参数是什么意思——这是第 13 章和第 14 章的内容。现在你只要知道：运算符是可以为自己类型定制的，做法是实现接口，不是写 `operator`。

9.6 记住：结构体是值

第 6 章那条规则对结构体同样生效，而且这里最容易踩：

```
struct Rect
{
    int64 width;
    int64 height;
};

int64 area(Rect rect)
{
    return rect.width * rect.height;
}

int64 main()
{
    Rect r;
    r.width = 3;
    r.height = 4;
    println("{}", area(r));
    return 0;
}
```

```
12
```

area(r) 复制了一份 r。对小的 Rect 无所谓；对大的结构体就要写 Rect const&：

```
int64 area(Rect const& rect)
{
    ...
}
// 不复制
void resize(Rect mut& rect)
{
    ...
}
// 要改
```

另外，结构体可以嵌套结构体，也可以装进容器——赋值时都是整套复制，而且复制是递归到底的（第 6 章）。

结构体还能实现 Drop / Closeable 来管理资源，那些第 8 章讲过了。

9.6.1 从字段派生 Equal、Hash 和 Clone

比较、哈希和显式克隆通常要逐个处理字段。可以在结构体前写 #derive(Equal, Hash, Clone)，让编译器生成这些重复的方法。

```
#derive(Equal, Hash, Clone)
struct BookKey
{
    String label;
    int64 id;
}

#derive(Equal, Hash, Clone)
struct BookRecord
{
    BookKey key;
}

int64 main()
{
    BookRecord original;
    original.key.label = "left";
    original.key.id = 7;
    BookRecord copied = original.clone();
    Map<BookKey, int64> cache;
    cache.put(original.key, 10);
    println("{} {} {}", original == copied,
            original.hash(original) == copied.hash(copied),
            cache.has(original.key) != 0);
    copied.key.label = "changed";
    println("{} {}", original.key.label, copied.key.label);
    return 0;
}
```

```
1 1 1
left changed
```

请求	生成的能力	调用方式
`Equal`	按字段比较，生成 `leftEqual` 方法	通常使用 `left == right`
`Hash`	按字段组合哈希，生成 `hash` 方法	`value.hash(value)`；容器按 Hash 协议使用
`Clone`	按字段显式克隆，生成 `clone` 方法	`value.clone()`

这里的 `BookRecord` 包含 `BookKey`，所以后者也必须实现对应接口。字段嵌套了缺少接口的结构体时会直接报错，即使这个类型暂时没有被调用。泛型结构体在具体类型实例化后检查其字段。已有同名方法不会被派生代码覆盖。

不能写 `#derive(Drop)` 或 `#derive(Closeable)`。拥有值离开作用域时已经会按语言规则清理，并递归清

理字段；再生成一套析构方法会破坏所有权。派生只支持 Equal、Hash、Clone，不能把任意接口名当作代码生成请求。引用、函数值、宿主对象及原始数组也不会被自动按地址比较或哈希；它们需要类型明确提供语义。

这些方法在编译时生成，运行时不做反射字段遍历，也不引入 GC。枚举的活动变体和载荷如何处理，见第 10 章。

9.7 常见错误

现象	原因	怎么办
字段声明报缺少分号	字段末尾漏了 ；	字段仍写作 `int64 x;`
`struct X 没有方法 f`	方法名写错，或者定义忘了写	检查拼写与作用域
不能隐式转换	字段类型和赋的值对不上	显式强转，或改字段类型
改不动调用方	结构体按值传了	形参写 `X mut&`，调用写 `&obj`
`X` 没声明的类型	忘了写结构体定义，或忘了 `import`	补上定义 / import

顺带一句：结构体的方法里必须写 `self.`。漏掉会报「找不到这个名字」——这不是编译器啰嗦，是在帮你区分字段和局部变量。

9.8 小结

你会了	关键点
定义	`struct X { ... }` ——结尾分号可选
字段	声明后自动清零；不支持声明处默认值
方法	写在结构体里，用 `self.x` 访问字段
`static`	属于类型，用 `X::name` 访问，里面没有 `self`
构造器	与类型同名、无返回类型，用 `new(实参)` 调用（目标类型由左边声明决定）
栈 vs 堆	`X v = new(实参);` 栈上；`X mut& p = new;` / `new(实参)` 堆上要 `delete`
具名入口	`X::new(...)` 用 `static X new(...);` `new(实参)` 不认它，它只认真构造器
运算符	实现 `Add` / `Equal` / `Negate` 等内建接口
传参	是复制；大结构体用 `const&` / `mut&`

9.9 下一步

结构体解决的是「一组字段永远一起出现」。但还有一类需求它解决不了：

> **提示**
> 一个值要么是这种，要么是那种。

比如「一个形状」要么是圆（带半径），要么是方形（带边长）；「一次操作结果」要么成功（带数据），要么失败（带原因）。

这种「多选一」的数据，是枚举的活儿——也是 Magel 里最漂亮的一块。

翻到第 10 章枚举与模式匹配。

第 10 章 枚举与模式匹配

第 9 章的结构体解决的是「这些字段永远一起出现」。但有一类需求它搞不定：

> 提示
> 一个值要么是这种，要么是那种。

「一个形状」要么是圆（带半径），要么是方形（带边长）。「一次操作的结果」要么成功（带数据），要么失败（带原因）。

这种「多选一」的数据就是枚举。配合 `match`，它是 Magel 里最漂亮的一块。

10.1 定义枚举

```
enum Message
{
    Quit,
    Move(int32 x, int32 y),
    Text(String text)
};
```

读法是：

- `Message` 这个类型，有三种变体（variant）：`Quit`、`Move`、`Text`；
- `Quit` 不带数据；
- `Move` 带两个 `int32`；
- `Text` 带一个 `String`。

变体可以带字段，这在类型理论里叫「代数数据类型」——但用起来没那么玄：枚举就是「有几个可能形状，每个形状各自带自己的数据」。

枚举声明在 `}` 处结束，尾分号可选，与 `struct` 和 `interface` 一致。

造一个值用 `类型::变体`（片段，`Message` 假定已定义）：

```
Message first = Message::Quit;
Message second = Message::Move((int32)3, (int32)7);
Message third = Message::Text("hello");
```

（`(int32)3` 那个强转是必须的——字面量默认是 `int64`，第 3 章的老规矩。）

10.2 用 match 处理

拿到一个枚举值，怎么知道它是哪个变体、里面的数据是什么？用 `match`：

```
enum Message
{
    Quit,
    Move(int32 x, int32 y),
    Text(String text)
};

void show(Message message)
{
    match (message)
    {
        Message::Quit =>
        {
            println("quit");
        }
        Message::Move(x, y) =>
        {
            println("move {} {}", x, y);
        }
        Message::Text(text) =>
        {
            println("text {}", text);
        }
    }
}

int64 main()
{
    show(Message::Quit);
    show(Message::Move((int32)3, (int32)7));
    show(Message::Text("hello"));
    return 0;
}
```

```
quit
move 3 7
text hello
```

写法是：

```
match (要看的东西) {
    模式 => { 该做什么 }
    模式 => { 该做什么 }
}
```

注意三个语法细节：

1. 被匹配的表达式要用括号包起来：`match (message) {`。
2. 箭头是 `=>`（和 C++ 的 `switch` 完全不同），每个分支的主体是一对花括号。
3. `match` 后面不加分号——它是语句，不是表达式。这和 `loop` 正好相反（`loop` 是表达式，语句位置要写 `};`）。写错了会报「期望字符串 / 数字 / 标识符……，实际是分号 (;)」。

分支里的变量是「自动绑定」的。`Message::Move(x, y) => ...` 里写的 `x`、`y` 不是提前声明的变量，而是当场从变体里取出来的（片段，一条 `match` 分支）：

```
Message::Move(x, y) =>
{
    println("move {} {}", x, y);
}
```

这一句同时干了两件事：判断「是不是 `Move`」，以及「把里面那两个 `int32` 取出来叫 `x`、`y`」。

10.3 穷尽性：编译器帮你查漏

这是 `match` 比 `if / else` 强得多的地方。

```
enum Color
{
    Red, Green
};
```

```
enum Color
{
    Red,
    Green
};

int64 name(Color color)
{
    match (color)
    {
        Color::Red =>
        {
            return 1;
        }
    }
    return 0;
}

int64 main()
{
    return 0;
}
```

```
magel: enum match 必须覆盖所有变体或提供 `_` 兜底分支；缺少：Color::Green
```

漏了一个变体是编译错误，不是运行期的意外。

这条规则的价值在于：以后你给枚举加一个变体，所有没处理它的 `match` 会立刻全部报错。编译器会挨个指出「你还没处理新来的那个」。

这才是「用类型保护重构」——改动带来的影响，编译器替你数清楚了。用 `if / else` 加字符串比较的写法，永远得不到这层保护。

实在想偷懒（比如只想处理其中一种），加一个兜底分支（`...` 是省略，不是真语法）：

```
match (color)
{
    Color::Red =>
    {
        ...
    }
    _ =>
    {
        ...
    }
    // 剩下的都到这儿
}
```

_ 是通配符，表示「其他所有情况」。但每次写 _ 都是在放弃后面的检查——加新变体时不会再有提醒。能列全就列全。

10.4 枚举方法

枚举和结构体一样可以有方法，方法里用 `self` 指当前的枚举值：

```
enum Shape
{
    Circle(int64 radius),
    Square(int64 side);

    int64 area()
    {
        match (self)
        {
            Shape::Circle(radius) =>
            {
                return 3 * radius * radius;
            }
            Shape::Square(side) =>
            {
                return side * side;
            }
        }
        return 0;
    }
};

int64 main()
{
    println("{}", Shape::Circle(2).area());
    return 0;
}
```

```
12
```

`match (self)` 就是「看看我自己是哪个变体」。把「一个形状的面积」这件事放在形状自己身上，用起来是 `shape.area()` ——这正是前面说的「把行为和数据放在一起」。

10.4.1 为枚举派生接口

枚举也能使用 `#derive`。相等判断先区分变体，再比较当前活动变体的载荷；哈希同时计入变体和载荷，克隆保留变体并克隆载荷。

```
#derive(Equal, Hash, Clone)
enum BookPacket
{
    Empty,
    Text(String text)
}

int64 main()
{
    BookPacket first = BookPacket::Text("value");
    BookPacket second = first.clone();
    assert(!(first == BookPacket::Empty));
    println("{} {}", first == second, first.hash(first) == second.hash(second));
    return 0;
}
```

```
1 1
```

每个变体的载荷都必须满足所请求的接口；不能因为某个变体暂时没有在程序里构造，就跳过检查。与结构体一样，禁止派生 Drop 或 Closeable。包含无对应接口的结构体载荷时，应先为那个结构体实现或派生接口。

10.5 不止能匹配枚举：标量也支持

match 也能直接匹配整数、浮点、bool，用常量或区间：

```
int64 grade(int64 score)
{
    match (score)
    {
        100 =>
        {
            return 3;
        }
        [90, 99] =>
        {
            return 2;
        }
        [60, 89] =>
        {
            return 1;
        }
        _ =>
        {
            return 0;
        }
    }
    return 0;
}

int64 main()
{
    println("{} {} {} {}", grade(100), grade(95), grade(60), grade(59));
    return 0;
}
```

```
3 2 1 0
```

区间仅支持两种写法：左闭右开 `[a, b)` 与闭区间 `[a, b]`。

写法	含义
`[a, b)`	`a <= x < b`
`[a, b]`	`a <= x <= b`

一个限制

标量分支只接受常量或区间。写变量、函数调用或者别的表达式会报「match 分支必须是整数 / 浮点 / bool 常量或区间模式」。想按「算出来的值」分支，就老老实实用 `if / elif`。

这时候 `_` 就是必需的——标量有无穷多取值，不可能列全。

10.5.1 区间重叠的警告

整数区间与区间、单值与区间的静态重叠会产生 MGL1601 警告，程序仍可编译。编译器按开闭端点和完整的 int64/uint64 精度判断相交，不会把警告改成类型错误，也不会重排分支；运行时仍按原顺序和守卫选择。重复的无守卫单值模式则沿用错误规则。命令行、JSON 与编辑器使用同一警告级别。

10.6 守卫：先绑定，再加条件

有时候光看「是哪个变体」不够，还要看「里面的值满足什么条件」。在模式后面加 `if`：

```
enum Choice
{
    Value(String text)
};

int64 kind(Choice choice)
{
    match (choice)
    {
        Choice::Value(text) if text == "good" =>
        {
            return 1;
        }
        Choice::Value(text) =>
        {
            return 0;
        }
    }
    return -1;
}

int64 main()
{
    println("{} {}", kind(Choice::Value("good")), kind(Choice::Value("bad")));
    return 0;
}
```

```
1 0
```

`Choice::Value(text) if text == "good"` 的意思是：先确认是 `Value` 并取出 `text`，再判断 `text` 是不是 `"good"`。两个条件都满足才进这个分支。

注意第二条 `Choice::Value(text) => ...` 不能省——match 还是要覆盖所有可能性，守卫不算「覆盖」。

> **提示**
>
> 守卫的关键字是 `if`。你在一些资料里可能看到写成 `when` 的版本——那份写法在当前工具链上是不认的（会报「期望条件 (if)」）。记住 `if` 就好。

10.7 什么时候用 match，什么时候用 if

场景	用什么
判断「一个值属于哪种形状」	`match` ——变体全列出来，编译器查漏
一个数落在哪个区间	`match` ——比一串 `if / elif` 清楚
复杂条件（多个变量、算出来的值）	`if / elif / else`
只有两个分支的简单判断	`if / else` ——别为了用 `match` 而用

一条经验：当「分支条件是某个值的形状/范围」时用 `match`；当「分支条件是几个东西的组合判断」时用 `if`。

10.7.1 match 表达式

可以直接把匹配结果用于声明、赋值或调用。所有分支必须产生同类型值；枚举仍要求穷尽，守卫不算覆盖。

```
enum Packet
{
    Data(String text), Empty
};
int64 main()
{
    Packet packet = Packet::Data("original");
    String label = match (packet)
    {
        Packet::Data(text) =>
        {
            text = "copy";
            break text;
        }
        Packet::Empty =>
        {
            "empty"
        }
    };
    String original = match (packet)
    {
        Packet::Data(text) =>
        {
            text
        }
        Packet::Empty =>
        {
            "empty"
        }
    };
    println("{} {}", label, original);
    return 0;
}
```

```
copy original
```

表达式匹配的载荷绑定是值副本，修改副本不修改被匹配对象。单个结果写 `{ value }`，多语句块写 `break value;`。外层声明或赋值仍需要分号。语句位置也支持 `match (...) { ... };`；语句也允许省略末尾分号。

10.8 小结

你会了	关键点
定义枚举	`enum X { A, B(int64 v) }` ——结尾分号可选
造值	`X::A`、`X::B(3)`
`match`	可作语句或表达式；表达式分支给出同类型值，外层声明仍加分号
自动绑定	`X::B(v) => ...` 里的 `v` 是当场取出来的
穷尽性	漏变体是编译错误；`_` 兜底但会放弃检查
标量匹配	常量与区间：`100`、`[90, 99]`、`[a, b)`
守卫	模式 `if` 条件 `=> { ... }`
方法	枚举也能有方法，里面用 `match (self)`

10.9 下一步

现在你能定义自己的类型了：结构体（几个字段一起）和枚举（几种形状选一）。但你还在用 `List<int64>`、`Map<String, int64>` 这些内建容器，而且总在数下标。

下一章把容器和遍历讲透。

翻到第 11 章容器。

第 11 章 容器

前面几章你已经偷偷用过容器了——猜数字里没有，但从第 5 章的 `List<int64>` 开始就一直在用。这一章把它讲清楚。

Magel 内建三种容器，不用 `import`：

容器	装什么	什么时候用
`List<T>`	有序的一串，按位置访问	顺序重要、要按下标取
`Map<K, V>`	键 → 值	按名字查东西
`Set<T>`	不重复的一堆，自动按升序	去重、判断「有没有」

11.1 List<T>：有序表

```
int64 main()
{
    List<int64> scores;
    scores.push(30);
    scores.push(10);
    scores.push(20);

    println("len={} at1={} has10={} idx={}",
        scores.len(), scores.at(1), scores.has(10), scores.indexOf(20));

    scores.insert(0, 7);
    println("insert 后: {} len={}", scores[0], scores.len());

    int64 last = scores.pop();
    println("pop={} len={}", last, scores.len());

    scores.remove(0);
    println("remove(0) 后: {} len={}", scores[0], scores.len());

    scores.clear();
    println("clear 后: len={}", scores.len());
    return 0;
}
```

```
len=3 at1=10 has10=1 idx=2
insert 后: 7 len=4
pop=20 len=3
remove(0) 后: 30 len=2
clear 后: len=0
```

常用操作：

操作	作用	注意
`push(x)`	在末尾加一个	最常用
`insert(i, x)`	在位置 i 插入	后面的元素整体后移
`pop()`	拿走末尾那个并返回	空表上调用会出错
`remove(i)`	删除位置 i	按下标删，不是按值
`clear()`	清空	
`len()`	元素个数	
`at(i)` / `list[i]`	取位置 i	两种写法等价
`has(x)`	有没有这个值	返回 bool（打印是 1/0）
`indexOf(x)`	第一次出现在哪	没有时返回什么见下

`indexOf` 找不到时返回 `-1`——这是惯例，用的时候要判断：

```
int64 position = scores.indexOf(99);
if (position >= 0)
{
    println("在位置 {}", position);
}
else
{
    println("没有这个值");
}
```

11.1.1 越界会被抓到

```
int64 main()
{
    List<int64> numbers;
    numbers.push(1);
    println("{}", numbers[5]);
    return 0;
}
```

```
magel: 运行期错误: vec 下标越界: 5（长度 1）
出错位置: hello.mgl:4（main）
调用栈（外 → 内）:
  main  hello.mgl:4 [thread 0 coroutine 0 ip 25]
```

不是未定义行为，是明确的运行期错误，还告诉你「下标 5、长度 1」。比起读到一块随机的内存，这个错误至少能让你立刻定位。

11.2 Map<K, V>：键 → 值

```
1  int64 main()
2  {
3      Map<String, int64> ages;
4      ages.put("amy", 11);
5      ages["cid"] = 33;
6
7      println("len={} amy={} hasBob={} cid={}",
8          ages.len(), ages["amy"], ages.has("bob"), ages["cid"]);
9
10     println("查一个没有的键: {}", ages["zoe"]);
11
12     for (int64 i = 0; i < ages.len(); i = i + 1)
13     {
14         println("{} -> {}", ages.keyAt(i), ages.valAt(i));
15     }
16     return 0;
17 }
```

```
len=2 amy=11 hasBob=0 cid=33
查一个没有的键: 0
amy -> 11
cid -> 33
```

写法：

- put(k, v) 和 m[k] = v 是一回事，后者顺手一点；
- m[k] 读；has(k) 判断有没有；remove(k) 删掉并返回是否删到了；
- 遍历用 keyAt(i) / valAt(i) 按下标走，i 从 0 到 len() - 1。Map 的遍历顺序是按升序键排列的，所以同一份数据每次遍历结果一致；

> 提示
>
> 小心「查不存在的键」。上面 ages["zoe"] 返回了 0，而不是报错。因为 V 是 int64，缺省值就是 0。这不是「没有这个键」的意思，而是「我按 0 处理了」——想知道键在不在，先用 has(k) 问一次。这是 Map 最容易埋 bug 的地方。

11.3 Set<T>：不重复，自动升序

```
int64 main()
{
    Set<int64> unique;
    unique.add(5);
    unique.add(3);
    unique.add(5);
    // 重复的加不进去

    println("len={} has5={} has9={}", unique.len(), unique.has(5), unique.has(9));
    println("按升序看：at0={} at1={}", unique.at(0), unique.at(1));
    println("remove(5) 拿回 {}，现在 len={}", unique.remove(5), unique.len());
    return 0;
}
```

```
len=2 has5=1 has9=0
按升序看：at0=3 at1=5
remove(5) 拿回 1，现在 len=1
```

Set 的两个特点：

- 自动去重：add(5) 两次，长度还是 1；
- 自动排序：遍历出来是升序（at(0) 是最小的）。

remove 返回的是「有没有删到」。所以「删一个可能不存在的元素」不用先查 has，看返回值就知道。

11.4 集合字面量

不想一行行 push 的时候，可以直接写：

```
int64 main()
{
    List<int64> order =
    {
        3, 1, 2
    };
    println("{} {}", order.len(), order[0]);

    Set<int64> numbers =
    {
        3, 1, 2
    };
    println("{}", numbers.len());

    Map<String, int64> table =
    {
        "a": 1, "b": 2
    };
    println("{} {}", table.len(), table["b"]);
    return 0;
}
```

```
3 3
3
2 2
```

同样一对花括号，左边的类型决定它是什么。

左边写的类型	{...} 的含义
List<T>	有序表，元素按写的顺序排
Set<T>	去重的集合，自动升序
Map<K,V>	键值对（括号里带冒号）

跨类型写会报错——List<int64> v = {"a": 1}; 里带了冒号，那是 Map 的形态：

```
magel: 集合字面量类型不匹配: map 不能初始化 List<int64>
```

空表也有讲究：List<int64> v = {}; 是空表，Set<int64> v = {}; 是空集，类型必须写清楚（auto x = {...}; 这种不给上下文的老写法仍然当成 Set）。

要 List 也可以继续 push（或者用第 17 章的 collect()），两条路都通。

11.5 数组：两种

除了容器，还有两种数组：

```
int64 main()
{
    int64 fixed[3];
    // 长度写在类型里
    fixed[0] = 1;
    fixed[1] = 2;
    fixed[2] = 3;
    println("{}", fixed[0] + fixed[1] + fixed[2]);

    int64 runtime[] = new[4];
    // 长度运行时决定
    runtime[0] = 10;
    println("{} {}", len(fixed), len(runtime));
    return 0;
}
```

```
6
3 4
```

写法	长度	什么时候用
`T a[N]`	编译期定死	大小你知道，要省内存、要对齐硬件
`T a[] = new[n]`	运行时决定	大小要算出来或者读进来

数组和 `List` 的区别：数组长度不可变，`List` 可以 `push` / `pop`。日常业务逻辑几乎都用 `List`；数组留给「缓冲区」「硬件寄存器映射」「性能敏感的定长表」这类场合。

`len(a)` 对数组和容器都能用。越界同样会被检查到。

11.6 别忘了：容器也是值

第 6 章那条规则对容器照旧。这一点在容器上最容易踩：

```
int64 main()
{
    List<int64> original;
    original.push(1);

    List<int64> copy = original;
    // 整表复制一份
    copy.push(2);

    println("original={} copy={}", original.len(), copy.len());
    return 0;
}
```

```
original=1 copy=2
```

`copy = original` 之后两张表毫无关系。从 Java / Python 过来的人第一次写「把表传进函数想让它加元素」，会发现函数里加的东西外面看不到——

```
void addOne(List<int64> numbers)
{
    numbers.push(1);
    // 改的是副本！
}
```

要真的改到外面那张表，写引用：

```
void addOne(List<int64> mut& numbers)
{
    numbers.push(1);
}
// 调用：addOne(&numbers);
```

这也是性能上的关键点：容器一般都有一份堆上的数据，按值传一次就是整份复制。容器参数默认写 `const&` 或 `mut&`，除非你确实想要一份独立副本。

11.6.1 容器里的东西也是副本

```
struct Player
{
    String name;
    int64 score;
};

int64 main()
{
    List<Player> players;
    Player p;
    p.name = "amy";
    p.score = 30;
    players.push(p);
    // 存进去的是一份副本

    p.score = 99;
    // 改原来的 p
    println("表里还是 {}", players[0].score);

    Player back = players[0];
    // 取出来的也是副本
    back.score = 1;
    println("表里还是 {}", players[0].score);
    return 0;
}
```

```
表里还是 30
表里还是 30
```

存进去是副本、取出来也是副本。想原地改容器里的元素，得用引用（第 7 章）：`foreach (Player mut& item in players)` 那类写法在讲完引用之后你就理解了。

> **提示**
> 字符串元素也一样：取出来的是新的一份，所以容器活得比某根字符串久也没关系。

11.6.2 容器归声明它的那一层

容器在离开作用域时自动释放（第 8 章）：

```
int64 main()
{
    {
        List<int64> inner;
        inner.push(1);
        inner.push(2);
        println("块里的表: len={}", inner.len());
    }
    // ← 这里把 inner 还回去了
    println("done");
    return 0;
}
```

```
块里的表: len=2
done
```

不用手动释放，也不用担心泄漏。要提前还就 `delete inner;`。

11.7 遍历

遍历用 `foreach`，不要数下标：

```
int64 main()
{
    List<int64> scores;
    scores.push(12);
    scores.push(55);
    scores.push(41);

    int64 total = 0;
    int64 best = 0;
    foreach (int64 score in scores)
    {
        total = total + score;
        if (score > best)
        {
            best = score;
        }
    }
    println("总和 {} 最高 {}", total, best);
    return 0;
}
```

```
总和 108 最高 55
```

foreach 遍历出来的是元素的副本，所以「边遍历边改」改不到容器。要用下标改，就用 for：

```
for (int64 i = 0; i < scores.len(); i = i + 1)
{
    scores[i] = scores[i] + 10;
    // 这样才改到表里
}
```

> 提示
>
> 遍历时不要增删元素。foreach 途中 push / remove 会让「下一个是谁」变得不可预测。要筛要删，用第 17 章的迭代器 filter / collect 造一张新表，或者先记下要删的下标再统一删。

11.8 小结

容器	常用操作
List<T>	push pop insert remove clear len at [] has indexOf
Map<K,V>	put [] has remove len keyAt valAt
Set<T>	add has remove len at

要记住的：

- 字面量：左边写 List<T> / Set<T> / Map<K,V> 决定 {...} 是什么，带冒号的是 Map；
- 越界是明确的运行期错误，不是未定义行为；
- Map 读不存在的键得到缺省值（不报错），先 has 再取；
- 容器是值：赋值、传参、存取元素都是复制；传参默认写 const& / mut&；
- 遍历用 foreach，遍历出来是副本；遍历时不要增删；
- 容器在出作用域时自动释放。

11.9 下一步

现在你会用容器了，但要写「取分数前 3 名」「把所有名字变大写」这种活儿，用 foreach 加手工判断会很啰嗦。

不过在那之前，先解决一个你已经在用的东西但没细讲过的：字符串。它的下标是按字节的——这跟中文直接相关，值得单独一章。

翻到第 12 章字符串与文本。

第 12 章 字符串与文本

String 你从第 1 章就在用了，但一直没细讲。这一章补上——而且有个坑必须早点讲：

> **提示**
> String 的下标和切片是按「字节」算的，不是按「字」算的。对英文没差别，对中文会出事。

先讲基本操作，再讲这个坑，最后是 std::text 里那堆好用的工具。

12.1 String 是什么

```
String name = "Magel";
```

- 双引号括起来的是 String；
- String 是 std::lang::String 的简写，属于核心模块 lang，不用 import；
- 里面有转义："\{}n"（换行）、"\{}t"（制表）、"\{}""（一个双引号）、"\{}\{}"（一个反斜杠）。

一个重要区别：单引号是 char（一个字符），双引号是 String。这两者不能混：

```
char letter = 'M';
// 一个字符
String word = "Magel";
// 一串字符
```

12.2 拼接、下标、切片、长度

```
int64 main()
{
    String first = "Magel";
    String second = " 2026";
    String joined = first + second;

    println("{}", joined);
    println("len={} 首个字符={}", len(joined), joined[0]);

    println("切片 [0:5] = {}", joined[0: 5]);
    println("切片 [5:]  = {}", joined[5:]);
    return 0;
}
```

```
Magel 2026
len=10 首个字符=M
切片 [0:5] = Magel
切片 [5:]  =  2026
```

- + 拼接两个字符串。和数字的 + 不是一回事——1 + "a" 是编译错误。
- s[i] 取第 i 个「字符」（先按这个理解，12.4 节纠正它）。
- 切片写 s[from:to]，左闭右开：joined[0:5] 是下标 0、1、2、3、4 这五个。
- s[from:] 从 from 一直到结尾，s[:to] 从开头到 to。
- len(s) 是长度。

改单个字符也可以，但要赋一个 char 给它：

```
int64 main()
{
    String word = "hello";
    word[0] =(char)72;
    // 72 就是 'H'
    println("{}", word);
    return 0;
}
```

```
Hello
```

字符串是值（第 6 章），所以赋值得到的是副本：

```
String original = "hello";
String copy = original;
copy[0] = (char)72;      // 只改 copy
```

这一点和第 6 章讲的一模一样：original 还是 hello。

12.3 格式化：format

println 的 {} 你已经很熟了。光要字符串、不要打印，用 format：

```
int64 main()
{
    int64 count = 42;

    // {} 按实参顺序填
    println("{}", format("值={}，下一个是 {}", count, count + 1));

    // {:x} / {:X} / {:b}：整数专用的进制样式
    println("{}", format("255 的十六进制是 {:x}，二进制是 {:b}", 255, 255));

    // 浮点要写成 {索引:小数位数F}，索引从 1 数起
    println("{}", format("圆周率约 {1:2F}", 3.14159));

    return 0;
}
```

```
值=42，下一个是 43
255 的十六进制是 ff，二进制是 11111111
圆周率约 3.14
```

占位符的要点：

- `{}` 是位置占位符，按实参顺序填，类型由实参推导，字符串 / 整数 / 浮点都能填；
- `{:x}` / `{:X}` / `{:b}` 把整数按十六进制 / 大写十六进制 / 二进制打印——只有整数和 `bool` 能用这几个样式。写别的样式（比如 `{:5}`、`{:03}`）会直接报「整数和 bool 的格式仅支持 :b、:x、:X」；
- 浮点用 `{索引:小数位数 F}`，索引从 1 数起：`{1:2F}` 表示「把第 1 个实参保留 2 位小数」；
- 补零 / 对齐没有单独的样式，要用 `std::text::Text::padStart`（见 12.6 节）——想知道 序号 007 怎么写，翻到 12.6 节的那个例子。

> **一个容易踩的坑**
>
> Magel 的 `{}` 不是 `printf` 的 `%d` / `%s`。写了 `%d` 不会报错，但会原样打印出 `%d` 这两个字符，不会填值。

`+` 拼字符串、`format` 拼带变量的字符串、`println` 直接打印——三种手段各有各的位置。短的两三个用 `+`，带变量的用 `{}` 更清楚。

12.4 下标是按字节算的

这一节是本章的重点。

```
int64 main()
{
    String s = "中文";
    println("len(s) = {}", len(s));
    println("第 0 个字节 = {}", (int64)s[0]);
    println("第 3 个字节 = {}", (int64)s[3]);
    return 0;
}
```

```
len(s) = 6
第 0 个字节 = 228
第 3 个字节 = 230
```

"中文" 明明是两个字，len 却说 6。

原因是 Magel 的字符串在内存里是 UTF-8 字节串，而 UTF-8 里一个汉字占 3 个字节。所以：

- "中文" = 6 个字节；
- "Magel" = 5 个字节（和字符数相同，因为都是 ASCII）；
- len、下标、切片，都按字节算。

(int64)s[0] 得到的 228（十六进制 0xE4）是「中」这个字的第一个字节，(int64)s[3] 得到的 230（0xE6）是「文」的第一个字节。

12.4.1 取到「半个字」会怎样

s[1]、s[2] 落在「中」这个字中间，语言不允许你这么读：

```
magel: 运行期错误：字符串下标落在多字节字符中间（String 按字节算）
出错位置：hello.mgl:2（main）
调用栈（外 → 内）：
  main  hello.mgl:2 [thread 0 coroutine 0 ip 41]
```

切片同理：

```
magel: 运行期错误：切片端点落在多字节字符中间（String 的切片按字节算）
```

注意这不报「越界」而是报「落在了多字节字符中间」——说明它知道你在处理 UTF-8，只是拒绝给你半个字。这是个明确的设计：要么给你一个完整的字节序列边界，要么明确拒绝，不给你一段乱码。

12.4.2 所以中文怎么按字处理？

别用下标切中文。三种办法：

1. 整个串用：println("{}", s) 打印是对的——因为整串的边界永远合法；
2. 按分隔符切：用 std::text::Text::split(s, ", ")（12.6 节）。分隔符本身是完整的字符，切出来的每一段边界也合法；

3. 遍历要自己算：想知道 UTF-8 里每个字有几个字节，看首字节的高位就行：| 首字节范围 | 这个字占几字节 | | — | — | | 0x00–0x7F | 1（ASCII） | | 0xC0–0xDF | 2 | | 0xE0–0xEF | 3（含常见汉字） | | 0xF0–0xF7 | 4 | 按这张表跳过续字节（0x80–0xBF），就能一个个字地走。

简单说：处理中文文本，用 std::text 那些按字符/子串工作的函数，不要自己拿下标去数。

12.5 比较

```
int64 main()
{
    println("{} {} {}", "abc" == "abd", "abc" < "abd", "abc" > "abd");
    return 0;
}
```

```
0 1 0
```

== / != / < / <= / > / >= 都能用在字符串上。比较用的是字典序（按字节比大小）。

两个实用的写法（片段，name / expected 假定已存在）：

```
if (name == "")
{
    println("名字是空的");
}
if (name != expected)
{
    println("不匹配");
}
```

判空就用 == ""，没有 is_empty 之类的专用函数。

12.6 std::text：文本工具箱

切分、查找、大小写、去空白这些活儿在 std::text::Text 里。要 import：

```
import std::text;

int64 main()
{
    String raw = "  Hello, Magel!  ";

    println("[{}]", std::text::Text::trim(raw));
    println("{} {}", std::text::Text::toUpper("abc"), std::text::Text::toLower("ABC"));
    println("{} {}", std::text::Text::find("hello", "ll"), std::text::Text::contains("hello", "lo"));
    println("{} {}", std::text::Text::startsWith("hello", "he"),
        std::text::Text::endsWith("hello", "lo"));

    List<String> parts = std::text::Text::split("a,b,c", ",");
    println("{} -> {}", parts.len(), std::text::Text::join(parts, "-"));

    println("[{}]", std::text::Text::replace("aaa", "a", "b"));
    println("[{}][{}]", std::text::Text::padStart("7", 3, (char)48),
        std::text::Text::repeat("ab", 3));
    return 0;
}
```

```
[Hello, Magel!]
ABC abc
2 1
1 1
3 -> a-b-c
[bbb]
[007][ababab]
```

对照着记：

函数	作用	备注
`trim(s)`	去掉两端空白	还有 `ltrim` / `rtrim` 只去一边
`toUpper` / `toLower`	大小写转换	只对 ASCII 字母有效
`find(s, needle)`	找子串位置	找不到返回 `-1`；`rfind` 从右找
`contains(s, needle)`	子串在不在	返回 `bool`
`startsWith` / `endsWith`	前后缀判断	
`split(s, sep)`	按分隔符切开	返回 `List<String>`
`join(parts, sep)`	用分隔符接起来	
`replace(s, from, to)`	替换所有出现	
`padStart(s, width, fill)`	左边补到指定宽度	还有 `padEnd`
`repeat(s, count)`	重复	
`charAt(s, i)`	按字节取字符	和 `s[i]` 等价
`compare(a, b)`	比大小	返回整数，不用管具体值

> **提示**
>
> `padStart("7", 3, (char)48)` 里那个 `(char)48` 是字符 `'0'`。补位字符要传 `char`，不是 `String`——这是类型严格带来的小麻烦，写一次就记住了。

12.7 字符串是值：别在循环里拼

`String` 是值，每次 `+` 都在造一根新字符串。所以这样写会很慢（片段，`lines` 假定已存在）：

```
String text = "";
foreach (String line in lines)
{
    text = text + line + "\n";
    // 每轮都复制一遍已经拼好的部分
}
```

数据量小的时候无所谓。上千行的时候，代价是平方级的。

怎么办？Magel 没有 `StringBuilder` 这种类，但有更顺手的办法：把要拼的东西先塞进 `List<String>`，最后 `join` 一次。

```
import std::text;

int64 main()
{
    List<String> lines;
    lines.push("第一行");
    lines.push("第二行");
    lines.push("第三行");

    String text = std::text::Text::join(lines, "\n");
    println("{}", text);
    return 0;
}
```

```
第一行
第二行
第三行
```

一条经验：循环里拼字符串，先攒 List，出循环再 join。

12.7.1 插值字符串

插值字符串写成 f"..."，只有 f 后紧跟双引号才识别为插值，单独的 f 仍是标识符。花括号里是完整表达式，每个实参按顺序求值一次，最终沿用 format 的语义。

```
int64 main()
{
    int64 count = 3;
    String text = f"count={count + 1}, {f"inner={count}"}, \{literal\}";
    println("{}", text);
    return 0;
}
```

```
count=4, inner=3, {literal}
```

\{ 和 \} 输出字面大括号。嵌套插值由内到外展开；没有花括号包围的文本不求值。

12.7.2 整行输入和文件文本

std::lang::readLine() 读取一行，保留空格和空行，去掉 LF/CRLF；EOF 返回空串。std::lang::readLines(int32 N) 最多读 N 行，用 LF 连接，EOF 提前结束，不额外追加行末换行。另有 int64 重载接受普通整数实参，检查后转为 int32。N 为零时不读输入，负数或超出 int32 正范围触发错误。String 不能包含内嵌零字节。

```
int64 main()
{
    String name = std::lang::readLine();
    String notes = std::lang::readLines(2);
    println("name=[{}], notes=[{}]", name, notes);
    return 0;
}
```

inputLine() 和 input(variable) 按空白取项。--input 已按空白切分，每项供新接口视为一行；要保留真实空格和空行，使用终端或标准输入管道。

导入 std::io 后，FileStream.readString(count) 从当前游标读至多 count 个字节，readLine() 读一行，readAllString(maxBytes) 读到 EOF。三者返回 Result<String,IoError>；全读默认预算 16 MiB，保留文件换行字节，零字节文本报错。已有按路径的 readAllText(path) 继续可用。直接持有 File 句柄时有 fileReadString、fileReadLine、fileReadAllString 包装，错误会 panic。

完整验证程序见 行输入 与 文件读取。

12.8 小结

你会了	关键点
字面量	双引号 "…"；单引号是 char
拼接	+；数字和字符串不能混着拼
下标 / 切片	s[i]、s[a:b]（左闭右开）、s[a:]、s[:b]
长度	len(s)
按字节	len / 下标 / 切片都按字节算；中文一个字 3 字节
半个字	下标或切片落在多字节字符中间 → 运行期错误
格式化	format: {} 自动填、{:x} 整数进制、{1:2F} 浮点位数
比较	== != < >，字典序
判空	s == ""
工具箱	import std::text; ——trim split join replace find toUpper padStart …
性能	循环里别用 + 拼，攒 List 再 join

12.9 下一步

第四部分到这儿结束了：结构体、枚举、容器、字符串，你有了组织数据的所有工具。

第五部分讲抽象——怎么写出「不关心具体类型」的代码。第一个是接口，它解决「一堆类型都有同一个动作」的问题。

翻到第 13 章接口与运行期多态。

第 13 章 接口与运行期多态

到第 12 章为止，你写的每个函数都只认一种具体类型：int64 也好，Circle 也好，参数的类型写死了，就只能吃那一种。

可现实里常有这种事：一个函数想同时处理「圆」和「正方形」，因为它俩都需要「算面积」。你不会想给每个类型都抄一份逻辑。这一章讲 Magel 怎么解决它——interface。

13.1 问题：长得不一样，但都会做同一件事

先看一个真实场景。你有一个圆、一个正方形，都要算面积：

```
struct Circle
{
    int64 r;
};
struct Square
{
    int64 side;
};

int64 circleArea(Circle c)
{
    return 3 * c.r * c.r;
}
int64 squareArea(Square s)
{
    return s.side * s.side;
}
```

两个函数，逻辑几乎一样，只是字段名不同。再加一个三角形呢？再抄一份。

问题不在「代码重复」，而在没有共同的类型。circleArea 收不了 Square，squareArea 收不了 Circle。你没法写「一个对所有图形都能算面积的函数」。

接口就是来解决这件事的：把「会算面积」这件事本身，变成一个类型。

13.2 interface：给「能力」起个名字

```
interface Shape
{
    int64 area();
    String describe();
};
```

这段在读作：

- interface Shape 声明了一个叫 Shape 的接口；
- 里面两条方法声明（只有签名，没有函数体）：想要成为 Shape，就得有 area() 和 describe()；
- 接口里只能有方法，不能有字段——这是硬规矩。

换句话说，Shape 描述的不是「有什么数据」，而是「会做什么」。这种类型，学名就叫契约（contract）：谁签了它，谁就得履行这两条。

> 提示
>
> interface、struct、enum 的声明在 } 处结束，末尾的 ; 可省略。接口中的抽象方法声明仍需要分号。

13.3 结构体实现接口

写一个真的圆，让它「实现」Shape——写法是在结构体名字后面加 : Shape：

```
interface Shape
{
    int64 area();
    String describe();
};

struct Circle: Shape
{
    int64 r;

    int64 area()
    {
        return 3 * r * r;
    }
    String describe()
    {
        return "圆";
    }
};

int64 main()
{
    Circle c;
    c.r = 2;
    println("{}, 面积 {}", c.describe(), c.area());
    return 0;
}
```

```
圆, 面积 12
```

`struct Circle : Shape` 的意思就是「Circle 承诺自己是 Shape」。它把 Shape 要的两条方法都写了，于是契约生效。

关键规矩：必须完全实现，一个都不能少。少写一条，编译器直接拒绝：

```
interface Shape
{
    int64 area();
    String describe();
};

struct Bad: Shape
{
    int64 r;
    int64 area()
    {
        return 3 * r * r;
    }
};

int64 main()
{
    return 0;
}
```

报错说得非常清楚：

```
magel: struct Bad 实现了接口 Shape，但没有完全实现：describe() -> std::lang::String
（接口要求同名、同返回类型、同形参类型；一个都不能少）
```

注意「同名、同返回类型、同形参类型」这三点——返回类型写错也算没实现。把 `area()` 写成返回 `String`，一样通不过。

> **提示**
>
> 一个细节：在 `main` 里 `c.describe()` 是静态调用——编译器早知道 `c` 是 `Circle`，直接找到 `Circle::describe` 就行，根本不需要接口。接口的价值要等到 13.7 节，类型在运行期才确定的时候才显出来。

13.4 默认方法：接口可以带实现

接口里的方法可以自带函数体，成为「默认实现」：

```
interface Greeter
{
    String hello()
    {
        return "你好，我是";
    }
};

struct Dog: Greeter
{
    String name()
    {
        return "旺财";
    }
};

int64 main()
{
    Dog d;
    println("{}{}", d.hello(), d.name());
    return 0;
}
```

```
你好，我是旺财
```

Dog 只写了 name()，hello() 直接用接口给的默认体。这在「大部分实现都一样、少数才特殊」时很省事——想让某个类改写默认行为，在结构体里重写一遍同名方法即可。

> **提示**
>
> 接口方法也可以有默认参数，但枚举载荷字段和接口方法声明这两处不支持默认参数（见语言参考第 6 节）。这一条初学阶段用不到，记个位置就行。

13.5 接口继承

接口之间也能继承。Named : Shape 读作「Named 是更具体的 Shape」：

```
interface Shape
{
    int64 area();
};

interface Named: Shape
{
    String name();
};

struct Square: Named
{
    int64 side;

    int64 area()
    {
        return side * side;
    }
    String name()
    {
        return "正方形";
    }
};

int64 main()
{
    Square s;
    s.side = 4;
    println("{}, 面积 {}", s.name(), s.area());
    return 0;
}
```

```
正方形, 面积 16
```

Square 签名签的是 Named，但 Named 继承了 Shape，所以两个要求一起生效：area() 和 name() 都得有。

13.6 一个结构体实现多个接口

结构体可以同时签多个契约，用逗号隔开：struct X : A, B。如果两个接口要的是同一个签名的方法，只算一条要求——去重的：

```
interface Shape
{
    String describe();
};
interface Named
{
    String describe();
    String name();
};

struct Circle: Shape, Named
{
    int64 r;
    String describe()
    {
        return "圆";
    }
    String name()
    {
        return "Circle";
    }
};

int64 main()
{
    Circle c;
    println("{} {}", c.describe(), c.name());
    return 0;
}
```

```
圆 Circle
```

Shape 和 Named 都要 String describe()，它们同名、同返回、同参数，所以 Circle 里只写一遍 describe() 就同时满足了两个接口。反过来，要是两个接口要的同名方法签名不一样，那就是冲突，写不出来。

13.7 接口引用：多态真正登场的地方

前面六节，接口看起来只是「一份要遵守的清单」——实现完就完了，调用还是静态的。这一节才是重点：接口名可以当引用的目标类型，而这时调用的具体实现，由运行期决定。

先看它长什么样：

```
Shape mut& view;
// view 是「Shape 引用」
view.area();
// 调哪个 area()？运行期才知道
```

Shape mut& 不是普通引用。它是个「胖引用」——里面装了两样东西：对象的地址和对象的类型号。所以 view.area() 可以按类型号去挑对应的实现。这就是运行期多态：同一行代码，喂不同的对象，跑不同的函数。

那怎么拿到这么个引用？只能从 std::lang::Any 里解包。为什么要这么绕？因为 Shape mut& 需要类型号，而裸引用（&c 那种）里没有类型号——语言不让你硬凑：

```
Circle c;
Shape mut& v = &c;
// X 报错
```

报错会教你正确的路：

```
magel: 接口引用 Shape mut& 只能从 obj 解包来，不能拿 Circle mut& 凑：
先 `o.set(具体对象);` 再 `Shape mut& q = o.get();`（接口引用是「地址 + 类型号」的胖引用）
```

std::lang::Any 是第 8 章讲的那个「类型擦除的盒子」——它能装下任何类型，并记住装进去的是谁。解包时按接口取，它就去校验：这个对象真的实现了 Shape 吗？

完整流程：存进盒子 → 按接口取出来 → 当接口用。

```
interface Shape
{
    int64 area();
    String describe();
};

struct Circle: Shape
{
    int64 r;
    int64 area()
    {
        return 3 * r * r;
    }
    String describe()
    {
        return "圆";
    }
};

struct Square: Shape
{
    int64 side;
```

```
    int64 area()
    {
        return side * side;
    }
    String describe()
    {
        return "正方形";
    }
};

String report(Shape mut& shape)
{
    return shape.describe() + ": " + format("{}", shape.area());
}

int64 main()
{
    Circle c;
    c.r = 2;
    Square s;
    s.side = 3;
    Any box1;
    Any box2;
    box1.set(&c);
    box2.set(&s);
    Shape mut& shape1 = box1.getReference();
    Shape mut& shape2 = box2.getReference();
    println("{}", report(shape1));
    println("{}", report(shape2));
    return 0;
}
```

```
圆: 12
正方形: 9
```

一步步拆：

1. `std::lang::Any box1;` ——声明一个空盒子；
2. `box1.set(&c);` ——把 c 装进去。这里传的是 `&c`（c 的引用），盒子不复制对象，只是记住「地址 + 它是 Circle」；
3. `Shape mut& shape1 = box1.getReference();` ——按 Shape 把引用取出来。类型号校验就发生在这里：Circle 确实实现了 Shape，放行；要是取成没实现的接口，这里就报运行期错误；
4. `report(shape1)` ——report 只认 `Shape mut&`，它压根不知道里面有圆还是正方形。`shape.area()` 到运

行期才按类型号分派——这就是多态。

同一个 `report`，一行代码，两段输出。加第三种图形，`report` 一个字都不用改。

13.8 该用谁：静态调用还是接口？

刚学多态容易走极端，觉得「接口多高级，全用接口吧」。别。看这张表：

场景	用什么	为什么
你在编译期就知道具体类型	直接调用	快，清楚，别绕
一个函数要吃多种类型，且类型在运行期才定	接口引用	只有它能做
只是想让几个类型共享一段代码	先想想泛型（第 14 章）	泛型在编译期展开，往往更合适

经验法则：只有「调用者不知道具体是谁」的时候，才需要接口引用。能静态调用就静态调用——这也是官方样例把静态和动态两条路摆在一起对比的原因。

13.9 小结

你会了	关键点
声明接口	`interface Shape { int64 area(); }`；可有方法与关联类型声明，不能有值字段
实现接口	`struct Circle : Shape { …}`，必须完全实现，签名一字不差
默认方法	接口里的方法可以自带函数体
接口继承	`interface Named : Shape { …}`
多接口	`struct X : A, B`，同名同签名的要求去重
接口引用	`Shape mut& view`，是「地址 + 类型号」的胖引用
怎么拿	`box.set(&obj); Shape mut& v = box.getReference();`
运行时多态	方法按对象类型号分派；调用者无需知道具体类型
别滥用	能静态调用就别上接口

13.10 下一步

接口解决了「一个函数吃多种类型」，但它是运行期付账——胖引用、类型号、装箱解包，都有代价。

第 14 章的泛型解决同一个问题，却是编译期付账：代码为你用到的每种类型分别展开一份，跑起来没有运行期开销。学会这两样，你就能在自己的代码里做正确的取舍了。

翻到第 14 章泛型。

第 14 章 泛型

第 13 章的接口，解决的是「一个函数吃多种类型」——但它靠的是运行期的胖引用和类型号，每调一次都有分派开销。

这一章的泛型解决同一个问题，思路完全不同：在编译期为每一种用到的类型，分别生成一份专门的代码。跑起来没有分派，因为它压根就是一开头就定死的。

先看它要治什么病。

14.1 问题：换一种类型，逻辑就得抄一遍

写一个「谁大取谁」的函数。整数版：

```
int64 maxOf(int64 a, int64 b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}
```

字符串也想要一个——逻辑一字不差，只有类型不同：

```
String maxOf(String a, String b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}
```

`int64`、`float64`、`String`……每加一种类型就得抄一份。用第 4 章的重载能跑，但你心里清楚：这三份代码长得一模一样，只是类型不同。类型之间应该有办法复用。

14.2 泛型函数：把「类型」也变成参数

泛型的写法很简单：在函数名的尖括号里声明一个类型参数 `T`，然后在原本写类型的地方写 `T`。

```
T maxOf<T>(T a, T b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}

int64 main()
{
    println("{}", maxOf(3, 9));
    println("{}", maxOf("apple", "pear"));
    return 0;
}
```

```
9
pear
```

T 是一个占位符，代表「先不定，等调用时再说」的类型：

- `maxOf(3, 9)` ——编译器一看实参是 `int64`，就把 T 当成 `int64`；
- `maxOf("apple", "pear")` ——这次 T 是 `String`。

一个定义，两种类型。加第三种也不用改 `maxOf` 一个字。

> **提示**
>
> 调用时 T 是怎么定下来的？靠实参推导——编译器看实参是什么类型，就把 T 绑成什么类型。

14.3 显式写出来，或者让编译器推

推导不是每次都行。你可以把类型实参显式写出来，写法是把类型放在函数名后的尖括号里：

```
T maxOf<T>(T a, T b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}

int64 main()
{
    println("{}", maxOf<int64>(3, 9));
    println("{}", maxOf<String>("apple", "pear"));
    return 0;
}
```

```
9
pear
```

什么时候必须显式？实参推不出唯一答案的时候。比如 int64 和 float64 混着喂：

```
T maxOf<T>(T a, T b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}

int64 main()
{
    println("{}", maxOf(1, 2.0));
    return 0;
}
```

报错直接点名：

```
magel: 泛型函数 maxOf 的类型实参推导冲突，请显式指定类型实参
note: 第 2 个实参实际为 float64，形参模式为 ...
```

1 是 int64、2.0 是 float64，编译器推不出 T 该是哪一个——Magel 不做隐式转换（第 3 章），所以它宁可停下来让你说清楚，也不自作主张。想让这个调用成立，就写 maxOf<float64>(1.0, 2.0)（注意 1 也要改成 1.0，float64 之间才谈得上比较）。

14.4 泛型结构体

结构体也能带类型参数。声明的时候在名字后写 <>，字段就能用这些参数当类型：

```
struct Duo<T, U>
{
    T first;
    U second;
};

int64 main()
{
    Duo<int64, String> d;
    d.first = 1;
    d.second = "a";
    println("{} {}", d.first, d.second);
    return 0;
}
```

```
1 a
```

Duo<T, U> 要两个类型参数。Duo<int64, String> 是一个「第一格是整数、第二格是字符串」的结构。Duo<String, int64> 又是另一个。

> **一个坑**
>
> 别给泛型起名 Pair。Pair 是标准库自带的 Pair（随 lang 自动加载，不用 import），你再声明一个同名的泛型，会报「泛型：重复的泛型声明 Pair」——那个报错没提「和标准库撞了」，容易懵。

泛型结构体的方法也能用 T：

```
struct Box<T>
{
    T value;
    T get()
    {
        return value;
    }
    void put(T v)
    {
        value = v;
    }
};

int64 main()
{
    Box<int64> b;
    b.put(41);
    println("{}", b.get());
    Box<String> s;
    s.put("hi");
    println("{}", s.get());
    return 0;
}
```

```
41
hi
```

`Box<int64>` 和 `Box<String>` 各自有一份 `put` / `get`，参数和返回值都变成了具体的类型。

14.5 约束：给类型参数提要求

`maxOf<T>` 里用了 `a > b`。这要求 `T` 支持比大小——如果你拿一个没有 `>` 的结构体去实例化，它会报错。与其等报错，不如提前声明这个要求。

约束写在类型参数后面，用冒号接一个接口（第 13 章的接口）：

```
interface Valued
{
    int64 value();
};

struct Num: Valued
{
    int64 n;
    int64 value()
    {
        return n;
    }
};

int64 read_value<T: Valued>(T item)
{
    return item.value();
}

int64 main()
{
    Num x;
    x.n = 42;
    println("{}", read_value(x));
    return 0;
}
```

```
42
```

`<T: Valued>` 读作「T 必须是实现了 Valued 的类型」。有了这条，函数体里就能放心地调 `item.value()`——编译器知道 T 一定有这个方法。多个约束用逗号隔开：`<T: A, U: B>`。

约束把「报错时机」提前了。拿一个没实现 Valued 的类型去调，编译器立刻拦下：

```
interface Valued
{
    int64 value();
};

int64 read_value<T: Valued>(T item)
{
    return item.value();
}

int64 main()
{
    println("{}", read_value(3));
    return 0;
}
```

```
magel: read_value 的类型实参 int64 未实现接口 Valued
```

一句话说清：约束是写给编译器看的承诺，它保证函数体里那些方法一定存在。

14.6 单态化：编译期展开，运行期零开销

泛型最有意思的地方，是它根本不存在于运行期。

编译器在编译时，为每个实际用到的类型实参，生成一份专门的函数/结构体：`maxOf<int64>` 编译出一份只处理 `int64` 的代码，`maxOf<String>` 编译出另一份。这个做法叫单态化（monomorphization）——「单一形态化」。

三个直接后果：

1. 没有运行期分派。调用 `maxOf(3, 9)` 和调用一个手写的 `maxOf(int64, int64)` 一样快，机器码是同一份；这跟第 13 章的接口胖引用完全不同（那个要查类型号）。
2. 泛型代码的检查发生在实例化时。`maxOf<T>` 的函数体里写着 `a > b`，编译器在没看到具体类型之前不会去挑它的毛病；等你拿一个不支持 `>` 的类型来实例化，它才报错——而且报的是那个实例的错：

```
T twiceOf<T>(T v)
{
 return v + v;
}

struct P
{
 int64 x;
};

int64 main()
{
 P p;
 p.x = 1;
 println("{}", twiceOf(p).x);
 return 0;
}
```

```
magel: 没有匹配的运算符接口实现: P + P（由左操作数实现 Add<R,T> 这类左接口，
或由右操作数实现 RightAdd<R,T> 这类右接口）；不会偷偷按数值算
```

twiceOf<int64> 和 twiceOf<String> 都没问题（整数 + 整数、字符串 + 字符串），只有 twiceOf<P> 这一份不行。

3. 代码会变大。每个实例都是一份独立的机器码，用一百种类型就生成一百份。这是泛型的代价，换来的是运行期的干净。

> **提示**
>
> 顺带一提：Pair<K, V>（14.4 节提过的那个）本身就是泛型结构体——标准库也在用同一套机制。

14.7 默认类型参数

类型参数可以有默认值，写在参数名后面用 = 接：

```
struct Wrap<T, U = T>
{
    T a;
    U b;
};
```

Wrap<int64> 会写成 Wrap<int64, int64>——U 缺省就跟着 T 走。默认值可以引用前面的参数（U = T 合法），但不能引用后面的、也不能是自己。

> **提示**
>
> 默认类型参数在初学阶段基本用不到，先把 14.2–14.6 吃透。这一节只是让你在别人的代码里见到 `struct Foo<T, U = T>` 时知道那是什么意思。

14.8 接口还是泛型？

这两个都能「一个函数吃多种类型」，怎么选？看一遍这张表：

	接口（第 13 章）	泛型（本章）
何时定类型	运行期	编译期
开销	胖引用 + 类型号分派	无（代码已展开）
代价	每次调用有分派成本	代码体积变大
怎么用	类型在运行期才知道（如从容器里取）	类型在编译期就能定
共同点	用接口做约束（`<T: Valued>`）能结合起来	同左

经验法则：

- 编译期就知道类型 → 泛型；运行期才定 → 接口。
- 两者能合起来用：泛型加约束 `<T: Shape>` 就是「泛型 + 接口」，既零开销、又能调接口方法。

14.8.1 编译期整数参数

泛型支持 `N: int64` 等整数泛型参数，长度参与单态化，不能传入运行期变量。结构体的一维和二维定长数组字段使用这些常量确定布局。

```
struct Buffer<T, N: int64>
{
    T data[N];
    int64 len()
    {
        return N;
    }
};
int64 main()
{
    Buffer<int64, 4> buffer;
    buffer.data[0] = 7;
    println("{} {}", buffer.len(), buffer.data[0]);
    return 0;
}
```

```
4 7
```

`Buffer<int64,4>` 与 `Buffer<int64,8>` 是不同实例。二维矩阵同理：

```
struct Matrix<T, rows: int64, cols: int64>
{
    T data[rows][cols];
};
```

长度可以有默认值，仍须是可在编译期求值的 int64 常量；运行期长度继续用 `T a[] = new[n]`。

14.8.2 关联类型

接口体内的 `type Item;` 声明关联类型，实现者用 `type Item = T;` 提供具体类型，受约束泛型可以写 `C.Item`。

```
interface Container
{
    type Item;
    Item get(int64 index);
};
struct Numbers: Container
{
    type Item = int64;
    Item get(int64 index)
    {
        return index + 10;
    }
};
C.Item first<C: Container>(C value)
{
    return value.get(0);
}
int64 main()
{
    Numbers numbers;
    println("{}", first<Numbers>(numbers));
    return 0;
}
```

```
10
```

接口内是关联类型产生式，模块层的 `type` 仍是类型别名，原有类型参数 `<T: Valued>` 保持可用。

14.8.3 泛型返回控制表达式

返回块、`if`、`match` 和 `loop` 同样可以使用类型参数 T。具体类型确定后，编译器再按该类型检查每条结果路径。

```
T fromLocal<T>(T value)
{
    return
    {
        T local = value;
        break local;
    };
}

T chooseValue<T>(bool first, T left, T right)
{
    return if (first)
    {
        T local = left;
        break local;
    }
    else
    {
        break right;
    };
}

int64 main()
{
    String word = "generic";
    String copied = fromLocal<String>(word);
    int64 number = chooseValue<int64>(false, 4, 9);
    println("{} {} {}", word, copied, number);
    return 0;
}
```

```
generic generic 9
```

`local` 在值表达式内部，通过 `break` 隐式移动；`right` 在表达式外部，按值复制。函数形参本身也是调用者值的副本，所以这里不会消耗调用者的 `word`。泛型不会取消分支类型一致、生命周期或清理检查，返回引用也不能指向已经退出的局部对象。

14.9 小结

你会了	关键点
泛型函数	`T maxOf<T>(T a, T b)`，类型参数写在名字后
调用	可推导 `maxOf(3, 9)`，可显式 `maxOf<int64>(3, 9)`
泛型结构体	`struct Duo<T, U> { T first; U second; }`
约束	`<T: Valued>`，保证函数体能调接口方法
单态化	编译期按类型实参各生成一份，运行期无分派开销
报错时机	约束不满足 → 调用处报；无约束泛型的体 → 实例化时报
命名坑	别用标准库已占的泛型名（如 `Pair`）
默认类型参数	`<T, U = T>`（进阶，先了解）

14.10 下一步

到现在为止，你写的所有函数都得先有名字才能用：`maxOf`、`read_value`……可有时候你只想临时写一小段逻辑，随手传给别的函数——排序的「怎么比」、遍历的「对每个元素做什么」。

这就要把函数本身当值来传。第 15 章讲函数值、`fn` 和闭包。

翻到第 15 章函数值与闭包。

第 15 章 函数值与闭包

到目前为止，函数在你眼里还只是「一段有名字的代码」。可有时候，你并不关心那段代码叫什么，你只想把它本身当参数传出去——排序时告诉程序「怎么比」，遍历时告诉它「对每个元素做什么」。

这一章讲的就是这件事：函数也是一种值。而且它可以记住定义它的地方的一些变量——那就是闭包。

15.1 问题：逻辑一样，只有「条件」不同

写一个「从列表里筛出满足条件的元素」的函数。筛偶数是一个条件，筛大于 10 的是另一个条件：

```
List<int64> filter(List<int64> xs, bool fn(int64) keep)
{
    List<int64> out;
    foreach (int64 x in xs)
    {
        if (keep(x))
        {
            out.push(x);
        }
    }
    return out;
}

int64 main()
{
    List<int64> xs;
    xs.push(3);
    xs.push(8);
    xs.push(15);
    xs.push(42);
    List<int64> evens = filter(xs, fn(int64 x)
    {
        x % 2 == 0
    });
    List<int64> big = filter(xs, fn(int64 x)
    {
        x > 10
    });
    println("{} {}", evens[0], evens[1]);
    println("{}", big.len());
    return 0;
}
```

```
8 42
2
```

`filter` 只管「遍历、判断、收集」，判断的标准由第二参数 `keep` 带进来。`keep` 的类型是 `bool fn(int64)`——「吃一个 `int64`、吐一个 `bool` 的函数」。传进去的两个 `fn(…)` 就是两个不同的条件。

一个 `filter`，无数种条件。这就是「把函数当值」的价值。

15.2 函数类型：R fn(A, B)

先认这个写法：

```
int64 fn(int64, int64)  // 吃两个 int64，返回 int64
bool fn(int64)  // 吃一个 int64，返回 bool
String fn()  // 不吃参数，返回 String
```

读法从左到右：返回类型 fn(参数类型)。函数类型也是类型，能写在变量、字段、参数、数组上——凡是你写 int64、String 的地方，都能写 R fn(…)。

15.3 函数可以当值用

把一个函数名赋给函数类型的变量，之后就能像调用原函数一样调它：

```
int64 add(int64 a, int64 b)
{
    return a + b;
}
int64 sub(int64 a, int64 b)
{
    return a - b;
}

int64 apply(int64 fn(int64, int64) f, int64 a, int64 b)
{
    return f(a, b);
}

int64 main()
{
    int64 fn(int64, int64) op = add;
    println("{}", op(3, 4));
    op = sub;
    println("{}", op(3, 4));
    println("{}", apply(add, 20, 22));
    return 0;
}
```

```
7
-1
42
```

op 这个变量里装的是一个函数：

- op = add; 把 add 装进去；
- op = sub; 换成 sub——同一个变量，指哪打哪；
- apply(add, 20, 22) 把函数当普通实参传进去。

函数值能放的地方和你想象的一样多——字段、数组都行：

```
struct Calculator
{
    int64 fn(int64, int64) op;
};

int64 add(int64 a, int64 b)
{
    return a + b;
}
int64 mul(int64 a, int64 b)
{
    return a * b;
}

int64 main()
{
    Calculator c;
    c.op = mul;
    println("{}", c.op(3, 4));

    int64 fn(int64, int64) table[2];
    table[0] = add;
    table[1] = mul;
    println("{} {}", table[0](6, 7), table[1](6, 7));
    return 0;
}
```

```
12
13 42
```

table 是一张「操作表」，按下标取函数来调——这是实现「操作码分派」最朴素的写法。

15.4 fn：现场写一个小函数

add / sub 都是提前定义好、有名字的函数。可很多时候，你只想当场写一小段逻辑，懒得给它起名。这就是 fn：

```
int64 main()
{
    int64 fn(int64) square = int64 fn(int64 x)
    {
        x * x
    };
    println("{}", square(6));

    auto cube = fn(int64 x)
    {
        x * x * x
    };
    println("{}", cube(3));
    return 0;
}
```

```
36
27
```

写法拆开看：返回类型 `fn(参数) { 体 }`。

- `fn` 是关键字；
- `(int64 x)` 跟普通函数的参数一模一样；
- `fn` 前面的 `int64` 是返回类型，可以省略（`cube` 那份就没写，编译器自己推）；
- `{ …}` 是函数体。

`auto cube = …`让编译器根据右边推出 `cube` 的类型——省得手写一长串 `int64 fn(int64)`。

15.5 省略参数类型：靠上下文推

如果 `fn` 是要传给某个参数的，而那个参数的类型是已知的，那么连参数类型都能省——编译器按上下文补上：

```
int64 apply(int64 fn(int64, int64) f, int64 a, int64 b)
{
    return f(a, b);
}

int64 main()
{
    println("{}", apply(fn(x, y)
    {
        x + y
    }, 3, 4));
    println("{}", apply(fn(x, y)
    {
        x * y
    }, 3, 4));
    return 0;
}
```

```
7
12
```

apply 的第一参数类型是 int64 fn(int64, int64)，所以 fn(x, y) 里的 x / y 自动就是 int64。这种「看上下文定类型」的能力，让 fn 写起来非常轻。

> **提示**
>
> 但别滥用。参数类型省掉后，读代码的人得自己回头找上下文。类型不是一眼能看出来的地方，就老老实实写上。

15.6 闭包：记住定义处的变量

fn 最厉害的地方，是它能看见外面的变量：

```
int64 main()
{
    int64 base = 10;
    auto addBase = fn(int64 x)
    {
        x + base
    };
    base = 100;
    println("{}", addBase(5));
    return 0;
}
```

```
105
```

addBase(5) 读到的是调用那一刻 base 的值 100，所以输出 105。换句话说，闭包里那个 base 不是一份拷贝，而是一条指向外部变量的线。

这条线就叫捕获（capture）：闭包把外部变量”抓”进来，从此你在闭包里读写它，读写的其实是外面那个变量本身。

> **提示**
>
> 默认捕获是只读借用。不写任何修饰时，闭包按 const& 抓住外部变量：能读、能看到外面的新值，但不能改。想改就得显式声明，见 15.7 节。

15.7 四种捕获方式

捕获方式写在 fn 关键字后面，一共四种：

写法	名称	闭包里能改？	外面那个变量	寿命限制
fn const(...)（也是默认）	只读借用 const&	不能改	不受影响	来源要活过闭包
fn mut(...)	可变借用 mut&	能改	跟着变	来源要活过闭包
fn copy(...)	拥有副本	能改（改副本）	不受影响	无，自己有一份
fn move(...)	转移所有权	能改（独占）	被搬走，须重新声明	无，自己有一份

一句话区分：const& / mut& 是”借”，copy / move 是”拿”。借来的东西要还，所以有寿命限制；拿了就是自己的，想活多久活多久。

15.7.1 const&：默认，只读借用

不写修饰就是它，写 const 也是它——下面两种写法完全等价：

```
int64 main()
{
    int64 n = 1;
    auto implicit = int64 fn()
    {
        n
    };
    /* 默认：const& */
    auto explicit = int64 fn const()
    {
        n
    };
    /* 写全，同一个意思 */
    n = 2;
    println("{} {}", implicit(), explicit());
    return 0;
}
```

```
2 2
```

两者都跟着外面的新值走（n 改成 2 之后读出来就是 2）。想在闭包里改它，编译器直接拦下：

```
int64 main()
{
    int64 base = 10;
    auto bump = fn(int64 x)
    {
        base += x;
        return base;
    };
    println("{}", bump(1));
    return 0;
}
```

```
magel: 不可变变量（const）不能再赋值：base
```

15.7.2 mut&：写回来源

```
int64 main()
{
    int64 counter = 0;
    auto tick = fn mut()
    {
        counter += 1;
        return counter;
    };
    println("{} {}", tick(), tick());
    println("outside {}", counter);
    return 0;
}
```

```
1 2
outside 2
```

闭包里的 counter 就是外面那个 counter：两次 tick() 累加到 2，外面读出来也是 2。这是写计数器、累加器最直接的写法。

15.7.3 copy：拿一份独立副本

```
int64 main()
{
    int64 n = 5;
    auto independent = int64 fn copy(int64 x)
    {
        n += x;
        return n;
    };
    n = 100;
    println("{} {} {}", independent(2), independent(1), n);
    return 0;
}
```

```
7 8 100
```

闭包从 n = 5 起家，之后外面把 n 改成 100 跟它没关系：它自己从 5 加到 7、再加到 8，外面始终是 100。副本对 String、List 这类带资源的值同样成立——闭包内部改的是自己那一份：

```
int64 main()
{
    String text = "old";
    auto grow = String fn copy()
    {
        text = text + "!";
        return text;
    };
    text = "outside";
    println("{} / {} / {}", grow(), grow(), text);
    return 0;
}
```

```
old! / old!! / outside
```

```
int64 main()
{
    List<int64> values;
    values.push(4);
    auto bump = int64 fn copy()
    {
        values[0] += 1;
        return values[0];
    };
    println("{} {} {}", bump(), bump(), values[0]);
    return 0;
}
```

```
5 6 4
```

copy 还有个额外好处：即使来源是 const 也能拷——拷出来的是一份新值，动不到原值：

```
int64 main()
{
    const int64 fixed = 7;
    auto change = int64 fn copy()
    {
        fixed += 1;
        return fixed;
    };
    println("{} / {}", change(), fixed);
    return 0;
}
```

```
8 / 7
```

15.7.4 move：把来源整个搬进闭包

```
int64 main()
{
    String moved = "payload";
    auto take = String fn move()
    {
        return moved + "-in";
    };
    String moved = "reset";
    println("{} / {}", take(), moved);
    return 0;
}
```

```
payload-in / reset
```

move 之后，原来的 moved 已经被搬空，不能再用——除非重新声明同名变量。上面就是在 move 之后立刻 String moved = "reset"; 创建一个新的同名绑定。

没有重新声明就接着读写旧绑定，编译器会拦：

```
int64 main()
{
    String moved = "payload";
    auto take = String fn move()
    {
        return moved;
    };
    println("{}", moved);
    return 0;
}
```

```
magel: [MGL1205] 借用来源已移动或销毁，不能继续访问（源对象已经销毁）：moved
```

> **提示**
>
> copy 还是 move？之后还要用原值 → copy；原值从此归闭包 → move。对 String / List / 带 Drop 的 struct，move 省一次拷贝，代价是来源作废。

15.8 捕获列表：逐个变量指定方式

fn mut(...) 是”所有外部变量都按 mut& 抓”。想给不同变量不同待遇，就在 fn 后面加一对方括号：

```
int64 main()
{
    int64 a = 2;
    int64 b = 3;
    int64 c = 4;
    int64 d = 5;
    auto mixed = int64 fn copy[a] mut[b] const[c] move[d](int64 x)
    {
        a += x;
        b += x;
        d += x;
        return a + b + c + d;
    };
    a = 100;
    c = 10;
    d = 200;
    println("{} | a={} b={} c={} d={}", mixed(1), a, b, c, d);
    println("{} | b={}", mixed(1), b);
    return 0;
}
```

```
23 | a=100 b=4 c=10 d=200
26 | b=5
```

逐个看：a 拷了一份（外面改 100 不影响它），b 借用（闭包加 1 后外面就是 4），c 只读借用（不能改，但能看到新值 10），d 被搬进闭包（外面重新赋成 200 是另一回事）。

15.8.1 列表后面还能写「其余变量的默认方式」

```
fn copy[a, b] mut(int64 x)
{
    ...
}
/*     ^^^^^^^^^^ ^^^                          */
/*     a、b 按 copy 抓，其余变量按 mut& 抓      */
```

```
int64 main()
{
    int64 left = 1;
    int64 right = 2;
    int64 shared = 3;
    auto f = int64 fn copy[left, right] mut(int64 x)
    {
        left += x;
        right += x;
        shared += x;
        return left + right + shared;
    };
    println("{} | left={} right={} shared={}", f(1), left, right, shared);
    return 0;
}
```

```
9 | left=1 right=2 shared=4
```

left / right 是副本（外面不动），shared 没列出、按默认 mut& 借用，所以闭包加完之后外面读到的 shared 变成了 4。

不写默认方式时，没列出的变量一律按 const& 抓——和全局默认一致。默认方式只能写一次，写两个会报错：

```
int64 main()
{
    int64 a = 1;
    auto f = int64 fn copy[a] mut const()
    {
        a
    };
    println("{}", f());
    return 0;
}
```

```
magel: lambda: lambda 默认捕获方式只能声明一次：mut 与 const
```

方括号也可以是空的——fn copy[](...) 表示”一个都不特殊对待”，其余变量仍按默认 const&：

```
int64 main()
{
    int64 bias = 9;
    auto empty = int64 fn copy[]()
    {
        bias
    };
    bias = 10;
    println("{}", empty());
    return 0;
}
```

```
10
```

传给函数参数的 fn 同样能带捕获列表：

```
int64 apply(int64 fn(int64) f, int64 x)
{
    return f(x);
}
int64 main()
{
    int64 bias = 9;
    println("{} / {}", apply(fn copy[bias](x)
    {
        bias += x;
        return bias;
    }, 2), bias);
    return 0;
}
```

```
11 / 9
```

15.9 复制闭包：拥有状态会复制，借用仍然共享

闭包是值，能赋给另一个变量。那复制过去的到底是什么？

答案是：拥有的东西各复制一份，借的东西还是同一条线。

copy 捕获的闭包复制后，两份副本互不相干：

```
int64 main()
{
    int64 n = 5;
    auto a = int64 fn copy(int64 x)
    {
        n += x;
        return n;
    };
    auto b = a;
    println("b={} a={} b={}", b(3), a(1), b(1));
    return 0;
}
```

```
b=8 a=6 b=9
```

a、b 各有一份从 5 开始的副本：b 先加到 8，a 加到 6，b 再加到 9——三笔独立的账。

而 mut& 捕获的闭包复制后，两个闭包仍指向同一个来源：

```
int64 main()
{
    int64 n = 0;
    auto a = int64 fn mut(int64 x)
    {
        n += x;
        return n;
    };
    auto b = a;
    println("a={} b={} outside={}", a(1), b(1), n);
    return 0;
}
```

```
a=1 b=2 outside=2
```

a 加完 n 是 1，b 接着加变成 2，外面读也是 2——同一个 n，同一条线。

15.10 生命周期：借来的，不能活过来源

const& / mut& 是借用，就有一条铁律：

> **提示**
> 闭包（以及它的任何副本）不能比它借的那个变量活得更久。

最容易踩的地方是把闭包返回出去：

```
int64 fn() counter()
{
    int64 n = 4;
    return int64 fn mut()
    {
        n += 1;
        return n;
    };
}
int64 main()
{
    auto c = counter();
    println("{}", c());
    return 0;
}
```

```
magel: [MGL1204] 返回值中的引用不能超过局部来源的生命周期
```

n 是 counter 的局部变量，函数一返回它就没了；闭包要是带走了它的引用，调用时访问的就是一块已经死掉的内存。编译器不放行（报错里还会附上那个来源的名字）。

换成 copy 或 move，闭包自己拥有一份，就能安全返回：

```
int64 fn() counter()
{
    int64 n = 4;
    return int64 fn copy()
    {
        n += 1;
        return n;
    };
}
int64 main()
{
    auto c = counter();
    println("{} {}", c(), c());
    return 0;
}
```

```
5 6
```

move 同理——把 n 搬进闭包，闭包从此自带状态：

```
int64 fn() counter()
{
    int64 n = 8;
    return int64 fn move()
    {
        n += 1;
        return n;
    };
}
int64 main()
{
    auto c = counter();
    println("{} {}", c(), c());
    return 0;
}
```

```
9 10
```

判断口诀：闭包要返回 → 捕获只能是 copy / move，或者是借” 调用方” 的东西。

最后半句正是 15.14 节” 闭包当工厂” 的做法：借用调用方传进来的指针，那个变量活在外面，闭包带回去才安全。

15.11 带 Drop 的类型：谁负责销毁

如果捕获的是带 Drop 的类型（有 void drop() 的 struct），所有权规则就直接体现为什么时候调用 drop()。

copy 捕获 → 外面那个值和闭包里的副本是两个对象，各销毁一次：

```
int64 drops = 0;
struct Record: Drop
{
    String text;
    List<int64> values;
    void drop()
    {
        drops += 1;
    }
};
int64 main()
{
    Record record;
    record.text = "owned";
    record.values.push(1);
    {
        auto own = int64 fn copy()
        {
            record.values[0] += 1;
            return record.values[0];
        };
        println("own={}", own());
        {
            auto second = own;
            println("second={} drops={}", second(), drops);
        }
        println("drops={}", drops);
    }
    println("drops={} outside={}", drops, record.values[0]);
    return 0;
}
```

```
own=2
second=3 drops=0
drops=1
drops=2 outside=1
```

second 出作用域销毁一份副本（drops=1），own 出作用域再销毁一份（drops=2）。全程外面 record.values[0] 一直是 1——副本里加多少都影响不到它。

move 捕获 → 对象被搬进闭包，只有闭包销毁时才 drop()：

```
int64 drops = 0;
struct Record: Drop
{
    String text;
    List<int64> values;
    void drop()
    {
        drops += 1;
    }
};
int64 main()
{
    Record record;
    record.text = "moved";
    record.values.push(10);
    {
        auto own = int64 fn move()
        {
            record.values[0] += 1;
            return record.values[0];
        };
        println("own={} drops={}", own(), drops);
    }
    println("drops={}", drops);
    return 0;
}
```

```
own=11 drops=0
drops=1
```

闭包出作用域 → 它拥有的那个 Record 被销毁 → drops 从 0 变 1，全程只有一次。

15.12 捕获方式速查

写法	闭包内能改？	外面跟着变？	来源还能用？	能返回出去？
`fn(...)` / `fn const(...)`	否	—	能	否（借用）
`fn mut(...)`	是	是	能	否（借用）
`fn copy(...)`	是	否	能	是
`fn move(...)`	是	否	需重新声明	是
`fn copy[a] mut[b] const[c] move[d](...)`	各按各的	各按各的	各按各的	只要不含借用

写法	闭包内能改？	外面跟着变？	来源还能用？	能返回出去？
`fn copy[a, b] mut(...)`	其余按 `mut&`	—	—	否（含借用）

容易忘的几条：

- 默认就是 `const&`——不写 `mut` 就改不了外面的变量。
- `mut` 改的是外面那个，不是副本；想各算各的用 `copy`。
- `move` 之后来源作废，须重新声明新绑定，普通赋值不能恢复。
- 借用的闭包不能返回；要返回就用 `copy` / `move`，或借调用方的指针。
- 复制闭包：拥有的状态各复制一份，借用的线仍指向同一来源。

15.13 一个小坑：表达式体 vs 语句体

`fn` 的体有两种写法，差别就一个分号：

```
auto f = fn(int64 x)
{
    x * 2
};
// 表达式体：没有分号
auto g = fn(int64 x)
{
    return x * 2;
};
// 语句体：有分号 + return
```

表达式体是「一个表达式」，直接就是返回值——末尾不能带分号。一旦你手滑写成 `{ x * 3; }`（带分号），它就成了一「条语句」，编译器会要求你补 `return`：

```
int64 main()
{
    auto g = int64 fn(int64 x)
    {
        x * 3;
    };
    println("{}", g(5));
    return 0;
}
```

```
magel: 返回类型为 int64 的函数 __magel_lambda_… 不是所有路径都有 return
```

记住一句话：体里想要「最后一行的值」当返回值 → 别加分号；想用 `return` → 加分号。

15.14 闭包当工厂：返回一个函数

15.10 节说过：借用的闭包不能返回。想让工厂函数产出一个闭包，捕获来源就得由调用方提供——让调用方把变量的地址传进来，那个变量活在外面，闭包把它带回去才安全：

```
int64 fn(int64) makeAdder(int64 mut& amount)
{
    return fn(int64 value)
    {
        value + amount
    };
}
int64 main()
{
    int64 one = 1;
    int64 ten = 10;
    auto plus1 = makeAdder(&one);
    auto plus10 = makeAdder(&ten);
    println("{} {}", plus1(5), plus10(5));
    return 0;
}
```

```
6 15
```

`makeAdder(&one)` 和 `makeAdder(&ten)` 返回的 `plus1` / `plus10`，各自借着 `one` / `ten`：只要这两个变量还在，两个闭包就一直能用；它们要是先没了，闭包也就跟着失效——这正是 15.10 那条铁律的另一面。

函数值本身可以复制；复制闭包时到底复制了什么，见 15.9 节。

15.14.1 移动后访问诊断

下面的闭包将 a 的所有权搬进自身，创建之后读取源变量 a 会报 MGL1205：

```
int64 main()
{
    String a = "1";
    String fn() b = fn move()
    {
        a
    };
    println("{}", a);
    return 0;
}
```

如果还要保留 a，就按值复制捕获；如果使用 move，之后通过 b() 读取闭包拥有的值。

```
int64 main()
{
    String a = "1";
    String fn() b = fn copy()
    {
        a
    };
    println("{} {}", a, b());
    return 0;
}
```

```
1 1
```

编辑器对捕获位置的 move/copy/const/mut 显示对应说明，不套用普通 `move<T>` 函数签名；诊断红线标到后续读取的变量。

15.15 语句级具名闭包

如果你希望给局部闭包直接起名字，可以写成 返回类型 名字 （参数） { ... }。它仍是函数值，只在声明语句位置识别，不能塞进另一个表达式中间。返回类型是检查函数体的类型锚点。

```
int64 fn() makeNamedCounter()
{
    int64 current = 0;
    int64 next() move
    {
        current++;
        return current;
    }
    return next;
}

int64 main()
{
    int64 base = 10;
    int64 read()
    {
        return base;
    }
    int64 increment() mut
    {
        base++;
        return base;
```

```
    }
    int64 isolated() copy
    {
        base += 100;
        return base;
    }
    int64 initial = read();
    int64 changed = increment();
    int64 snapshot = isolated();
    println("{} {} {} {}", initial, changed, snapshot, base);

    auto counter = makeNamedCounter();
    int64 first = counter();
    int64 second = counter();
    println("{} {}", first, second);

    int64 callback(int64 value);
    callback = int64 fn(int64 value)
    {
        return value + 1;
    };
    println("{}", callback(5));
    return 0;
}
```

```
10 11 110 11
1 2
6
```

默认捕获方式是 const&，因此 read 能看到来源后续的变化，却不能修改来源。函数体前显式写 mut 是可写借用，copy 拥有独立状态，move 则接管捕获值的所有权。捕获状态在声明初始化时交给函数名绑定；普通后续赋值仍遵守值语义。

局部 int64 callback(int64 value); 只声明一个有确定签名的函数变量，后面再赋函数值；它与模块层的宿主函数声明不是同一个语义入口。借用捕获的闭包不能逃出来源的生命周期。工厂函数中的 next 使用 move 拥有状态，所以可安全返回；把它改成默认借用并返回会报错。

15.16 多层高阶闭包：函数也可以接收函数

R fn(A) 表示“接收 A、返回 R 的函数值”。这里的 A 也可以是函数类型，因此 fn 可以继续嵌套。下面保留五层嵌套的完整写法，并通过参数名和换行区分两个实参：

```
int64 hello(
    int64 fn(int64 fn(int64 fn(int64 fn(int64 fn(int64))))) pipeline,
    int64 fn(int64) finish)
{
    return finish(pipeline(level4));
}
```

这里是具名函数声明；有函数体时，模块层声明在右大括号结束，后面不加分号。形参要有名字，函数体才能调用它们。`fn` 后的小括号属于函数类型，不是一次函数调用。

由内向外阅读第一项形参的类型：

层次	接收什么	返回什么	下方程序中的值
第一层	`int64`	`int64`	`level1`
第二层	第一层函数	`int64`	`level2`
第三层	第二层函数	`int64`	`level3`
第四层	第三层函数	`int64`	`level4`
第五层	第四层函数	`int64`	`level5`

这些层次表示参数类型的嵌套深度，不表示连续调用五次后才得到一个返回函数；表中每一层的返回类型都是 `int64`。`hello` 的第一项实参是第五层函数，第二项实参是普通的一元闭包。

完整程序如下。最后的 `finish` 捕获了 `main` 中的 `factor`，因此这同时展示了高阶函数参数与带状态的闭包：

```
int64 level1(int64 value)
{
    return value + 1;
}

int64 level2(int64 fn(int64) callback)
{
    return callback(10);
}

int64 level3(int64 fn(int64 fn(int64)) callback)
{
    return callback(level1);
}

int64 level4(int64 fn(int64 fn(int64 fn(int64))) callback)
{
    return callback(level2);
}

int64 level5(int64 fn(int64 fn(int64 fn(int64 fn(int64)))) callback)
```

```
{
    return callback(level3);
}

int64 hello(
    int64 fn(int64 fn(int64 fn(int64 fn(int64 fn(int64))))) pipeline,
    int64 fn(int64) finish)
{
    return finish(pipeline(level4));
}

int64 main()
{
    int64 factor = 2;
    int64 finish(int64 value)
    {
        return value * factor;
    }
    println("{}", hello(level5, finish));
    return 0;
}
```

```
22
```

调用顺序是 `pipeline(level4)` → `level4(level3)` → `level3(level2)` → `level2(level1)` → `level1(10)`，先得到 11；随后 `finish(11)` 读取捕获的 `factor`，得到 22。具名函数和闭包都按各自的完整函数类型检查，某一层参数类型不匹配就不能传入。

`finish` 默认以 `const&` 借用 `factor`。这次调用在 `main` 的作用域内同步完成，借用始终有效。若要把这样的闭包返回或存到更长寿命的对象中，仍须满足前面介绍的捕获与生命周期规则；嵌套 `fn` 类型不会延长被借用变量的寿命。

15.17 组合闭包：接收函数，再返回函数

高阶函数还可以把几个操作组合成新的函数值。下面的 `combine` 接收两个 `int64 fn(int64)`，返回的类型也是 `int64 fn(int64)`：

```
auto combine = (int64 fn(int64)) fn(int64 fn(int64) a, int64 fn(int64) b)
{
    int64 fn copy(int64 x)
    {
        a(b(x))
    }
};
```

从左到右拆开这个声明：括号里的 int64 fn(int64) 是返回类型，接下来的 fn(...) 才是创建组合器的匿名函数。它的函数体每次创建一个新闭包，先调用 b，再把结果交给 a。内层的 a(b(x)) 与外层的闭包表达式都处在块末尾且不加分号，分别成为两层函数的返回值。

返回函数类型外面的括号用于分组，不会增加一层函数类型；也可以直接写 int64 fn(int64) fn(...)。参数列表最后一个参数后面不加逗号。与上一节的具名函数定义不同，这里是变量初始化语句，最外层右大括号后面需要分号。

完整程序如下：

```
int64 main()
{
    auto combine = (int64 fn(int64)) fn(int64 fn(int64) a, int64 fn(int64) b)
    {
        int64 fn copy(int64 x)
        {
            a(b(x))
        }
    };

    auto f1 = int64 fn(int64 x) { x + 1 };
    auto f2 = int64 fn(int64 x) { x * 2 };
    auto c1 = combine(f1, f2);
    auto c2 = combine(f2, f1);
    println("{} {}", c1(5), c2(5));

    auto c3 = combine(fn(x) { x * 5 }, fn(x) { x - 1 });
    println("{}", c3(5));
    return 0;
}
```

```
11 12
20
```

c1(5) 先执行 f2(5) 得到 10，再执行 f1(10) 得到 11；c2(5) 先执行 f1(5) 得到 6，再执行 f2(6) 得到 12。两次调用各自返回一个拥有 a、b 副本的组合闭包，交换实参会改变组合顺序。

内层函数不能默认借用本次调用的局部形参 a、b 后直接返回；这些形参会随 combine 返回而离开作用域。

fn copy 使返回闭包拥有这两个函数值的副本。本例的 f1、f2 都没有捕获外部变量；如果它们含有借用，复制函数值不会把被借用对象变成自有状态，其来源仍必须活得足够久。

c3 直接把两个匿名函数作为实参传入。目标形参 int64 fn(int64) 为 fn(x) 提供参数类型与返回类型的上下文，因此两个 x 都推断为 int64，不必重复写类型。组合顺序仍然是先执行第二项实参，再执行第一项实参：(5 - 1) * 5 = 20。这些匿名函数没有捕获状态，传入后同样由组合闭包持有其副本。

15.18 小结

你会了	关键点
函数类型	R fn(A, B)，和别的类型一样能当变量/字段/参数
函数当值	int64 fn(int64, int64) op = add;，可重新指向
传函数	把函数名或 fn 当实参传，实现回调
fn	返回类型 fn(参数) { 体 }，返回类型可省
上下文推断	目标类型已知时，参数类型也能省
闭包	记住定义处的变量；默认 const& 只读借用
四种捕获	const& 只读、mut& 写回、copy 独立副本、move 转移所有权
捕获列表	fn copy[a] mut[b] const[c] move[d](...)；列表后可加默认方式
复制闭包	拥有状态各复制一份，借用仍指向同一来源
生命周期	借用的闭包（及其副本）不能比来源活得更久
返回闭包	只有 copy / move、或借调用方的指针，才能安全带出函数
Drop	copy 两份各销毁一次，move 只在闭包销毁时销毁一次
体例坑	表达式体别加分号；用 return 就要加分号
工厂	返回闭包，配不同的「料」造出不同的函数

15.19 下一步

filter、map 这类「把函数当参数」的操作，写起来越来越顺手了。但你自己写 filter 还得手搓 List、手搓 foreach——一不小心就写出一堆模板代码。

第 16 章讲错误处理，先把「出错了怎么办」这件事说清楚；之后第六部分的迭代器，会把 map / filter 这些套路做成一条能直接用的链。

翻到第 16 章错误处理。

第 16 章 错误处理

前面十几章，代码都假定「一切顺利」：输入是合法的、列表不是空的、除法不会除以零。真实程序里这些假定经常不成立。这一章讲 Magel 怎么处理「事情没按预期走」。

先说结论：Magel 没有异常。没有 `try` / `catch` / `throw`。出错有两条路——「可能空」用 `Option`、「可能失败」用 `Result`，两者都放在返回值里，由调用者显式处理。

16.1 为什么不用异常

多数语言用异常：调用一个函数，它可能半路 `throw`，控制流突然跳到别处。问题是你看不出一个函数会不会抛——签名里没写，得读文档或者读源码。

Magel 把「可能失败」这件事写进类型里：返回 `Option<T>` 就表示「可能没有值」，返回 `Result<T, E>` 就表示「可能失败，且带一个原因 `E`」。失败变成了一种普通的返回值。

好处是你不会忘。函数签名上写着 `Option<…>`，你就知道必须处理「没有」的情况，编译器会盯着你。

16.2 `Option<T>`：可能没有值

`Option<T>` 只有两种状态：`Value(v)`（有个值）或者 `None`（没有）。拿「找第一个等于目标的元素」举例——找到了有值，没找到就是「没有」：

```
Option<int64> findFirst(List<int64> xs, int64 target)
{
    foreach (int64 x in xs)
    {
        if (x == target)
        {
            return Option<int64>::Value(x);
        }
    }
    return Option<int64>::None;
}

int64 main()
{
    List<int64> xs;
    xs.push(3);
    xs.push(7);
    println("{}", findFirst(xs, 7).isValue());
    println("{}", findFirst(xs, 9).isNone());
    println("{}", findFirst(xs, 7).unwrap());
    println("{}", findFirst(xs, 9).unwrapOr(-1));
    return 0;
}
```

```
1
1
7
-1
```

- 造值：`Option<int64>::Value(x)`，空：`Option<int64>::None`；
- `isValue()` / `isNone()` 是 `bool`，问「有没有」；
- `unwrap()` 取值——空的时候它会 panic（见 16.4）；
- `unwrapOr(-1)` 取值，但空的时给个兜底值——这是最常用的一个。

> **提示**
>
> `Option` 和 `Result` 都属于核心模块 `lang`，不用 `import`，拿来就用。

16.3 Result<T, E>：可能失败，还带原因

`Option` 只说「有没有」，不说「为什么没有」。要带原因就用 `Result<T, E>`：`Ok(v)` 表示成功、带着结果；`Err(e)` 表示失败、带着原因 `e`（`e` 可以是任意类型）。

```
Result<int64, String> safeDiv(int64 a, int64 b)
{
    if (b == 0)
    {
        return Result<int64, String>::Err("除以零");
    }
    return Result<int64, String>::Ok(a / b);
}

int64 main()
{
    println("{}", safeDiv(10, 2).isOk());
    println("{}", safeDiv(10, 0).isErr());
    println("{}", safeDiv(10, 2).unwrap());
    return 0;
}
```

```
1
1
5
```

- `Ok(v)` / `Err(e)` 造两个状态；
- `isOk()` / `isErr()` 判断；`unwrap()` / `unwrapOr(…)` / `expect(…)` 取值；
- 出错原因 `E` 类型随便挑：字符串、错误码、甚至一个自己的枚举。

`Option` 与 `Result` 的关系：`Option<T>` 差不多就是 `Result<T, ()>`——「失败了，但没什么可说的」。要带原因，就用 `Result`。

把两个状态都处理干净，用第 10 章的 `match`：

```
int64 main()
{
    Result<int64, String> r = Result<int64, String>::Err("坏了");
    match (r)
    {
        Result<int64, String>::Ok(v) =>
        {
            println("ok {}", v);
        }
        Result<int64, String>::Err(e) =>
        {
            println("err {}", e);
        }
    }
    Option<int64> o = Option<int64>::Value(3);
    match (o)
    {
        Option<int64>::Value(v) =>
        {
            println("v={}", v);
        }
        Option<int64>::None =>
        {
            println("none");
        }
    }
    return 0;
}
```

```
err 坏了
v=3
```

match 是唯一能安全取出载荷的办法——它逼你把两个分支都写全，不像 unwrap 那样偷偷 panic。

16.4 四个「取出来」的工具，别用错场合

拿到一个 Option / Result，想取出里面的值，有四种办法：

写法	空 / 失败时	什么时候用
`unwrap()`	panic，程序中止	你 * 确信 * 不可能是空（比如刚判过）
`expect("…")`	panic，带上你的话	同上，但想留一句解释
`unwrapOr(fallback)`	返回 `fallback`	空是正常情况，兜个默认值
`match`	两个分支都走	想分别处理（最稳）

`unwrap` 用错了，程序会在运行期直接崩：

```
int64 main()
{
    Option<int64> none = Option<int64>::None;
    println("{}", none.unwrap());
    return 0;
}
```

```
magel: 运行期错误: panic: called Option.unwrap() on None
出错位置: <内建 lang>:122（__magel_match_next_59）
调用栈（外 → 内）:
  main  hello.mgl:3 [thread 0 coroutine 0 ip 40]
```

`expect` 一样会 panic，只是消息换成你写的那句：

```
int64 main()
{
    Option<int64> none = Option<int64>::None;
    println("{}", none.expect("必须要有值"));
    return 0;
}
```

```
magel: 运行期错误: panic: 必须要有值
```

一句话记法：`unwrap` / `expect` 是「你说它不可能是空」的赌注。只要那份自信的来源不是刚刚检查过，就用 `unwrapOr` 或 `match`。

16.5 ?：失败就自动往上抛

一层层写 `match` 太啰嗦。假设你要算 `half(half(n))`，每次都要检查中间结果有没有值：

```
Option<int64> quarter(int64 n)
{
    Option<int64> h = half(n);
    if (h.isNone())
    {
        return Option<int64>::None;
    }
    // 手写传播
    Option<int64> q = half(h.unwrap());
    if (q.isNone())
    {
        return Option<int64>::None;
    }
    // 又来一遍
    return q;
}
```

`?` 运算符把这个套路压成一行：「如果没值 / 失败了，就地返回同样的空」。

```
Option<int64> half(int64 n)
{
    if (n % 2 == 0)
    {
        return Option<int64>::Value(n / 2);
    }
    return Option<int64>::None;
}

Option<int64> quarter(int64 n)
{
    int64 h = half(n)?;
    return half(h);
}

int64 main()
{
    println("{}", quarter(8).unwrap());
    println("{}", quarter(4).isNone());
    println("{}", quarter(6).isNone());
    return 0;
}
```

```
2
0
1
```

`int64 h = half(n)?;` 读作：

- `half(n)` 有值 → `?` 把值取出来赋给 `h`，继续往下；
- `half(n)` 是 `None` → 整个 `quarter` 立刻返回 `None`，后面的代码不执行。

`?` 对 `Result` 一样管用，而且会把 `Err` 原样往上传：

```
Result<int64, String> safeDiv(int64 a, int64 b)
{
    if (b == 0)
    {
        return Result<int64, String>::Err("除以零");
    }
    return Result<int64, String>::Ok(a / b);
}

Result<int64, String> chain(int64 a, int64 b, int64 c)
{
    int64 q = safeDiv(a, b)?;
    return safeDiv(q, c);
}

int64 main()
{
    println("{}", chain(100, 5, 2).unwrap());
    println("{}", chain(100, 0, 2).isErr());
    return 0;
}
```

```
10
1
```

用 `?` 有个硬条件：所在函数的返回类型必须接受操作数的失败载荷。标准 Option/Result 按各自约定接收，错误类型不匹配时需显式映射。否则编译器拦下：

```
Option<int64> half(int64 n)
{
    if (n % 2 == 0)
    {
        return Option<int64>::Value(n / 2);
    }
    return Option<int64>::None;
}

int64 quarter(int64 n)
{
    int64 h = half(n)?;
    return h;
}

int64 main()
{
    println("{}", quarter(8));
    return 0;
}
```

```
magel: 使用 `?` 的函数也必须返回同样的可空枚举，现在返回 int64
```

道理很简单：? 可能要把 None 往外抛，那外层函数必须接得住——它得也返回 Option<int64>。想用 ? 又不想让 quarter 变成 Option，是行不通的。

16.6 panic 和 assert：什么时候该「直接崩」

上面都是「可恢复的错误」——调用者能处理。但有些情况属于程序写错了，不该继续跑：数组越界、不变量被破坏、参数是 null 级别的荒谬……这时候就该立刻中止，别带着坏状态往下走。

- panic(" 消息") 无条件中止，打印消息；
- assert(条件, " 消息") 条件不成立就 panic。消息可省。

```
int64 mustPositive(int64 n)
{
    if (n <= 0)
    {
        panic("必须是正数");
    }
    return n;
}

int64 main()
{
    println("{}", mustPositive(5));
    return 0;
}
```

```
5
```

assert 失败的样子：

```
int64 main()
{
    int64 x = -1;
    assert(x > 0, "x 必须为正");
    println("不会到这");
    return 0;
}
```

```
magel: 运行期错误: panic: x 必须为正
出错位置: <内建 lang>:111（assert$bool$string）
调用栈（外 → 内）:
  main  hello.mgl:3 [thread 0 coroutine 0 ip 13]
```

怎么选？

情况	用什么	理由
输入不合法、文件打不开、找不到元素	`Option` / `Result`	调用者能处理，往上交
逻辑里「绝不该发生」的事	`panic` / `assert`	发生了就是 bug，早崩早发现

`assert` 是开发期的工具：它帮你抓「我以为是这样，其实是那样」。上线时可以保留（成本很低），也可以视情况删掉——但别拿它处理正常业务错误。

16.6.1 用户自定义传播

`?` 通过 `Try<T,R>.branch` 判断是否继续，通过返回类型的 `FromResidual<F,R>.fromResidual` 将失败载荷转为函数结果，F 为实现类型自身。`TryBranch<T,R>` 有 Continue(T) 和 Break(R) 两种变体。

```
enum Step: Try<int64, NoneResidual>
{
    Value(int64 value), Empty;
    TryBranch<int64, NoneResidual> branch()
    {
        match (self)
        {
            Step::Value(value) =>
            {
                return TryBranch<int64, NoneResidual>::Continue(value);
            }
            Step::Empty =>
            {
                return TryBranch<int64, NoneResidual>::Break(NoneResidual::None);
            }
        }
        panic("unreachable");
    }
};
Option<int64> addOne(Step step)
{
    int64 value = step?;
    return Option<int64>::Value(value + 1);
}
int64 main()
{
    println("{} {}", addOne(Step::Value(4)).unwrap(), addOne(Step::Empty).isNone());
    return 0;
}
```

```
5 1
```

这里继续值为 int64，失败载荷为 NoneResidual，所以标准 Option 可以接住失败；自定义返回类型也可实现 FromResidual。传播表达式求值一次，失败就立即返回，并执行已登记的作用域清理。不同 Result 错误类型可先用 `mapErr` 显式转换；Option 转 Result 可用 `okOr` 或 `okOrElse`。

16.7 小结

你会了	关键点
没有异常	用返回值表达失败，签名上看得见
`Option<T>`	`Value(v)` / `None`，`isValue` / `isNone`
`Result<T, E>`	`Ok(v)` / `Err(e)`，`isOk` / `isErr`，带原因
取值	`unwrap` / `expect`（空则 panic）、`unwrapOr`（兜底）、`match`（最稳）
`?`	空 / 失败就地返回；操作数实现 Try，返回类型实现对应的 FromResidual
`panic`	无条件中止，用于「不该发生」
`assert`	条件不成立就 panic，抓开发期的 bug
选型	可恢复 → `Option`/`Result`；不该发生 → `panic`/`assert`

16.8 下一步

第五部分讲完了：接口、泛型、函数值、错误处理——你有了写「通用」代码的全套家伙。

接下来的第六部分转向工程与工具链：怎么把程序组织成模块、怎么用 `magel new` 起工程、怎么跑测试。第一个是迭代器——它会把你第 15 章手写的 `filter`，变成一条能随意拼的链。

翻到第 17 章迭代器。

第 17 章 迭代器

第 15 章你手写过一个 filter：自己声明 List、自己 foreach、自己 push。能跑，但每换一种操作——求平方、取前几个、求和——都得再手搓一遍循环。

这一章讲 std::iter：它把这些套路做成了能随意拼的链。写起来像说话，读起来也像说话：

```
std::iter::range(1, 100).filter(是偶数).map(乘十).take(3).collect()
```

17.1 一条链由三段组成

迭代器的写法永远是同一个形状：

```
起点.中间操作.中间操作.….终结操作
```

- 起点：从哪儿开始——一个区间、一个列表；
- 中间操作：map / filter / take……可以串好几个；
- 终结操作：collect / fold / count……干活的是它。

要 import：

```
import std::iter;
```

17.2 起点：range 和 List.iter()

最常用的起点是 std::iter::range(start, stop)——从 start 到 stop（不含 stop）：

```
import std::iter;

int64 main()
{
    auto xs = std::iter::range(1, 6).collect();
    println("{} {}", xs.len(), xs[4]);
    return 0;
}
```

```
5 5
```

range(1, 6) 数是 1、2、3、4、5——和切片一样左闭右开（第 12 章）。collect() 把它们收进一个 List<int64>。

还有一个重载可以指定步长：range(start, stop, step)。

另一种起点是从现成的容器出发：列表.iter()。稍后会讲它和 range 的一个区别。

17.3 中间操作：map / filter / take / skip / rev

map——对每一项做同一个变换：

```
import std::iter;

int64 main()
{
    auto ys = std::iter::range(1, 6).map(fn(int64 x)
    {
        x * x
    }).collect();
    println("{} {} {}", ys[0], ys[1], ys[4]);
    return 0;
}
```

```
1 4 25
```

filter——只留下满足条件的：

```
import std::iter;

int64 main()
{
    auto evens = std::iter::range(1, 11)
    .filter(fn(int64 x)
    {
        x % 2 == 0
    })
    .collect();
    println("{} -> {} {} {}", evens.len(), evens[0], evens[1], evens[4]);
    return 0;
}
```

```
5 -> 2 4 10
```

take(n) / skip(n) 取前 n 个 / 跳过前 n 个；rev() 反转。它们和 map / filter 一样，都是「中间操作」，返回的还是迭代器，可以继续往下接。

17.4 终结操作：collect / fold / count

中间操作只是搭管道，真正让数据流动的是终结操作。常用的三个：

- collect() ——收成一个 List；

- `count()` ——数一共多少个；
- `fold(初值, 函数)` ——从头到尾累积，最灵活。

`fold` 是「求和、求积、求最大」这类操作的通用形式。求和：

```
import std::iter;

int64 main()
{
    int64 s = std::iter::range(1, 101).fold(0, fn(int64 acc, int64 x)
    {
        acc + x
    });
    println("{}", s);
    println("{}", std::iter::range(1, 101).count());
    return 0;
}
```

```
5050
100
```

`fold(0, …)` 里那个 0 是初值 `acc`，然后每来一个 `x`，用函数算出一个新的 `acc`：`0 → 0+1 → 1+2 → …`。想求最大值，初值给一个极小值，函数换成「取较大的」就行。

还有几个「问一句就走」的终结操作：

```
import std::iter;

int64 main()
{
    println("{}", std::iter::range(1, 6).all(fn(int64 x)
    {
        x > 0
    }));
    println("{}", std::iter::range(1, 6).any(fn(int64 x)
    {
        x > 3
    }));
    println("{}", std::iter::range(1, 6).find(fn(int64 x)
    {
        x % 2 == 0
    }).unwrap());
    return 0;
}
```

```
1
1
2
```

- `all(pred)` 全满足吗；`any(pred)` 有满足的吗——都返回 `bool`；
- `find(pred)` 返回第一个满足的（`Option`，第 16 章——所以后面能接 `.unwrap()`）。

17.5 串成一条链

现在把中间操作串起来看威力——「1 到 99 里，乘 2 之后能被 3 整除的前 3 个」：

```
import std::iter;

int64 main()
{
    auto ys = std::iter::range(1, 100)
    .map(fn(int64 x)
    {
        x * 2
    })
    .filter(fn(int64 x)
    {
        x % 3 == 0
    })
    .take(3)
    .collect();
    println("{} {} {} {}", ys.len(), ys[0], ys[1], ys[2]);
    return 0;
}
```

```
3 6 12 18
```

一行一句、从上往下读，和需求一一对应：`map` 乘 2、`filter` 要 3 的倍数、`take(3)` 取三个、`collect` 收起来。没有 `foreach`、没有下标、没有临时变量。

17.6 惰性：写了 ≠ 跑了

这是迭代器最容易误解的地方。看下面这段：

```
import std::iter;

int64 seen = 0;

void note(int64 x)
{
    seen = seen + 1;
}

int64 main()
{
    auto lazy = std::iter::range(1, 6)
    .inspect(note)
    .map(fn(int64 x)
    {
        x * 10
    });
    println("{}", seen);

    auto got = lazy.collect();
    println("{} {}", seen, got[0]);
    return 0;
}
```

```
0
5 10
```

`inspect` 是个「顺手看一眼」的中间操作——每过一项就调用一次 `note`。可注意：

- 第一次打印 `seen` 是 0——虽然链子搭好了，一项都还没走；
- 直到 `.collect()` 那一刻，才真的从头跑到尾，`seen` 变成 5。

中间操作是惰性的：不接终结操作，它们一行都不执行。这条设计有两个好处：

1. 省事——`take(3)` 时，源数据只要产出 3 个就够了，后面的根本不碰；
2. 能做无限序列——`std::iter::repeat(x)` 可以永远产下去，只要你用 `take` 限住。

> **提示**
> 记住一句话：迭代器链只有碰到 `collect` / `fold` / `count` 这类终结操作才会开跑。

17.7 一个坑：List.iter() 的元素是引用

`range` 产出的是实打实的 `int64`，所以 `fn(int64 x)` 就够了。但从列表出发时，`列表.iter()` 产出的是元素的只读引用，fn 的参数得写成引用：

```
import std::iter;

int64 main()
{
    List<int64> xs;
    xs.push(10);
    xs.push(20);
    xs.push(30);
    int64 total = xs.iter().fold(0, fn(int64 acc, int64 const& x)
    {
        acc + x
    });
    println("{}", total);
    return 0;
}
```

```
60
```

写 `fn(int64 acc, int64 x)` 会报「类型实参推导冲突」——因为元素实际是 `int64 const&`。这是借用检查器在提醒你：`.iter()` 借用了那个列表，而不是把它复制一份（省内存，也符合第 7 章「引用」那一套）。

要原地改元素，用 `.iterMut()`——它产出可写引用 `T mut&`（第 7 章）：

```
import std::iter;

void bump(int64 mut& x)
{
    x = x * 10;
}

int64 main()
{
    List<int64> xs;
    xs.push(1);
    xs.push(2);
    xs.push(3);
    xs.iterMut().forEach(bump);
    println("{} {} {}", xs[0], xs[1], xs[2]);
    return 0;
}
```

```
10 20 30
```

`bump` 拿到的是可写引用，所以它改的就是列表里的原件。

不想借、想要一份独立的元素值，用 .intoIter()——它拿走列表的元素，fn 参数写 int64 就行。

你想要	用	元素类型
只读遍历	xs.iter()	T const&
边走边改元素	xs.iterMut()	T mut&
拿走元素	xs.intoIter()	T

17.8 foreach：把链条跑起来

前面几节一直在”造一条链”，真正让它跑起来的是 foreach。三种写法：

```
foreach (int64 a in list)
{
    ...
}
/* 写上类型 */
foreach (a in list)
{
    ...
}
/* 省略类型，按元素推 */
foreach (int64 mut& p in span)
{
    ...
}
/* 引用形态：拿到元素地址 */
```

能当迭代源的有：数组、List<T>、std::memory::Span<T>、自己写的 Iterator<T>，以及整数区间（见 17.10）。break / continue 照常可用：

```
int64 partial = 0;
foreach (int64 i in [0, 10])
{
    if (i == 3)
    {
        continue;
    }
    /* 跳过 3 */
    if (i == 6)
    {
        break;
    }
    /* 到 6 就收工 */
    partial = partial + i;
}
println("break/continue: {}", partial);
```

```
break/continue: 12
```

0 + 1 + 2，跳过 3，再加 4、5，撞上 6 退出——一共 12。

17.9 foreach 拿到的是元素引用（最容易踩的一个坑）

循环变量不是副本，它是元素本身。给它赋值会直接写回容器：

```
import std::collections;

int64 main()
{
    int64 raw[] = new[3];
    raw[0] = 1;
    raw[1] = 2;
    raw[2] = 3;
    foreach (int64 a in raw)
    {
        a = a * 10;
    }
    println("array: {} {} {}", raw[0], raw[1], raw[2]);

    List<int64> list;
    list.push(4);
    list.push(5);
    foreach (int64 a in list)
    {
        a = a + 1;
    }
    println("list: {} {}", list[0], list[1]);
    return 0;
}
```

```
array: 10 20 30
list: 5 6
```

不少语言里 `for (x in xs)` 的 `x` 是一份拷贝，改它不影响容器——Magel 正好相反。想”只读地看一遍”就别给它赋值；想原地改，这反而是最省事的写法，连下标都不用写。

17.9.1 mut&：拿地址，而不是元素

`Span<T>` 这类”引用 + 长度”的视图，迭代器返回的是 `Option<T mut&>`。这时用 `T mut&` 形态能直接拿到元素地址，改结构体字段特别顺手：

```
std::memory::Span<int64> span = std::memory::Span<int64>::from(raw, len(raw));
foreach (int64 mut& p in span)
{
    * p = *p + 1;
}
```

`*p = *p + 1` 写回原缓冲区，不复制元素；结构体就写 `p->weight = ...`。反过来，值容器用值形态就够了——`List<int64>` 的元素是 `int64` 而不是引用，对它写 `mut&` 编译器会直接拦下来：

```
List<int64> list;
list.push(1);
foreach (int64 mut& x in list)
{
}
```

```
magel: foreach 变量类型与 Vector<T> 元素不一致：写的是 int64 mut&，容器元素是 int64
```

判断方法就一句：你要改的是”元素的值”还是”元素本身的位置”——前者写值形态，后者（指针视图、想省掉一次拷贝）才用 mut&。

17.10 foreach 整数区间

foreach (uint8 x in [0,255]) { ... } 依次产生 0 到 255，共 256 项；[0,255) 不包含 255。支持 int8/uint8/int16/uint16/int32/uint32/int64/uint64。省略变量类型时按端点整型推导，普通整数字面量默认 int64。

区间在内部表示为惰性 Iterator<T>，直接复用 foreach 的迭代协议，不创建列表。两个端点从左到右各求值一次，并且两个端点都必须在 T 的取值范围内，包括半开区间的上端点。常量错误在编译期拒绝，动态端点在循环前检查。int8 的范围是 -128..127，所以 foreach(int8 x in [0,255]) 会报错。反向区间 a>b 报错；[a,a) 是空区间，[a,a] 恰好产生一项。

闭区间到达上界后直接结束，不计算“最大整数 + 1”，因此 uint8 255 和 uint64 18446744073709551615 均可作为合法闭上界。循环变量是每轮取得的值，修改它不改变内部迭代进度；break、continue、返回与 fn 捕获沿用现有规则。

17.11 小结

你会了	关键点
三段式	起点 . 中间操作 . …. 终结操作
起点	std::iter::range(a, b)（左闭右开）、xs.iter()、xs.intoIter()
中间操作	map filter take skip rev inspect enumerate chain zip
终结操作	collect fold count all any find last
组合	中间操作可无限串联，从上往下读即需求
惰性	不接终结操作就不执行；take 能省掉多余计算
引用坑	.iter() 元素是 T const&；要独立值用 .intoIter()
引入	import std::iter;

17.12 下一步

单文件程序写多了，总得拆。函数、结构体、常量放哪儿？怎么让别的文件用上？测试和格式化怎么跑？

第 18 章讲模块与工程：`import` 到底导入了什么、`magel new` 起出来的工程长什么样、`magel test` / `magel fmt` 怎么用。

翻到第 18 章模块与工程。

第 18 章 模块与工程

前面十七章，程序都写在一个 .mgl 文件里。真实项目做不到——几千行塞一个文件，谁也没法读。这一章讲怎么把代码拆开，以及工具链给工程准备的那几个命令。

拆开有两条线：

- 语言层：module 声明一个文件属于哪个模块，import 把别的模块引进来；
- 工程层：magel new 起一个带目录结构的工程，magel test / magel fmt 管测试和风格。

18.1 module：给文件一个名字

每个文件首行都可以声明自己属于哪个模块：

```
module mylib::math;
```

mylib::math 是模块路径，用 :: 分隔——和标准库里的 std::iter 是同一套写法。

模块名不是随便起的：import 一个模块时，编译器要按这个名字去找对应的文件（见 18.2）。

核心模块 std::lang 不用声明也不用 import：println / format / assert / len / Any / Option / Result 这些都是它自动带进来的（前面十七章你一直在用，从没 import 过）。

18.2 import：把别的模块接进来

假设有两个文件。先写一个工具模块 util/text.mgl：

```
module util::text;

String shout(String s)
{
    return s + "!";
}
```

再用 import 在另一个文件里用它：

```
module app;
import util::text;

int64 main()
{
    println("{}", shout("hi"));
    return 0;
}
```

跑 magel run app.mgl，输出：

```
hi!
```

几个要点：

- `import util::text;` 之后，`util::text` 里的公开函数按裸名就能调——直接写 `shout("hi")`，不用写 `util::text::shout`；
- 模块名 `util::text` 对应文件 `util/text.mgl`：`::` 换成目录分隔符，末尾加 `.mgl`。文件名和模块名对不上，`import` 就找不到；
- 路径是相对当前文件解析的。

文件名 = 模块路径这条对应关系，是把「模块」和「目录结构」绑在一起的关键：看文件树就知道模块树。

> **提示**
>
> `import` 后也可以写完整路径来调用，或者用 `import_names` 引入短名——细节见语言参考第 12 节与 `example/samples/import_names.mgl`。初学阶段先把 18.2 这套用熟。

18.3 起一个工程：magel new

手写目录、手写 `magel.toml` 太麻烦。工具链给了一条命令：

```
magel new demo
```

它创建一个叫 `demo` 的新目录（目标目录必须不存在，免得覆盖你的东西），里面长这样：

```
demo/
├── .gitignore
├── README.md
├── magel.toml          # 工程配置
├── src/
│   └── main.mgl        # 应用源码
└── tests/
    └── test_main.mgl   # 测试
```

建完还会顺手 `git init` 建好仓库。`src/main.mgl` 是那个你熟得不能再熟的 hello：

```
int64 main()
{
    println("Hello, Magel!");
    return 0;
}
```

18.4 magel.toml：工程配置文件

工程靠一个 `magel.toml` 描述自己。默认生成的是：

```
[package]
name = "hello"
version = "0.1.0"

[magel]
run = "interpret"
main = "src/main.mgl"
```

- [package] 里是包名和版本；
- [magel] 里告诉工具链：入口是哪个文件（main）、默认用什么方式跑（run）。

工程根目录一旦有 magel.toml，在这个目录里执行命令就不用反复写路径了。

.gitignore 也一并给好了——构建产物不该进版本库：

```
/build/
/.magel/
/magel_out*
*.exe
```

18.5 三个日常命令

工程起来了，日常就三件事：跑、查、测。

```
magel run src/main.mgl      # 运行
```

```
Hello, Magel!
```

run 是解释执行（第 1 章讲过 run 和 check 的区别）：

```
magel check src/main.mgl    # 只检查，不运行
```

check 只做编译期检查，一行都不执行——改完代码先 check 一下，比 run 快，也安全（不会真的动到数据）。

测试用 test。工程里的 tests/test_main.mgl 是个普通的 main，里面全是 assert （第 16 章）：

```
1 int64 main()
2 {
3     assert(1 + 1 == 2);
4     return 0;
5 }
```

跑整个 tests/ 目录：

```
magel test tests
```

输出每个测试文件的结果和合计（路径按实际位置显示）：

```
[passed] tests/test_main.mgl
通过 1，失败 0，跳过 0
```

assert 就是测试的体检项：条件不成立，那个测试文件就判 失败。你不用装任何测试框架——main + assert 就是最小可用的测试。

18.6 magel fmt：让风格统一

一个团队里，最没意义的争论是「大括号换行吗」「缩进几格」。工具链直接给了一个格式化器：

```
magel fmt src/main.mgl            # 打印格式化后的源码到屏幕
magel fmt src/main.mgl --write    # 直接写回文件
magel fmt src/main.mgl --check    # 只检查「格式对不对」，不修改
```

--check 是给 CI 用的：文件已经规范 → 安静退出；需要格式化 → 打印文件名并以非零码退出：

```
magel fmt src/messy.mgl --check
```

```
需要格式化：src/messy.mgl
```

（退出码是 1。CI 里挂上这条，谁提交了没格式化的代码就会红。）

fmt 会干的事，拿一段挤成一团的代码试（它本身能跑，只是很难看）：

```
int64 main()
{
    int64 x = 1;
    if (x>0)
    {
        println("{}", x);
    }
    return 0;
}
```

magel fmt 之后：

```
int64 main()
{
    int64 x = 1;
    if (x>0)
    {
        println("{}", x);
    }
    return 0;
}
```

缩进统一成 4 空格、层级对齐好。注意它只管空白和换行，不动代码内容（x=1 里没有空格、"{}", x 里那串多余空格，都原样留着）——格式化器不替你做美学判断，只保证结构清楚。

18.6.1 终端查手册

`magel man` 是终端手册入口，从同一可执行文件中的标准库数据查询。无目标时按能力域列模块，模块目标列符号，类型和方法目标显示签名、参数、返回值、错误边界及已有完整示例。

```
magel man
magel man std::text
magel man std::text::Text::split
magel man --search delimiter
magel man --json std::text::Text::split
magel man std::text::Text::split --run
```

搜索默认忽略大小写，`--exact` 匹配完整字段；拼错目标会给候选。`--lang zh|en` 选择界面和诊断语言，库说明仍复用中文原文。`--run` 在临时目录运行完整示例并附上输出。普通查询不需要 Python、网络或仓库；开发用 `--update` 运行既有生成脚本，重新构建后才进入内嵌数据。

完整命令和 JSON 结构见 doc/man，标准库 API 说明仍在 doc/library。

18.6.2 编辑器增量提示

LSP 将补全、悬停、语义着色、工作区查询、宏预览、编译诊断交给不同工作队列。修改按 UTF-16 编辑范围同步，同一源码版本复用索引。诊断延迟 250 毫秒合并连续编辑，保存立即检查；取消过期任务，并且不把旧版本结果写回编辑器。

缓存在 magel 可执行文件旁的 `build/lsp/`，与工程目录无关。未保存的依赖修改和关闭会刷新引用者，不写回源文件。缓存自动清理七天未使用的普通文件，超过 256 MiB 时淘汰到 192 MiB，保护最近十分钟的文件；活跃任务可以暂时超过额度。索引内存也有淘汰阈值，缓存损坏或不可写会退回分析。详细策略见 LSP 增量提示与缓存。

18.6.3 `inline`：函数定义的内联属性

`inline` 写在模块级函数定义前，参数、返回类型与函数体仍按普通函数规则检查。

```
1  inline int64 square(int64 value)
2  {
3      return value * value;
4  }
5
6  int64 main()
7  {
8      println("{}", square(3));
9      return 0;
10 }
```

```
9
```

它是函数定义的属性，不是任意表达式的前缀，也不改变值语义。代码组织仍以清楚的职责为先，不需要给所有函数都加上它。

18.7 小结

你会了	关键点
模块声明	`module mylib::math;` 放首行
导入	`import util::text;`，公开函数按裸名调用
名字对应文件	`util::text` ↔ `util/text.mgl`，路径相对当前文件
`std::lang`	核心模块，自动加载，永不 import
起工程	`magel new demo`（目标目录不能已存在）
工程配置	`magel.toml` 写包名、入口、运行方式
日常命令	`run`（跑）、`check`（只查）、`test`（跑 `tests/`）
测试	普通 `main` + `assert`，无需框架
格式化	`magel fmt …[--check|--write]`，只动空白不碰代码

18.8 下一步

工程拆好了、测试挂上了、格式统一了。接下来两章讲工具链里更硬核的部分：接下来会见到 `%define` / `$macro` 这些带 `%` 和 `$` 的写法：它们是 Magel 的预处理与宏，能在编译前替你改源码。

翻到第 19 章预处理与宏。

书稿与关键字帮助也在程序内部：`magel man book` 导出书稿与图片到当前目录的 `book/` 并列出章节，`magel man book 6` 查看第 6 章，`magel man keyword break` 查看关键字语法与示例。`magel man skill` 输出 AI SKILL，可保存并交给支持 SKILL 的 AI 工具；VS Code 包也携带同一份 `skills/magel-programming/SKILL.md`。

18.8.1 终端手册的入口

```
magel man api std
magel man api std::text::Text
magel man keyword return
magel man keyword '#derive'
magel man macro
```

API 查询必须显式写 `api`。省略它的 `magel man std` 或 `magel man std::text::Text` 会报错并给出正确命令；`keyword` 查询语言关键字，`macro` 查询宏规则，二者不是 API 入口。可以给这些查询加 `--json` 供工具读取。

第 19 章 预处理与宏

你在第 9 章见过 `%token`、第 15 章见过 `$macro` 这类带 `%` 和 `$` 的写法。它们是 Magel 的预处理和宏——在代码被正式编译之前，先对源码做一次「文本层面」的加工。

这一章分两半：

- 预处理（`%` 开头）：条件编译、文件包含——「这段代码要不要编进去」；
- 宏（`$` 开头）：文本替换——「把这个缩写换成一长串代码」。

先说一句要紧的话：这两样都是”不得已才用”的工具。平时用函数、泛型、常量就够了。但读别人的代码、或者做嵌入式多平台构建，绕不开它们，所以得会认、会写。

19.1 条件编译：%define + %if

同一个程序，调试时想多打日志、发布时想去掉——这是条件编译最典型的用途：

```
%define DEBUG 1

int64 main() {
%if DEBUG
    println("debug on");
%else
    println("debug off");
%endif
    return 0;
}
```

```
debug on
```

把第一行改成 `%define DEBUG 0`，同一份代码就输出另一个分支：

```
%define DEBUG 0

int64 main() {
%if DEBUG
    println("debug on");
%else
    println("debug off");
%endif
    return 0;
}
```

```
debug off
```

几条要点：

- %define 定义预处理符号；
- %if 符号看它的值：非 0 走 %if 分支，0 走 %else；
- 不活跃的分支整块丢弃——一个字都不会编进去；
- 指令行必须顶格写在行首（% 前面不能有别的字符）；
- 被丢弃/指令占用的行用空行占位，所以预处理后行号和源文件一一对应，报错永远指回你真正写错的那一行。

> 提示
>
> %if 后面要写符号名，不能写 %if 1 这种字面量——写了会报「这里要写一个符号名」。这是它和 C 的 #if 最容易混的地方（见 19.6）。

19.2 按「有没有定义」分支：%ifdef / %ifndef

有时候你只关心「这个符号定义过没有」，不关心值：

```
int64 main() {
%ifdef FEATURE
    println("有 FEATURE");
%else
    println("没有 FEATURE");
%endif
    return 0;
}
```

```
没有 FEATURE
```

- %ifdef 符号：定义过就走；
- %ifndef 符号：没定义才走（%ifdef 的反面）。

%if 和 %ifdef 的分工记住一句话：%if 看值，%ifdef 看有没有。

19.3 从命令行传开关：-D

开关真正好用，是构建时才决定——同一份源码，编出不同版本：

```
magel run app.mgl -D FEATURE
```

```
有 FEATURE
```

-D FEATURE 等价于在文件开头写了 %define FEATURE。于是同一份代码：

- magel run app.mgl → 「没有 FEATURE」；
- magel run app.mgl -D FEATURE → 「有 FEATURE」。

发布版 / 调试版、不同硬件平台，就靠这个开关切。

19.4 %include：把文件原地拼进来

%include 在当前文件里原地展开另一个文件，像是把那边的文本复制过来：

```
%include "util.mgl"

int64 main() {
    println("{}", twiceValue(21));
    return 0;
}
```

如果 util.mgl 里定义了 twiceValue，这段就能跑，输出 42。

%include 和 import（第 18 章）解决的不是一回事：

	import（模块）	%include（预处理）
层次	语言层	预处理层，早于编译
效果	引入模块，按裸名调函数	文本原地展开
同一文件重复	只算一次	默认也只一次（%include_once 显式表达）
适用	正常组织代码	常量表、平台片段、生成代码

优先用 import。%include 是给「同一份片段要拼进很多文件」这种场景留的。

19.5 宏：$macro 文本替换

宏用 $macro 名字（参数）…$end_macro 定义，调用时名字前加 $：

```
$macro SQUARE(x)
((x) * (x))
$end_macro

int64 main() {
    println("{}", $SQUARE(5));
    println("{}", $SQUARE(2 + 3));
    return 0;
}
```

```
25
25
```

$SQUARE(5) 在编译前被文本替换成 ((5) * (5))；$SQUARE(2 + 3) 变成 ((2 + 3) * (2 + 3))。

不带参数的叫对象宏，调用时也不用括号：

```
$macro GREETING()
"hello, magel"
$end_macro

int64 main() {
    println("{}", $GREETING);
    return 0;
}
```

```
hello, magel
```

宏还能拼出新名字。`$concat(a, b)` 把两段拼成一个标识符：

```
$macro CAT(a, b)
$concat(a, b)
$end_macro

int64 main() {
    int64 $CAT(my, var) = 7;
    println("{}", myvar);
    return 0;
}
```

```
7
```

`$CAT(my, var)` 展开成 `myvar`——这在手写代码时基本没用，但代码生成器特别爱用它。

19.5.1 有序多括号分组

定义端可以按任意顺序组合 `< >`、`[ ]`、`( )` 和 `{ }`，也可以重复同一种括号。调用必须照定义中的顺序写；每一组的 `..参数` 只捕获本组当前层级的内容。

```
$macro SHOW<..Types>[..Attrs](..Args){..Body}
println("{}", $stringify(Types));
println("{}", $stringify(Attrs));
println("{}", $stringify(Args));
Body
$end_macro

$macro EMPTY()()
println("repeated groups");
$end_macro

int64 main()
{
    $SHOW<List<List<int64>>, String>[readonly](x, y){
        int64 total = 2 + 3;
        if (total > 0)
        {
            println("{}", total);
        }
    }
    $EMPTY()()
    return 0;
}
```

```
List<List<int64>>, String
readonly
x, y
5
repeated groups
```

这里的 `readonly`、`x`、`y` 只是被捕获和字符串化的原文，不需要是有效变量。宏仍位于词法分析前，不把参数解析成 AST，也不检查参数类型。尖括号里的 `>>` 按两个闭合字符分组，不会被宏阶段当作右移运算符。

大括号参数可以跨多行。嵌套括号、字符串和注释中的逗号不会切开外层参数；不同括号组的变长捕获互相隔离。顺序、结构或绑定不匹配时报告“宏 Token 分组不匹配”。展开片段保留来源文件及行列映射，后续诊断仍定位原始源码。

`$macro EMPTY()()` 是两个连续空组，调用时也写 `$EMPTY()()`。既有单组对象宏 `$macro NAME()` 仍沿用 `$NAME` 的对象宏调用方式。完整说明可直接执行 `magel man macro`，也支持 `--json`。

19.6 宏的第一个坑：括号要自己加

宏是纯文本替换，它不懂运算优先级。看这个「反面教材」：

```
$macro BAD_SQUARE(x)
x * x
$end_macro

int64 main() {
    println("{}", $BAD_SQUARE(2 + 3));
    return 0;
}
```

```
11
```

`$BAD_SQUARE(2 + 3)` 被替换成 `2 + 3 * 2 + 3`——按优先级一算是 11，不是 25。`*` 只管到了紧挨着的数。

所以写宏的铁律：参数、整个宏体、都要裹上括号。19.5 里 `SQUARE` 写成 `((x) * (x))` 就是这个道理。对比一下：

```
$macro SQUARE(x)
((x) * (x))
$end_macro

int64 main() {
    int64 five = 5;
    println("{} {}", $SQUARE(five), $SQUARE(five + 1));
    return 0;
}
```

```
25 36
```

`five + 1` 被裹进括号，先加后乘，结果才对。

19.7 第二个坑：%if 只认符号名

想按一个字面量分支？不行：

```
int64 main() {
%if 1
    println("ok");
%endif
    return 0;
}
```

```
magel: 前处理: hello.mgl:2:4: error: 这里要写一个符号名 [bad-expression]
```

`%if` 后面必须是预处理符号（`%define` 定义过的、或者 `-D` 传进来的）。要按数分支，先定义成符号：`%define LEVEL 1`，再 `%if LEVEL`。

`%error` 则是在活跃分支里主动报错——常用于「这种情况本不该编进来」：

```
%define LEVEL 3

int64 main() {
%if LEVEL
%error 不该编到这里
%endif
    return 0;
}
```

```
magel: 前处理: hello.mgl:5:1: error: 不该编到这里 [error-directive]
```

配套还有 `%warning`（报警告，不中断），用法一样。

19.8 括号形参：每次出现都包一层括号

19.6 说宏体是文本替换，`$TIMES_TWO(1 + 2)` 会散成 `1 + 2 * 2`。第一种挡法是用括号标注形参：

```
$macro TIMES_TWO((x))
    x * 2
$end_macro
int64 main() {
    println("{}", $TIMES_TWO(1 + 2));
    return 0;
}
```

```
6
```

括号标注只写在 `$macro` 头这一行（`(x)`），宏体里引用形参照旧写裸名字 `x`；效果是这个形参每次出现都换成 `(实参)`，于是 `(1 + 2) * 2 = 6`。

19.9 expr 宏：整段展开再包一层

第二种挡法是在宏名前面写 `expr` ——它给整段展开结果再包一层括号：

```
$macro expr PLUS((a), (b))
    a + b
$end_macro
int64 main() {
    println("{}", 10 / $PLUS(2, 3));
    return 0;
}
```

```
2
```

10 / ((2) + (3)) = 2；只包形参的话是 10 / (2) + (3) = 8。expr 只写在头这一行，同时默认给全部实参加括号；a 与 (a) 完全等价，宏名还是 PLUS，调用处写 $PLUS(2, 3)；调用点统一使用普通宏调用形式；旧的感叹号与反引号标注均已废除。

两种挡法怎么选：普通宏按需用 (a) 标注；整个宏作为表达式使用时声明为 expr 宏。

19.10 变长尾参 ...rest

形参表末尾写 ...名字，剩下的实参全归它：

```
1  $macro REPORT(...rest)
2  println("{} {} {}", rest);
3  $end_macro
4  int64 main() {
5      $REPORT(1, 2, 3)
6      return 0;
7  }
```

```
1 2 3
```

rest 会按 , 拼成一段文本再替换，所以在调用位置就是”一串实参”（和 C 的 __VA_ARGS__ 一个意思）——上面正好当三个实参喂给 println。

19.11 按实参个数重载

同一个宏名可以按实参个数挂多种形态：

```
1  $macro REPORT(a)
2  println("one {}", a);
3  $end_macro
4  $macro REPORT(a, b)
5  println("two {} {}", a, b);
6  $end_macro
7  int64 main() {
8      $REPORT(7)
9      $REPORT(1, 2)
10     return 0;
11 }
```

```
one 7
two 1 2
```

挑形态的顺序：个数正好对上的固定形态 → 变长形态。一个都对不上会报错并把候选列出来，不会悄悄挑一个不相干的形态。

19.12 $for：按尾参重复一段

$for 变量 in 尾参名…$end_for 把宏体里那一段按尾参逐个重复：

```
$macro SUM_SQUARES(...values)
int64 total = 0;
$for v in values
total = total + (v) * (v);
$end_for
println("平方和 = {}", total);
$end_macro
int64 main() {
    $SUM_SQUARES(2, 3, 4)
    return 0;
}
```

```
平方和 = 29
```

$for 只能遍历变长尾参。每轮把循环变量整体换成该轮的实参文本，所以 (v) * (v) 这种"把变量包在中间"的写法是安全的。$for 的结束符是 $end_for、宏体的结束符是 $end_macro ——两个"结束"分开写，嵌套时不用数层数。$for 里还能再嵌 $for。

19.13 尾参逐元素加括号 ...(values)

19.12 里靠手写 (v) 保平安。想让编译器代劳，就把尾参名放进括号，写成 ...(values)：

```
$macro SUM_SQUARES(...(values))
int64 total = 0;
$for v in values
total = total + v * v;
$end_for
println("平方和 = {}", total);
$end_macro
int64 main() {
    $SUM_SQUARES(1 + 1, 2 + 1)
    return 0;
}
```

```
平方和 = 13
```

values 仍按 , 摊开，但每个元素各包一层括号，所以 $for 的循环变量变成（元素）：(1+1)² + (2+1)² = 13。不作括号标注就是 1+1*1+1 + 2+1*2+1 = 8。... 必须放在括号外；(...values) 会报错。

19.14 $concat：拼出一个标识符

```
$macro DECLARE(prefix, suffix, value)
int64 $concat(prefix, suffix) = value;
$end_macro
int64 main() {
    $DECLARE(local, Count, 11)
    println("{}", localCount);
    return 0;
}
```

```
11
```

$concat(a, b) 的两个实参必须都是标识符，拼出来的结果也是标识符，而且不重新参与宏展开。拼出来的是普通名字，所以能和真正的函数同名共存（宏调用必须带 $，裸名是普通标识符）。

> 提示
>
> $concat 与 $stringify 都读取原始实参，不受表达式参数括号保护影响。

19.15 $stringify：取调用点写下的原文

```
$macro SOURCE(expr)
$stringify(expr)
$end_macro
int64 main() {
    println("{}", $SOURCE(1 + 2));
    return 0;
}
```

```
1 + 2
```

结果是字符串字面量：实参在宏体里是形参名时取调用点的原始实参文本（不加括号），否则就取写下的字面文本。写日志、断言信息时特别好用——变量名和它的值能一起打出来。

19.16 $replace：一行写完的宏

$replace 把声明压进一行，头这一行剩下的内容就是宏体，没有 $end_macro：

```
$replace expr ADD((a), (b)) a + b
int64 main() {
    println("{}", $ADD(1, 3));
    return 0;
}
```

```
4
```

它支持 $macro 的所有”行内”规则（参数括号、expr、$concat / $stringify、$line / $file / $module），但装不下 $for 这种块——要块请用 $macro。宏名和调用方式与 $macro 完全一样，两种声明混着写也行。

19.17 $delete_macro：撤掉再重新定义

同一名字、同一参数个数的宏不能重复定义。要”改”一个宏，先撤后立：

```
$macro REVISION()
1
$end_macro
$delete_macro REVISION
$macro REVISION()
2
$end_macro
int64 main() {
    println("{}", $REVISION);
    return 0;
}
```

```
2
```

19.18 $0～$9：宏内的唯一临时名

宏是文本替换，宏体里起个临时变量很容易撞上调用点的同名变量。$0～$9 由编译器在每次展开时换成一个全局唯一的名字：

```
$macro SWAP((a), (b))
    int64 $0 = a;
    a = b;
    b = $0;
$end_macro
int64 main() {
    int64 tmp = 111;
    int64 x = 1;
    int64 y = 2;
    $SWAP(x, y)
    println("x={} y={} tmp={}", x, y, tmp);
    return 0;
}
```

```
x=2 y=1 tmp=111
```

宏内的临时名和调用点的 tmp 互不干扰。同一次展开里同一个 $n 处处同名；不同展开、不同宏、嵌套调用各自独立。

只有一位数字合法（$10 会报错），字符串和注释里的 $0 原样保留。这是变量级近似卫生——解决的是”宏内临时变量被捕获”，不是完整卫生宏：宏体里引用的自由名字（比如外部函数名）仍然绑到调用点。

19.19 $line / $file / $module 和 $assert

```
module demo;

$macro HERE(tag)
println("{} 在第 {} 行，模块 {}", tag, $line, $module);
$end_macro

int64 main() {
    $HERE("这里")
    return 0;
}
```

```
这里 在第 8 行，模块 demo
```

写在宏体里时，取的是调用点那一行（和 C 的 __LINE__ 一个意思），所以基于它的报错总是指向你写宏调用的地方。$file / $module 展开成字符串字面量，可以直接拼接或喂给 format；根文件没写 module 时 $module 是 __root。$assert(条件) / $assert(条件, " 说明") 是系统给的断言宏，用的就是这套机制。

19.19.1 宏内条件与局部绑定

`$if` / `$elif` / `$else` / `$end_if` 在宏展开时选择分支，`$let` 创建只属于当前宏调用的局部文本绑定。

```
$macro expr PICK(value)
$let selected = value
$if selected > 0
selected
$else
0
$end_if
$end_macro
int64 main() {
    println("{} {}",$PICK(7),$PICK(-2));
    return 0;
}
```

```
7 0
```

`$argc` 取得当前调用的实参数量，`$count(...)` 计数变长尾参或局部文本列表，`$typecheck(T, expr)` 将展开表达式送进指定类型检查。实参计数不匹配在展开期报错；递归最多 64 层，一次处理最多展开 100000 次。没有 `$match` 迷你 DSL，`##` 与 `#` 形参均已移除，拼接和字符串化只用 `$concat` / `$stringify`。

`$macro expr ADD(a,b)` 中的参数自动括起来，定义里的 `(a)` 可作冗余标注；普通宏仍需 `(a)` 明确标记表达式实参。调用使用普通形式 `$ADD(x+1,y*2)`，宏定义不接受反引号标注。

19.20 完整指令速查

指令	作用
`%define 名字 [值]` / `%undefine 名字`	定义 / 取消符号
`%if` / `%elif` / `%else` / `%endif`	按值条件编译
`%ifdef` / `%ifndef`	按是否定义条件编译
`%include " 路径"`	原地展开文件（默认只一次）
`%error` / `%warning`	活跃分支里报错 / 报警告
`$macro N(参数) …$end_macro`	函数式宏，调用 `$N(…)`
`$macro N() …$end_macro`	对象宏，调用 `$N`
`$concat(a, b)`	拼标识符
`$stringify(x)` / `$line`	把参数变成字符串 / 当前行号
`$delete_macro N`	取消宏（先撤后立，好重定义一个同名宏）
‘ `$macro N((a), (b))` ‘	显式形参括号：该形参每次出现都替换成 (实参)
`$macro expr N(…) …$end_macro`	宏名前的 `expr`：整段展开再包一层括号
`$replace N(参数) 宏体`	单行宏，没有 `$end_macro`

指令	作用
`$macro N(...rest)`	变长尾参：rest 按 , 摊成一段文本
‘ `$macro N(...(rest)) ‘`	变长尾参逐元素加括号
`$for v in rest …$end_for`	按变长尾参重复宏体里的这一段
`$concat(a, b)`	拼标识符（两个实参都必须是标识符）
`$stringify(x)`	取实参的源码文本，结果是字符串字面量
`$0`～`$9`	宏体内的唯一临时名（每次展开换一个新名字）
`$line` / `$file` / `$module`	调用点行号 / 文件名 / 模块名（宏内外都能用）

> **提示**
>
> 字符串里的 % 不是指令；真想打一个百分号，写 %%。

19.21 什么时候该用，什么时候别用

预处理和宏是” 元编程的粗活”，能不用就别用：

你的想法	更好的选择
「一段逻辑重复了」	写成函数（第 4 章）
「同一段逻辑要适配多种类型」	泛型（第 14 章）
「想要一个常量」	const（第 3 章）
「给函数起个短别名」	想清楚——直接调原函数更清楚
「按平台/调试开关切代码」	✓ %define + %if，正解
「把常量表拼进多个文件」	✓ %include，合理
「生成一大堆重复代码」	✓ $macro，正当用途

宏的坏处：它在类型系统之外。写错了编译器不一定能帮你抓，报错也常指向展开后的乱码（虽然 Magel 会贴心地标 [宏 MIN] 告诉你出错在哪个宏里）。所以能上函数就上函数。

19.22 小结

你会了	关键点
`%define` / `-D`	定义预处理符号；`-D` 从命令行传
`%if` / `%else` / `%endif`	按值分支（`%if` 后只能写符号名）
`%ifdef` / `%ifndef`	按是否定义分支
行号	指令/丢弃行用空行占位，报错指回真实位置
`%include`	文本原地展开；正常组织代码请用 `import`
`$macro`	`$macro N(x) …$end_macro`，调用写 `$N(…)`
对象宏	`$macro N() …$end_macro`，调用写 `$N`
括号铁律	宏体、参数都要裹括号，否则优先级出错
`$concat`	拼标识符（代码生成用）
选型	优先函数 / 泛型 / 常量；宏留给构建开关和代码生成

19.23 下一步

预处理和宏改的是源码文本。第 20 章讲一个再上一层的能力：注解——在声明上贴元数据，运行期还能反过来查。它是「框架式代码」的常见基础。

翻到第 20 章注解与反射。

第 20 章 注解与反射

第 19 章的预处理改的是源码文本。这一章讲一个更「上层」的能力：注解——在声明旁边贴一张标签，标签上带点数据。平时它安安静静躺着不干活；但程序可以在运行期把标签读回来，据此决定怎么行动。

一句话概括：注解 = 给代码贴的、可以被程序自己读到的元数据。

20.1 声明一个注解类型

注解类型用 `@annotation` 声明，参数写成「名字: 类型」，可以带默认值：

```
@annotation Route(path: String, level: int32 = 2)
```

读作：声明一个叫 `Route` 的注解，它有两个参数 `path`（字符串）和 `level`（整数，缺省 2）。

20.2 把注解贴到声明上

声明完，就能贴了——写在被标注的东西前面（下面这段要在反射版工具链下跑，见 20.6）：

```
@annotation Route(path: String, level: int32 = 2)

@Route("/health", level: 1)
int64 health()
{
    return 200;
}

int64 main()
{
    println("{}", health());
    return 0;
}
```

```
200
```

注意：这个程序的行为一点没变——`health()` 还是返回 200。注解本身不影响执行，它只是一份「挂上去的资料」。要让它起作用，得靠 20.4 节之后的手段去读它。

参数可以按位置写（`"/health"`），也可以按名字写（`level: 1`）。

20.3 能贴在哪里

注解几乎能贴在所有声明上：

位置	例子
函数	`@Route("/health") int64 health() { …}`
结构体 / 接口 / 枚举	`@Component struct Service { …};`
字段	`@Note("field") int64 value;`
方法 / 构造器	`@Route("/read") int64 read() { …}`
静态成员	`@Note("static") static int64 shared;`
形参	`@Note("contract") int64 read();`（接口方法声明）

多个注解可以叠着贴：

```
@Component
@Route("/read", level: 3)
int64 read()
{
    return self.value;
}
```

20.4 读注解：运行期反射

注解真正的用处，是程序在运行期把标签读回来。这套能力叫反射，在标准模块 `std::reflect` 里，要 `import`：

```
import std::reflect;

@Reflect
struct Point
{
    int64 x;
    int64 y;
};

int64 main()
{
    Option<std::reflect::StructInfo> info = std::reflect::getStructInfo<Point>();
    println("{}", info.isValue());
    std::reflect::StructInfo s = info.unwrap();
    println("{} {}", s.name, s.fields.len());
    return 0;
}
```

```
1
Point 2
```

几个要点：

- `@Reflect` 是「暴露」标记：只有被 `@Reflect` 标过的声明，运行期才查得到。没标的类型，`getStructInfo` 会返回 `None`（不是报错）；
- `getStructInfo<Point>()` 用类型参数取元数据，返回 `Option<StructInfo>`（第 16 章）；
- `StructInfo` 里写着 `name`、`fields`、`methods`、`staticMembers`、`annotations`……——一个类型能问的都在里头。

读注解的实参：

```
import std::reflect;

@annotation Route(path: String, level: int32 = 2)

@Reflect
@Route(path: "/health")
int64 health()
{
    return 200;
}

int64 main()
{
    List<std::reflect::Annotation> anns = std::reflect::getFuncAnnotations("health");
    println("{} {}", anns.len(), anns[1].args.len());
    println("{}", anns[1].args[0].asText().unwrap());
    return 0;
}
```

```
2 2
/health
```

- `getFuncAnnotations("函数名")` 按名字取函数上的注解列表；
- 这一份列表里有两项：`@Reflect`（0 个实参）和 `@Route`（2 个实参）；
- 实参是一串 `std::reflect::ReflectValue`（一个能装任意类型的小盒子），用 `asText()` / `asI32()` 等按类型取出来，都返回 `Result`。

> **提示**
>
> 注解实参会补全默认值：`Route` 的 `level` 缺省是 2，所以 `args` 里 `level` 位永远有值。

20.5 按名字调用函数

反射还能按字符串名字调函数——名字在运行期才知道时，只有这条路：

```
import std::reflect;

@Reflect
int64 plusOne(int64 x)
{
    return x + 1;
}

int64 main()
{
    List<std::reflect::ReflectValue> args;
    args.push(std::reflect::ReflectValue::I64(41));
    Result<std::reflect::ReflectValue, std::reflect::ReflectError> r = std::reflect::call("plusOne",
    args);
    println("{}", r.isOk());
    return 0;
}
```

```
1
```

实参要自己手工装箱（`ReflectValue::I64(41)`），返回值也是 `Result<ReflectValue, …>`。

调方法要用 `callMethod<T>`，并且把接收者写成引用：

```
import std::reflect;

@Reflect
struct Counter
{
    int64 n;
    int64 bump()
    {
        self.n = self.n + 1;
        return self.n;
    }
};

int64 main()
{
    Counter c;
    c.n = 0;
    List<std::reflect::ReflectValue> args;
    Result<std::reflect::ReflectValue, std::reflect::ReflectError> r =
    std::reflect::callMethod<Counter>(&c, "bump", args);
    println("{} {}", r.isOk(), c.n);
    return 0;
}
```

```
1 1
```

按名的接口分派有 `callInterface<I>`——写法一样，把接口名当类型参数。

20.6 构建与目标 VM 的反射能力

根目录工具链默认同时编入前端和运行时反射，单次编译也默认启用。本例使用默认工具链即可运行：

```
@annotation Route(path: String, level: int32 = 2)

@Route("/health", level: 1)
int64 health()
{
    return 200;
}

int64 main()
{
    println("{}", health());
    return 0;
}
```

```
200
```

方式	含义
`make`	根目录默认前端 MAGEL_WITH_REFLECTION=1、运行时 STACKVM_WITH_REFLECTION=1
`make reflection`	保留的显式启用目标，仍须两项同时开启
不写单次开关	支持反射的当前工具链默认启用
`--reflection`	显式要求反射，缺少能力时报错
`--no-reflection`	显式关闭；关闭后使用注解或反射会报错

```
magel run app.mgl
magel svmb app.mgl --no-reflection --out build/board-app
```

第二条用于无需反射的源程序：产物必须匹配默认无反射的板端 VM。开启反射的产物携带特征位，关闭能力的 VM 会拒绝加载。若源程序依赖反射，应为目标编入能力并评估元数据和运行时预算，不能通过省略 `--reflection` 假装关闭。独立生成的编译器工程按其 CMake 配置单独启用两项宏。

20.7 什么时候该用注解

注解与反射提供运行期元数据和受限动态调用，会增加产物、分配和调用开销。默认桌面工具链支持它，裁剪目标需要显式评估能力和预算。

你的想法	该不该用
「想给函数打点标记，运行时按标记分派」	✓ 典型场景（路由、插件注册）
「想让代码生成器读元数据」	✓ 合理
「想根据标记做一遍扫描」	✓ 合理
「想省一个函数参数」	✗ 用参数

你的想法	该不该用
「想实现多态」	✗ 用接口 / 泛型（第 13、14 章）
「想在板子上用」	默认板端关闭；按预算编入能力，或移除依赖并用 –no-reflection 生成

经验法则：注解是「框架和工具用的」，不是「业务逻辑用的」。写业务代码时，几乎总有更直接的办法。

20.8 小结

你会了	关键点
声明注解	`@annotation 名字（参数：类型 = 默认值）`
贴注解	写在声明前；可叠多个；能贴允许的函数/类型/字段/方法/静态成员；不能贴形参
注解不改变行为	它只是一份挂在声明上的资料
`@Reflect`	暴露标记：没标过的声明，运行期查不到
查类型	`std::reflect::getStructInfo<T>()` → `Option<StructInfo>`
读注解	`getFuncAnnotations("名字")` / `getAnnotations<T>()`
取实参	`Annotation.args` 是 `ReflectValue`，用 `asText()` 等取（返回 `Result`）
按名调用	`std::reflect::call` / `callMethod<T>(&obj, …)` / `callInterface<I>`
构建	根目录默认同时编入并启用；无反射目标显式 –no-reflection
取舍	框架/工具才用；业务逻辑优先接口/泛型

20.9 下一步

到这里，语言本身讲完了——从变量到注解，你能写出完整的、有类型安全、有资源管理、有抽象能力的 Magel 程序。

第七部分开始「走出去」：让 Magel 跟外部世界打交道。第一个是与 C 互操作——调用现成的 C 库、把自己的函数交给 C 调用。

翻到第 21 章与 C 互操作。

第 21 章 与 C 互操作

到第 20 章为止，你的程序都活在自己的世界里。可现实中有太多东西是现成的 C 库：压缩、加密、数据库、图形、系统调用……你不可能全都重写一遍。

这一章讲怎么跨过 Magel 和 C 的边界，两个方向：

- Magel 调 C——用 `abi` 块调动态库里导出的函数；
- C 调 Magel——把你的函数交给 C 的运行时。

21.1 三种集成方式

工具链支持三条路，按「需要多少 C 侧配合」排：

方式	用在哪	你要准备什么
`abi` 块	调动态库里按 C ABI 导出的函数	一份 `.dll` / `.so`
宿主胶水绑定	把 C++ 函数注册进工具链，Magel 侧当普通函数调	写一点 C++
嵌入（AOT + `runMagel`）	宿主程序调用编译好的 Magel 逻辑	AOT 产物

`abi` 块是最轻的一条——不用改工具链，不用重新编译，运行时直接加载库。

21.2 `abi` 块：调动态库里的 C 函数

假设你有一个 C 文件 `native.c`：

```c
#include <stdint.h>

#if defined(_WIN32)
#define MAGEL_ABI_EXPORT __declspec(dllexport)
#else
#define MAGEL_ABI_EXPORT
#endif

MAGEL_ABI_EXPORT int32_t abi_add(int32_t left, int32_t right) {
    return left + right;
}

MAGEL_ABI_EXPORT const char *abi_name(void) {
    return "native";
}
```

先把它编成动态库（说句实话：「怎么编动态库」是 C 的事，和 Magel 无关）：

```
gcc -shared -o native.dll native.c
```

然后在 Magel 里声明想用的函数：

```
abi "./native.dll"
{
    int32 native_add(int32 left, int32 right) = "abi_add";
    String native_name() = "abi_name";
}

int64 main()
{
    println("{} {}", native_add((int32)20, (int32)22), native_name());
    return 0;
}
```

```
42 native
```

abi 块的语法是：

```
abi "<动态库路径>" {
    <返回类型> <Magel侧名字>(<形参表>) = "<库里的符号名>";
}
```

关键就是那个 =：等号右边是库里真实导出的符号名，左边是你想用的名字。两者可以不一样——给一批 C 函数起统一风格的 Magel 名，就靠它。

> **提示**
> 路径是运行时按当前工作目录找的。所以 `./native.dll` 得跟你的程序在同一个目录下跑。

21.3 结构体、引用、字符串

标量只是热身。真实场景里你还要传结构体（按值）、引用（原地改）、字符串：

```
API Point move_point(Point p, int32_t dx, int32_t dy) {
    p.x += dx; p.y += dy;
    return p;
}
API void bump(int32_t *value) { ++*value; }
API const char *greet(void) { return "hello from C"; }
```

Magel 侧：

```
struct Point
{
    int32 x;
    int32 y;
};

abi "./abi_demo.dll"
{
    Point move_point(Point p, int32 dx, int32 dy) = "move_point";
    void bump(int32 mut& value) = "bump";
    String greet() = "greet";
}

int64 main()
{
    Point p;
    p.x = 1;
    p.y = 2;
    Point q = move_point(p, (int32)10, (int32)20);
    println("{} {} / {} {}", p.x, p.y, q.x, q.y);

    int32 n = 41;
    bump(&n);
    println("{}", n);

    println("{}", greet());
    return 0;
}
```

```
1 2 / 11 22
42
hello from C
```

三个要点：

1. 结构体按值传——C 的 `Point p` 对应 Magel 的 `Point p`，布局默认按 C 规则对齐。所以 Magel 这边得声明一个字段顺序、类型都一致的 `struct`；
2. C 的 `int32_t *value` 对应 `int32 mut& value`（第 7 章的可变引用），调用处写 `bump(&n)`——取地址用 `&`，和 Magel 内部引用是同一套写法；
3. C 的 `const char *` 对应 `String`。

21.3.1 支持的类型一览

Magel 写法	含义
int8…int64、uint8…uint64、float32、float64、bool	标量，按值传
String	字符串（C 侧 const char *）
T mut& / T const&	可变 / 只读引用（C 侧 T * / const T *）
Option<T mut&> / Option<T const&>	可空引用（C 侧可以是空指针）
T[] + 另给长度	数组视图（C 侧 T *）
结构体	按值传，默认 C 布局
struct X packed { …}	紧凑布局，对接 C 的 #pragma pack

21.4 数组与「可空」：边界上的两个特殊话题

数组用 T[] 传，长度单独给一个参数；如果返回的数组长度由某个形参决定，在末尾用 [count] 标注：

```
abi "./native.dll"
{
    Packed[] packed_numbers(int32 count) = "packed_numbers" [count];
    void packed_fill(Packed[] values, int32 count) = "packed_fill";
}
```

「空」是这个边界上唯一允许出现空的地方。Magel 内部的引用永远非空（第 7 章），但 C 函数可能返回空指针。这时候用 Option<T mut&> / Option<T const&> 接：

```
abi "./native.dll"
{
    Option<Packed const&> packed_snapshot() = "packed_snapshot";
}
```

返回 None 就代表 C 那边给了个空指针；传参时也可以传 Option<Packet mut&>::None，C 侧收到的就是空指针。

> 提示
> 这是「空」唯一能跨过边界的通道。跨过边界之后，一切又回到「引用非空」的老规矩。

packed 结构体用于对接 C 侧 #pragma pack 的紧凑布局——C 里用 #pragma pack(push, 1) 压过的地方，Magel 侧就声明成 struct X packed { …}。

完整例子见 doc/examples/abi_buffers/——它覆盖了结构体嵌套、紧凑布局、数组读写、可空引用、返回值长度等全部边界情况。

21.5 宿主胶水：把 C++ 函数注册进来

不想走动态库、想让 C++ 函数像普通函数一样调？写一个 bindMagelGlue 把它注册进工具链：

```
#include <cstdint>
#include <string>

struct ExportCounterTag;
using ExportCounter = magel_glue::Handle<ExportCounterTag, std::uint64_t>;

bool bindMagelGlue(magel_glue::Registry &registry, Memory::String *error)
{
    return registry.bindTyped<std::uint32_t, Memory::String>(
               "demo_cpp_text_size",
               [](Memory::String arg0) {
                   return static_cast<std::uint32_t>(arg0.size());
               }, error);
}
```

Magel 侧就像声明普通函数：

```
int32 demo_cpp_text_size(String text);

int64 main()
{
    println("{}", demo_cpp_text_size("hello"));
    return 0;
}
```

两条约定：

- 复杂宿主对象一律用整数句柄过边界——不要把 C++ 对象本体塞进 Magel 值里，而是用 `magel_glue::Handle<Tag, AbiType>` 给它一个「名义类型」，Magel 侧拿到的是个整数；
- 泛型函数实例化后键名带后缀：`demo_cpp_identity<int32>` 对应键名 `demo_cpp_identity__int32`。

宿主类型的名字是编译期就绑死的，所以这条路比 `abi` 强类型、报错也早，胶水需要编入宿主或 AOT 产物；AOT 可直接通过 –glue 指定文件。

21.5.1 `impl`：把宿主函数绑定到协议

结构体用继承列表声明普通接口实现；`impl 类型：接口 = 宿主绑定名;` 则把宿主函数接到单方法接口或转换协议。它不是一套任意 `impl Type { ... }` 语法。

下面使用工具链自带的演示胶水。实际项目需要先在 C++ 注册同名绑定，不能只写声明就假设运行时存在它。

```
type HostHandle<T> = uint64;
impl uint64 : ExplicitConvert<HostHandle<int32>> = demo_counter_from_uint;
impl HostHandle<int32> : ExplicitConvert<uint64> = demo_counter_to_uint;

interface HostValue
{
    int64 hostValue();
}
impl HostHandle<int32> : HostValue = demo_counter_to_uint;
HostHandle<int32> demo_cpp_add(HostHandle<int32> left, HostHandle<int32> right);

int64 main()
{
    HostHandle<int32> left = cast<HostHandle<int32>>((uint64)20);
    HostHandle<int32> right = cast<HostHandle<int32>>((uint64)22);
    HostHandle<int32> total = demo_cpp_add(left, right);
    println("{} {}", cast<uint64>(total), total.hostValue());
    return 0;
}
```

```
42 42
```

接口签名、宿主参数和返回值必须一致。同步回调通过 `NativeCallContext::invoke` 与 `magel_glue::Function` 使用同一套 VM/AOT 规则；不要把函数值中的运行时地址当作宿主机器函数指针。完整胶水约定见宿主集成。

21.6 反方向：宿主调用 Magel

宿主有三种可用方式：

方式	调用路径
独立 VM	宿主加载.svmb 和数据区，注册胶水，由 Machine 执行；适用于当前 MCU 工程
AOT C 导出	Magel 用 export 声明，AOT 生成库和 C 头文件，宿主按头文件创建上下文并调用导出函数
同步回调	Magel 将具名函数或 fn 值传给 C++ 胶水，胶水用 Function 包装或 Context::invoke 调回；VM/AOT 共用

库可以没有 main，但至少有一个 export。按后缀输出.dll / .so 共享库或.o 可重定位文件，并生成同名 C 头文件；.o 需要宿主链接器，不是动态加载器直接加载的共享库。不支持.lib / .a 静态库。上下文初始化不自动调用源码的 main。

export 只供 AOT，VM 执行和字节码导出遇到它会报错。普通同步回调不要求 export，支持闭包、高阶函数、引用和多层重入；函数值与上下文只在所属宿主回调期间同步使用。

AOT 的 static runMagel 是生成 C++ 内部的整程序入口，不能当作通用动态库 C 导出或字节码加载器。完整 API 与可运行代码见宿主集成及胶水回调示例。

21.7 小结

你会了	关键点
三种集成	`abi` 块（轻）/ 宿主胶水（要编 C++）/ AOT 嵌入（宿主调 Magel）
`abi` 语法	`abi "./x.dll" { 返回类型名字（形参）= "C 符号名"; }`
名字映射	等号右边是 C 侧真实符号名，左边随你起
结构体	按值传；Magel 侧声明字段一致的 `struct`
引用	C 的 `T *` ↔ Magel 的 `T mut&`，调用处写 `&x`
字符串	C 的 `const char *` ↔ Magel 的 `String`
数组	`T[]` + 长度另给；返回长度用 `[count]` 标注
「空」	只有 `Option<T &>` 能把空送过边界
紧凑布局	`struct X packed { …}` 对接 `#pragma pack`
胶水	`bindMagelGlue` 注册 C++ 函数；复杂对象用整数句柄
反向	VM 宿主执行、AOT export C 接口，或胶水同步回调 fn

21.8 下一步

你能调 C 了、能被 C 调了。最后一个技术问题是：怎么把程序交出去——变成对方机器上能跑的东西。

第 22 章讲交付与部署：`.svmb`、AOT、符号压缩，以及嵌入式与看门狗。

翻到第 22 章交付与部署。

第 22 章 交付与部署

代码写完了，怎么把它交出去？交给同事、交给客户、交到一块板子上——这三种情况要的东西不一样。这一章讲 Magel 给的三种跑法和它们的加固手段。

22.1 三种跑法

同一份 Magel 逻辑，有三种执行方式：

方式	命令	启动速度	需要什么才能跑	适合
解释执行	`magel run app.mgl`	每次都要编译	Magel 工具链	开发、脚本、工具
字节码	`magel svmb app.mgl …`	快	StackVM + `.svmb`	交付、嵌入式、热更新
AOT 本地程序	`magel aot app.mgl …`	不经过 VM 装载	运行时服务编入产物，外部依赖仍须准备	本地交付、性能敏感

一句话选型：开发用 `run`，要交给别人用 `.svmb`，要本地代码交付用 AOT。

这一章下面都用同一个 hello 程序举例：

```
int64 main()
{
    println("hello, delivery");
    return 0;
}
```

```
hello, delivery
```

22.2 解释执行：开发者的默认档

```
magel run src/main.mgl
```

预处理 → 解析检查 → 降级成四元式 → 在同一个进程里跑起来。改完立刻看结果，不用等任何构建。前面二十一章你一直用的是这个。

它的问题也在这：每次运行都要重新编译一遍。给用户交付时，你不能要求对方装一整套工具链。

22.3 字节码：`.svmb`

```
magel svmb src/main.mgl --out build/app
```

产物是一组文件，其中主角是 build/app.svmb——交给 StackVM 加载执行。这是交付的主推形态，原因有三：

1. 不需要编译器：只要一个 VM（几百 KB）就能跑；
2. 跨平台：同一份 .svmb 在各平台的 VM 上都能跑，不用维护多份构建；
3. 可加固：能压缩符号（见 22.5）。

--out 决定产物前缀，同时会生成配套的 .quad / .data.bin / .layout。下发给客户默认只需要那个 .svmb。

> 提示
>
> magel svmb 默认不带调试节——它会明确告诉你「不带调试节（缺省）」。要留着调试信息，用 --debug。

22.4 AOT：编成本地可执行文件

需要库时，源码可省略 main，使用 export 声明公开函数，再按产物选择命令：

```
magel aot math.mgl --out math.dll  # Windows GCC 目标
magel aot math.mgl --out math.so   # Linux GCC 目标
magel aot math.mgl --out math.o    # 目标文件，供宿主链接
```

三者均附带 C 头文件，宿主创建上下文后调用导出函数。–shared / –object 可显式选择类型。.o 不能直接由动态加载器加载；最终链接需准备同一工具链的 C++ 运行时与系统库。不支持.lib / .a 静态库输出，修改扩展名不能转换 Windows / Linux ABI。

```
magel aot src/main.mgl out build/native
```

生成的 build/native 就是一个独立的本地程序，不要求另装 Magel VM；所需运行时服务已编入产物，系统库和使用的外部依赖仍须准备：

```
已生成可执行文件：build/native（运行方式：aot）
```

```
./build/native
```

```
hello, delivery
```

AOT 的做法是先生成 C++，再编成目标平台的本地程序，所以机器上要有 C++ 工具链。

常用选项：

选项	作用
out 路径 / --out 路径	产物路径
opt LEVEL / --opt-level=N	优化级别（默认保持数值语义的 -O3；对照调试时可显式降低）
checked / unchecked	是否保留运行时检查（默认 checked）
--separate-modules	按模块缓存目标文件，增量更快
--module-cache DIR	指定模块缓存目录

```
magel aot src/main.mgl out build/native checked
```

```
优化级别：-O3
```

默认使用 checked，保留分配有效性检查和 panic 调用栈。显式使用 unchecked 可关闭这些可选检查；数组下标与基础地址检查仍然保留。性能对比应使用相同检查配置。

22.5 交付安全：符号压缩

普通 .svmb 保存可读符号名；对外交付时可以导出无名变长符号格式：

```
magel svmb src/main.mgl --out build/app --compress-symbols
```

映射表另存为 <前缀 >.symbol_map.json，用 --symbol-map FILE 可指定路径。映射表由开发方保存，用于离线定位问题。

符号压缩减少体积并移除可读名字，但可执行逻辑仍可被分析。字节码加密功能已删除。交付形态及边界见交付安全。

22.6 嵌入式：同一份逻辑，压到板子上

板子跑不了桌面那套内存预算，也不想为了省内存换一套裁剪过的语言。Magel 的做法是共享 VM 核心，前端配置控制模块门禁与资源上限，独立 MCU 宿主还通过宏、内存池和胶水选择平台能力。配置写进 magel.toml：

```
[embedded]
enabled = true
restrict_modules = ["std::math"]      # 在内置禁用之外再追加
max_memory = 1048576                  # 运行内存上限（字节）
```

门禁在编译期。程序一 import 被禁的模块，直接编译报错、带源码位置，不会等到上板才发现：

```
1 import std::net;
2
3 int64 main()
4 {
5     println("net 打开了");
6     return 0;
7 }
```

在开了 [embedded] 的工程里跑，会得到：

```
magel: 模块: 嵌入式模式不允许使用模块 std::net: 它在嵌入式运行时里没有后端;
诊断位置及具体文案以当前编译器为准; restrict_modules 只追加限制
```

前端配置固定禁用清单：std::net、std::process、std::sql、std::io、std::thread、std::coroutine、std::concurrency。

restrict_modules 只能追加，不能解除内置禁用——这是故意的：这是前端配置的固定限制，不表示独立 MCU VM 的全部能力；后者可按实际配置接 FatFS 或协作式协程。

去掉那行 import std::net;，同一个工程照常跑：

嵌入式也能跑普通程序

产物仍是普通的 .svmb，共同支持的功能遵循相同语义。目标 VM 的特征位、模块、胶水和内存预算须匹配，默认无反射的板端要求源程序显式 –no-reflection。

22.7 内存与看门狗

长期运行的设备最怕「跑着跑着崩了」。这组开关专治它：

选项	作用
`--memory-size 字节`	运行时堆容量（缺省 512 MiB，最小 64）
`--watchdog`	打开软重启看门狗
`--no-watchdog`	关闭（缺省）
`--max-restarts N`	最多软重启几次（缺省 3，隐含 `--watchdog`）
`--watchdog-watermark N`	内存水位（字节，隐含 `--watchdog`）
`--step-limit N`	本次执行的指令条数上限

```
magel run app.mgl --memory-size 1048576
magel run app.mgl --watchdog --max-restarts 2
```

两行都照跑，输出 hello，delivery。

看门狗的思路和「让进程崩掉」不一样：它在分配失败 / 显式请求 / 超过内存水位时重跑一遍程序，而不是让程序死在那里。这样一次偶发故障最多退化成「逻辑重启一次」，进程本身活着。配合 `--max-restarts` 限制次数，避免无脑重启。

`--step-limit` 则是「这一趟最多跑多少条指令」——超了就停，防止某个死循环拖垮设备。

22.8 把 Magel 嵌进宿主程序

AOT 生成的 C++ 里带着一个可以直接调的入口：

```
1 static int runMagel(int argumentCount, const char *const *arguments,
2                     void *buffer, std::size_t memoryBytes);
```

生成的代码里，`int main(...)` 被 `#ifndef MAGEL_NO_MAIN` 包着——定义 `MAGEL_NO_MAIN` 再编译，就只剩 `runMagel` 给你的固件调用，不带自己的 `main`。

典型用法：固件启动时划一块静态内存，反复调 `runMagel` 跑业务逻辑；该 static 入口供集成生成 C++ 的同一翻译单元调用。AOT 逻辑更新后须重编本地程序；要在 VM 中替换程序，加载新字节码映像，不承诺运行中栈、闭包和对象的迁移。

参数	含义
`argumentCount` / `arguments`	传给 Magel `main` 的参数（不含程序名）
`buffer`	运行内存缓冲区；传 `nullptr` 就让程序自己分配
`memoryBytes`	缓冲区大小 = 运行内存上限

这是「C 调 Magel」的主线（第 21 章讲的三个方向之一）。

22.9 选型速查

场景	推荐
开发调试	解释执行 `magel run`
桌面交付给最终用户	AOT（`checked` / `unchecked` 按需）
要给外部客户的交付件	`.svmb` + `--compress-symbols`
跨平台、不想维护多份构建	`.svmb`
嵌入式板端	.svmb + 共享核心的 MCU 宿主，按需使用前端模块门禁
长期运行的设备	加 `--watchdog` / `--step-limit`
需要替换业务程序	VM 加载新.svmb；宿主管理停止、清理与重新启动

22.10 小结

你会了	关键点
三种跑法	解释（开发） / `.svmb`（交付、嵌入式） / AOT（本地代码与编入的运行时服务）
`magel svmb`	产物前缀 `--out`；默认不带调试节
`magel aot`	生成 C++ 再编本地程序；`checked/unchecked`、`opt`
压缩符号	`--compress-symbols`，另存 `<前缀 >.symbol_map.json`（要留好）
嵌入式	`magel.toml` 的 `[embedded]`；编译期门禁；不换 VM
资源限制	`--memory-size`、`--step-limit`
看门狗	`--watchdog` / `--max-restarts` / `--watchdog-watermark`
嵌入宿主	AOT + `MAGEL_NO_MAIN` + `runMagel`，静态内存反复调用

22.11 下一步

语言、工具链、交付，都讲完了。最后一章把前面二十一章的东西串起来，从零写一个真能用的程序：命令行待办清单。

翻到第 23 章综合项目。

第 23 章 综合项目：命令行待办清单

前二十二章，每一章解决一个问题。这一章把它们串起来：从零写一个真能用的小工具——命令行待办清单。

它要能：

- 加一条待办、列出全部、标记完成、删掉一条；
- 把数据存到文件里，下次打开还在；
- 处理「文件不存在」「序号不合法」这些真实世界的小意外。

写完你会发现：几乎没有新语法——全是前面学过的东西。

23.1 先想清楚数据长什么样

一个待办项有两样东西：内容和一个完成标记。用结构体（第 9 章）：

```
struct Task
{
    String text;
    bool done;
};
```

清单就是 `List<Task>`（第 11 章）。

存储格式也要现在定下来——这是设计里最容易忽略、又最影响后面代码的一步。选一个人能直接看懂的格式：一行一条，0 或 1 表示完成，| 后面是内容。

```
1|买菜
0|写作业
```

这样出问题时，用记事本打开就能查。

23.2 读：把文件变回清单

读文件用 `FileStream`（`import std::io;`，第 18 章讲的 import）。

这里有个关键设计决定：文件不存在 = 空清单，不当错误。

```
List<Task> loadTasks()
{
    List<Task> tasks;
    Result<FileStream, IoError> opened = FileStream::open(storePath(), "r");
    if (opened.isErr())
    {
        return tasks;
    }
    FileStream stream = opened.unwrap();
    while (true)
    {
        Result<String, IoError> line = stream.read();
        if (line.isErr())
        {
            break;
        }
        String raw = std::text::Text::trim(line.unwrap());
        if (raw == "")
        {
            break;
        }
        Task t;
        t.done = std::text::Text::startsWith(raw, "1");
        int64 bar = std::text::Text::find(raw, "|");
        t.text = raw[bar + 1:];
        tasks.push(t);
    }
    stream.close();
    return tasks;
}
```

三处值得说：

- `FileStream::open` 返回 `Result`（第 16 章）。打不开不是崩溃，是一个可以处理的结果——这正是 `Result` 存在的意义；
- `stream.read()` 一次读一行。读到头（EOF）返回的是空串 `Ok("")`，不是错误——所以退出条件写成 `if (raw == "") { break; }`。这是实测踩出来的，别想当然写成 `isErr`；
- `raw[bar + 1:]` 是切片（第 12 章）。`|` 是 ASCII，字节边界天然合法，所以这里按字节切中文内容也没问题。

23.3 写：把清单存回文件

反过来，写文件是先打开、逐行写、关闭：

```
void saveTasks(List<Task> tasks)
{
    Result<FileStream, IoError> opened = FileStream::open(storePath(), "w");
    if (opened.isErr())
    {
        println("无法写入 {}", storePath());
        return;
    }
    FileStream stream = opened.unwrap();
    foreach (Task t in tasks)
    {
        String flag = "0";
        if (t.done)
        {
            flag = "1";
        }
        stream.write(flag + "|" + t.text + "\n");
    }
    stream.close();
}
```

模式 `"w"` 是覆盖写——每次保存都整份重写。对几千条以内的清单，这足够快，而且不会出现”改一半”的中间状态。

`foreach (Task t in tasks)` 拿到的 `t` 是元素的副本（第 6 章值语义），因此这里只读不改；要改得按下标写回去（`done` 命令就这么做的）。

23.4 命令分派：main 是入口

程序接收命令行参数，靠的是带参的 `main`（第 4 章）：

```
int64 main(List<String> args)
{
    …
}
```

`args` 是不含程序名的参数列表。所以 `todo add 买菜` 传进来的是 `["add", "买菜"]`。

命令分派就是一串 `if`（第 5 章），每种命令都返回一个退出码：

```
String command = args[0];
List<Task> tasks = loadTasks();

if (command == "add")
{
    String text = joinFrom(args, 1);
    if (text == "")
    {
        println("add 后面要写内容");
        return 1;
    }
    Task t;
    t.text = text;
    t.done = false;
    tasks.push(t);
    saveTasks(tasks);
    println("已添加，现在有 {} 条", tasks.len());
    return 0;
}
```

两个小设计：

- add 后面的内容可以带空格：joinFrom(args, 1) 把第 2 个参数起的部分用空格拼起来（std::text::Text::join）。于是 add 周五下午交报告存的就是一整句；
- 每一档都 return：用返回码告诉外面「成功（0）」还是「用法错（1）」——这正是第 1 章 main 返回 0 那个约定的用法。

序号参数是字符串，要转成整数：std::parse::Parse::parseInt64 返回 Result，所以用第 16 章的 isErr 来兜底：

```
int64 parseIntOr(String s, int64 fallback)
{
    Result<int64, ParseError> r = std::parse::Parse::parseInt64(s);
    if (r.isErr())
    {
        return fallback;
    }
    return r.unwrap();
}
```

传个 done abc 进来，parseInt64 失败 → 返回兜底值 0 → 后面「序号超出范围」的判断把它挡住。非法输入没有变成崩溃，而是变成一句人话提示。

23.5 完整程序

把上面几块拼起来。整份代码（这段可以整段复制出去跑）：

```
import std::io;
import std::text;
import std::parse;

struct Task
{
    String text;
    bool done;
};

String storePath()
{
    return "todo.txt";
}

List<Task> loadTasks()
{
    List<Task> tasks;
    Result<FileStream, IoError> opened = FileStream::open(storePath(), "r");
    if (opened.isErr())
    {
        return tasks;
    }
    FileStream stream = opened.unwrap();
    while (true)
    {
        Result<String, IoError> line = stream.read();
        if (line.isErr())
        {
            break;
        }
        String raw = std::text::Text::trim(line.unwrap());
        if (raw == "")
        {
            break;
        }
        Task t;
        t.done = std::text::Text::startsWith(raw, "1");
        int64 bar = std::text::Text::find(raw, "|");
```

```
        t.text = raw[bar + 1:];
        tasks.push(t);
    }
    stream.close();
    return tasks;
}

void saveTasks(List<Task> tasks)
{
    Result<FileStream, IoError> opened = FileStream::open(storePath(), "w");
    if (opened.isErr())
    {
        println("无法写入 {}", storePath());
        return;
    }
    FileStream stream = opened.unwrap();
    foreach (Task t in tasks)
    {
        String flag = "0";
        if (t.done)
        {
            flag = "1";
        }
        stream.write(flag + "|" + t.text + "\n");
    }
    stream.close();
}

void printTasks(List<Task> tasks)
{
    if (tasks.len() == 0)
    {
        println("（清单是空的，用 add 加一条）");
        return;
    }
    int64 i = 1;
    foreach (Task t in tasks)
    {
        String mark = "[ ]";
        if (t.done)
        {
            mark = "[x]";
```

```
        }
        println("{} {} {}", i, mark, t.text);
        i = i + 1;
    }
}

int64 parseIntOr(String s, int64 fallback)
{
    Result<int64, ParseError> r = std::parse::Parse::parseInt64(s);
    if (r.isErr())
    {
        return fallback;
    }
    return r.unwrap();
}

String joinFrom(List<String> args, int64 start)
{
    List<String> rest;
    for (int64 i = start; i < args.len(); i = i + 1)
    {
        rest.push(args[i]);
    }
    return std::text::Text::join(rest, " ");
}

void usage()
{
    println("用法: todo <命令> [参数]");
    println("  add <内容>    加一条待办");
    println("  list          列出全部");
    println("  done <序号>   标记完成");
    println("  remove <序号> 删除一条");
    println("  clear         清空");
}

int64 main(List<String> args)
{
    if (args.len() == 0)
    {
        usage();
        return 0;
```

```
124     }
125
126     String command = args[0];
127     List<Task> tasks = loadTasks();
128
129     if (command == "add")
130     {
131         String text = joinFrom(args, 1);
132         if (text == "")
133         {
134             println("add 后面要写内容");
135             return 1;
136         }
137         Task t;
138         t.text = text;
139         t.done = false;
140         tasks.push(t);
141         saveTasks(tasks);
142         println("已添加，现在有 {} 条", tasks.len());
143         return 0;
144     }
145
146     if (command == "list")
147     {
148         printTasks(tasks);
149         return 0;
150     }
151
152     if (command == "done")
153     {
154         if (args.len() < 2)
155         {
156             println("done 后面要写序号");
157             return 1;
158         }
159         int64 index = parseIntOr(args[1], 0);
160         if (index < 1 || index > tasks.len())
161         {
162             println("序号超出范围");
163             return 1;
164         }
165         Task t = tasks[index - 1];
```

```
        t.done = true;
        tasks[index - 1] = t;
        saveTasks(tasks);
        println("完成: {}", t.text);
        return 0;
    }

    if (command == "remove")
    {
        if (args.len() < 2)
        {
            println("remove 后面要写序号");
            return 1;
        }
        int64 index = parseIntOr(args[1], 0);
        if (index < 1 || index > tasks.len())
        {
            println("序号超出范围");
            return 1;
        }
        List<Task> kept;
        int64 i = 1;
        foreach (Task t in tasks)
        {
            if (i != index)
            {
                kept.push(t);
            }
            i = i + 1;
        }
        saveTasks(kept);
        println("已删除第 {} 条", index);
        return 0;
    }

    if (command == "clear")
    {
        List<Task> empty;
        saveTasks(empty);
        println("已清空");
        return 0;
    }
```

```
208
209     println("未知命令：{}", command);
210     usage();
211     return 1;
212 }
```

直接 magel run todo.mgl（不带参数），它会打印用法：

```
用法：todo <命令> [参数]
  add <内容>     加一条待办
  list           列出全部
  done <序号>    标记完成
  remove <序号>  删除一条
  clear          清空
```

把这段存成 todo.mgl，下面就真能用了。

23.6 跑一遍

先清空，然后加两条、看一眼：

```
magel run todo.mgl -- clear
magel run todo.mgl -- add 买菜
magel run todo.mgl -- add 写作业
magel run todo.mgl -- list
```

```
已清空
已添加，现在有 1 条
已添加，现在有 2 条
1 [ ] 买菜
2 [ ] 写作业
```

> **提示**
>
> -- 后面的东西原样传给程序自己（第 1 章讲过 run 和 check 的区别时提过参数传递）。没有 --，clear 会被当成 magel 的选项。

标记完成、再看、然后删掉第一条：

```
magel run todo.mgl -- done 2
magel run todo.mgl -- list
magel run todo.mgl -- remove 1
magel run todo.mgl -- list
```

```
完成：写作业
1 [ ] 买菜
2 [x] 写作业
已删除第 1 条
1 [x] 写作业
```

数据真的落盘了——todo.txt 里是这样（人能直接读懂）：

```
1|写作业
```

再试几个「不老实」的输入：

```
magel run todo.mgl -- done abc
magel run todo.mgl -- add 周五 下午 交 报告
magel run todo.mgl -- list
```

```
序号超出范围
已添加，现在有 2 条
1 [x] 写作业
2 [ ] 周五 下午 交 报告
```

done abc 没有崩，而是回了一句「序号超出范围」；add 后面带空格的整句也完整存下来了。

关掉重开，数据还在——这就是 loadTasks / saveTasks 那二十来行的价值。

23.7 复盘：这个项目用了哪些章

用到的	哪一章
int64 main(List<String> args) 入口	4 · 函数
struct Task { …};	9 · 结构体
List<Task>、push、下标读写、len	11 · 容器
String 拼接、切片、trim、startsWith、find、join	12 · 字符串
if / foreach / for / while	5 · 控制流
值语义（t 是副本；改完写回）	6 · 值语义
Result / isErr / unwrap 处理失败	16 · 错误处理
FileStream 读写文件	18 · 模块与工程（import）
std::parse 字符串转整数	19 之后的附录 B（标准库）

没有出现：接口、泛型、fn、宏、反射……它们不是必需，只是在需要的时候有用。一个能用的程序，往往是这些基础件的直白组合。

23.8 接下来可以做什么

这本书到这儿结束了，但这个程序没有。给你几个练手方向，每一个都对应书里的一章：

想加的功能	该复习
加优先级 / 分类 → 加字段和 `enum`	10 · 枚举与模式匹配
加一条「按关键词过滤」	17 · 迭代器（`filter`）
把 `Task` 的打印逻辑收进方法	9 · 结构体（方法）
让输出对齐（序号补零）	12 · 字符串（`padStart`）
抽出 `Store` 接口，支持「文件 / 内存 / 数据库」三种后端	13 · 接口
让清单支持任意元素类型	14 · 泛型
一行 `sort` / `map` 的写法	17 · 迭代器
用 `magel new` 起工程、把命令拆成模块	18 · 模块与工程
给函数写测试文件 + `magel test`	18 · 模块与工程
打包成 AOT 可执行文件发给别人	22 · 交付与部署

回顾一下你走过的路：

- 第 1–2 章：跑起来，把小块知识串成第一个程序；
- 第 3–5 章：语言基础（类型、函数、控制流）；
- 第 6–8 章：Magel 的骨头（值语义、引用、资源与作用域）——这三章决定了你和这门语言相处得顺不顺；
- 第 9–12 章：组织数据；
- 第 13–16 章：抽象（接口、泛型、函数值、错误处理）；
- 第 17–20 章：工程与工具链；
- 第 21–22 章：与外部世界打交道、把程序交给别人；
- 第 23 章：把它们拼起来。

现在你已经能独立写 Magel 程序了。遇到语法拿不准的，翻语言参考；想查某个标准库函数，翻库函数 API 参考；想看官方给的可运行样例，翻 `example/samples/`。

继续阅读学生信息管理系统和单片机上的 Magel 解释器，把同一套语言用于浏览器应用和板端宿主。

第 24 章 学生信息管理系统：前后端分离

第 23 章把业务放在命令行里。这一章把交互搬到浏览器：浏览器负责显示和表单，Magel 负责 HTTP、JSON、权限与数据库。我们直接使用仓库的前后端分离项目，不另写一套只有页面的演示。

24.1 先找到三个入口

入口	职责
main.mgl	Magel API 服务、登录会话、路由、业务校验、MySQL 访问
web/index.html	页面外壳与登录表单
web/app.js	调接口、渲染表格、提交表单
web/config.js	后端地址配置

一次请求经过浏览器、HTTP API、权限判断和 SQL，结果再以 JSON 返回。前端静态服务器与 API 是两个进程、两个端口；API 的 / 返回 404，页面由静态服务器提供。

这个项目使用 MySQL。仓库还有一个更小的 SQLite Web 示例，它由同一服务提供页面，适合先练增删改查；本章讲的跨源部署、会话与权限来自前后端分离项目。

24.2 准备独立练习数据库

先构建 Magel，并准备能连接的 MySQL 服务及客户端库。MySQL 客户端库路径可用 `MYSQL_LIBRARY` 指定，具体连接方式见 SQL 示例说明。

仓库内的练习夹具包含五张表及四种角色账号。脚本含有删除并重建表的语句，下面明确使用独立数据库 `magel_book_students`；不要把该参数改成已有业务库。

以下命令从 Magel 仓库根目录运行。第一个终端启动后端：

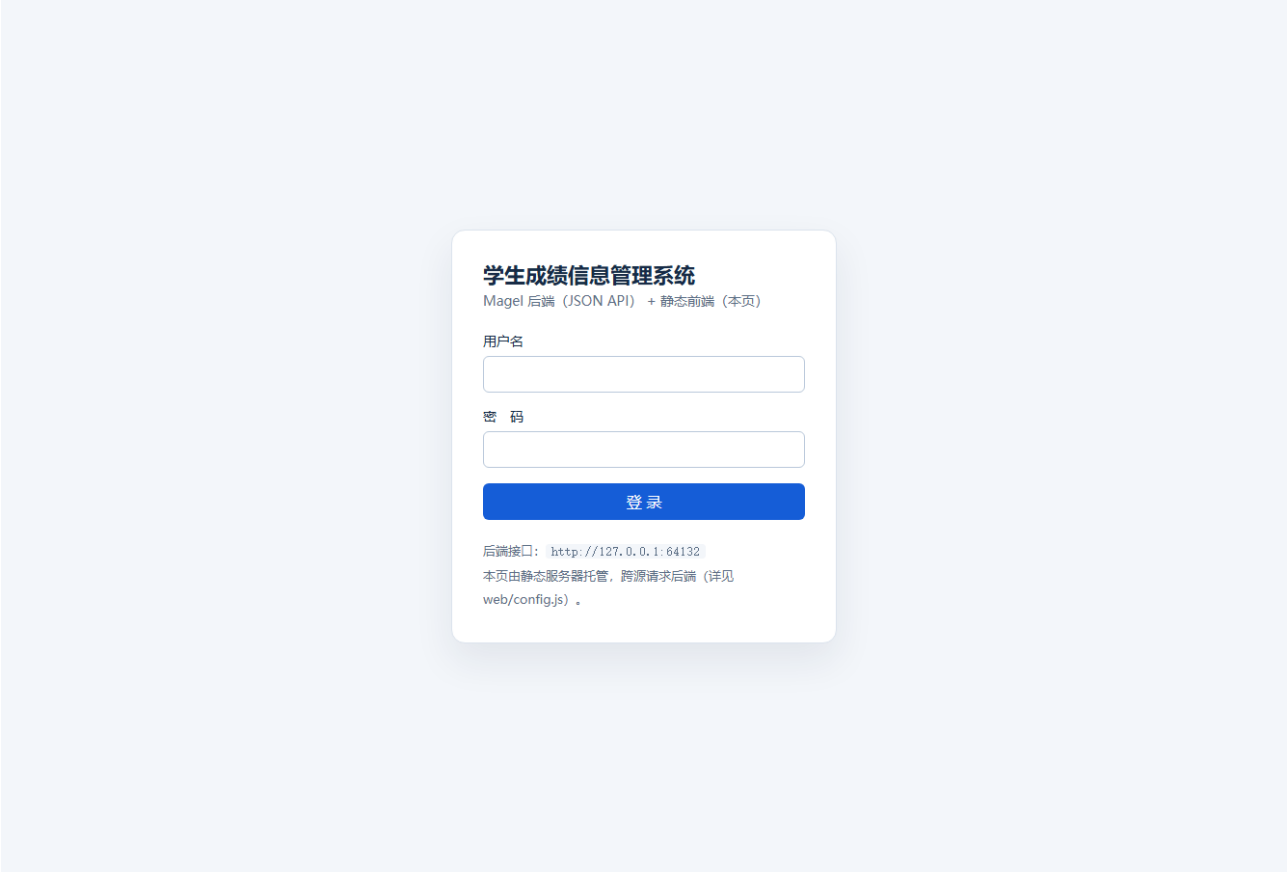
```
./magel.exe run example/sql-magel/example/ipage-student-server/main.mgl -- --db magel_book_students
    --mysql-user root --mysql-password 123456 --init example/sql-magel/tests/fixture_ipage.sql
    --listen-port 8080 --allow-origin http://127.0.0.1:5500
```

将 MySQL 用户与口令替换为本机练习环境的配置。初始化只在缺少业务表时执行；重复练习需要重置数据时，先确认数据库名称，再显式添加 `--reinit`。

第二个终端托管静态页面：

```
python -m http.server 5500 --bind 127.0.0.1 --directory
    example/sql-magel/example/ipage-student-server/web
```

打开 `http://127.0.0.1:5500/`。页面默认访问 `http://127.0.0.1:8080`；后端换端口时，可用查询参数 `?api=http://127.0.0.1:9000` 或修改 `web/config.js`。浏览器页面上的 API 地址可以帮助核对两端是否连接正确。



前后端分离项目的登录界面，页面显示独立的 API 地址

图中的端口来自自动回归启动时分配的本地端口，不必与手工练习的 5500 和 8080 相同。



浏览器与 API 的跨源请求、登录会话和后端权限判定流程

24.3 先把接口跑通

操作	接口	关注点
健康检查	`GET /api/health`	实际执行数据库查询
登录	`POST /api/login`	JSON 字段 `userName`、`password`
当前会话	`GET /api/session`	返回用户、模块和动作权限
学生列表	`GET /api/students`	返回 `items`、`count`、`total`、`limit`
新增学生	`POST /api/students`	教师及以上角色可用
修改学生	`PUT /api/students/{id}`	校验字段和班级关系
删除学生	`DELETE /api/students/{id}`	管理员及以上角色可用

登录成功后取得令牌，其余受保护请求使用 `Authorization: Bearer <token>`。令牌存放在后端内存会话表中，后端重启后失效。先看 `Request`、`Response`、`Session` 类型，再沿 `route` 与各模块分发函数阅读，比从两千多行文件的第一行读到底更容易建立调用关系。

浏览器带 Authorization 调用另一个源，会触发 CORS 预检。后端处理 OPTIONS 并返回允许的源、方法和请求头；只放行页面所在的来源并不替代登录或权限检查。

24.4 权限必须在后端生效

夹具里的学生账号为“方智堃 / zhongli”，超级管理员账号为“admin / 123456”，均仅用于演示数据。学生登录后能看到学生、成绩和修改密码三个入口；看不到新增和删除按钮。



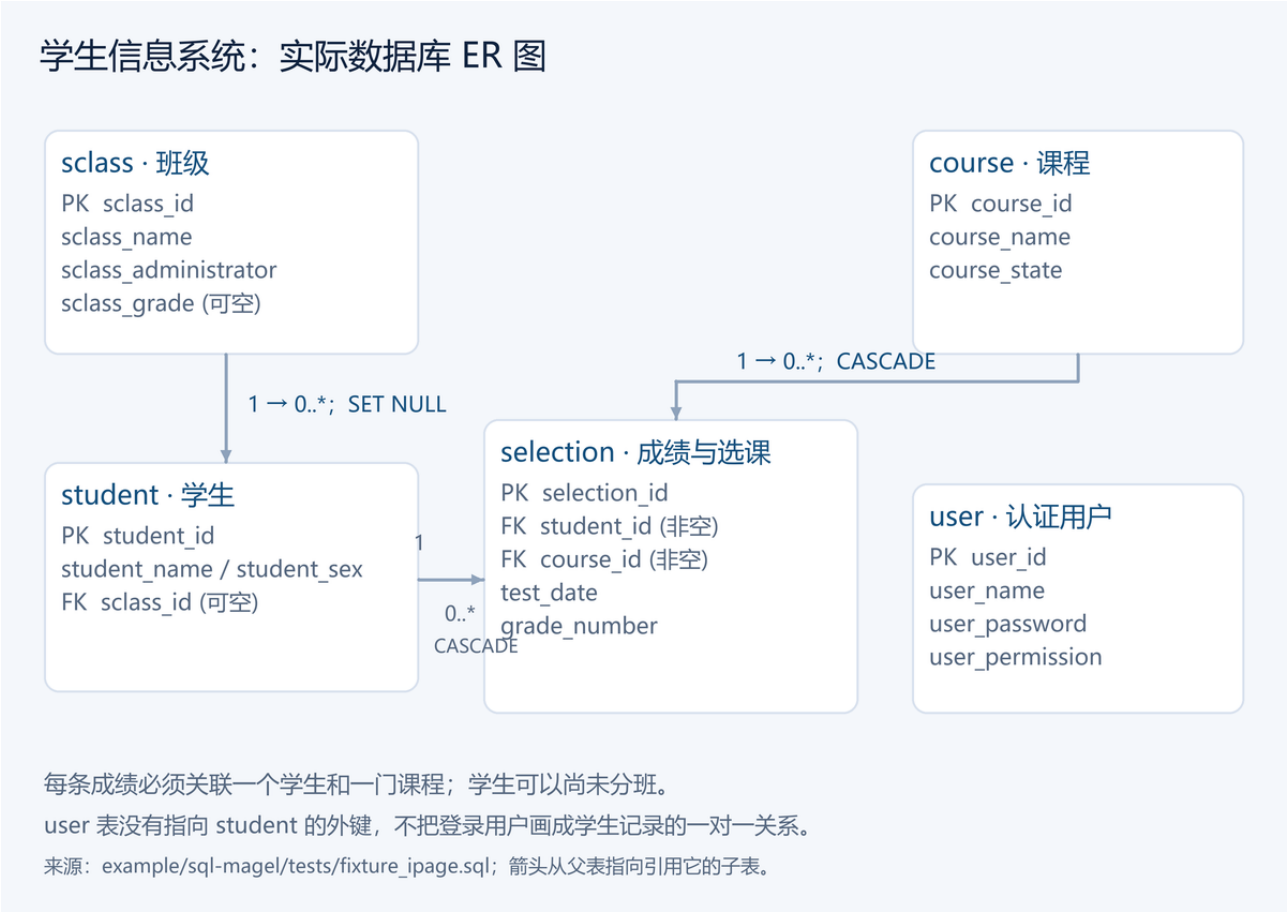
学生角色查看学生信息，侧栏与操作按钮按权限收缩

权限等级为学生 0、教师 2、管理员 3、超级管理员 4。actionNeedLevel 查询动作门槛，后端按“用户等级大于等于门槛”执行判定。前端读取服务端返回的模块与动作，只负责显示合适的入口。即使手工构造 HTTP 请求，后端仍会拒绝越权动作。

练习时先登录学生，再登录超级管理员，对照同一张学生表的按钮变化。接着尝试直接调用一个权限不足的接口，观察 403，而不能仅凭“按钮不见了”认定权限已经正确。

24.5 数据关系与完整操作

表	作用	删除时的关系
`user`	登录用户与角色	受用户模块规则约束
`sclass`	班级	删除后学生变为未分班
`student`	学生	删除时级联删除对应成绩
`course`	课程	删除时级联删除对应成绩
`selection`	选课、考试日期与成绩	关联学生和课程



依据实际 SQL 绘制的学生信息系统 ER 图，标明可空外键与删除行为

ER 图中的 `user` 是认证表，没有指向学生表的外键。`selection` 是学生与课程之间的关联记录，表结构没有把学生和课程组合声明为唯一键；重复录入的拒绝由应用规则执行。

用管理员账号新增一名学生，编辑姓名与班级，按姓名查询，最后删除。再打开成绩模块的按课程统计，核对人数、平均分、最高最低分与及格率。



后端 SQL 聚合结果在成绩统计页显示

统计由后端查询数据库产生，页面负责展示。这样浏览器没有加载全部成绩时，也能取得完整统计。删除接口返回级联或脱离关系的计数，确认框可以说明本次操作将影响什么。

输入校验也在后端执行：主键必须为正整数，姓名等文本按数据库字符长度限制，枚举字段只接受约定值。前端用 `textContent` 展示用户数据，避免把姓名当作 HTML。阅读实现时，分别找“表单可用性”“业务校验”“数据库约束”，它们各自解决不同层次的问题。

24.6 沿 main.mgl 读完一次请求

现在回到 `main.mgl`，把前面的界面、ER 图与真实代码对应起来。这个文件没有使用 Web 框架，HTTP 协议、会话和业务分发都能在同一文件里找到。建议先读文件末尾的 `main`，再沿函数调用往回追。

24.6.1 main：先准备长期状态，再处理短期请求

入口是 `int64 main(List<String> arguments)`。参数循环识别监听地址、端口、MySQL 连接、数据库名称、初始化文件和跨源来源。`decimal` 返回 `Option<int64>`，端口必须在 0-65535 内；它逐字检查十进制数字，并在乘十之前判断是否会超出上限，不把错误输入悄悄变成 0。

参数读取完成后，`main` 创建一个 `App app`。`App` 把数据库连接、`List<Session>` 会话表、随机状态和 `allowOrigin` 集中在一起，生命周期覆盖整个服务。`connect` 建立 MySQL 连接、设置 `utf8mb4`、选择数据库；`ensureSchema` 判断业务表是否就绪，必要时读取初始化 SQL。失败路径调用 `app.stop()` 关闭连接并返回。

随后才创建 TcpListener。当配置端口为 0 时，必须用 listener.localPort() 取得系统实际分配的端口；就绪文件写入的也是这个值，测试客户端据此连接。主循环的核心代码如下，摘自实际入口：

```
while (maximum == 0 || handled < maximum)
{
    TcpSocket socket = listener.accept((int32)1000);
    if (!socket.isOpen())
    {
        continue;
    }
    Request request = readRequest(&socket, actualPort);
    Response response = route(&app,&request);
    bool sent = sendResponse(&socket,&response, app.allowOrigin);
    socket.close();
    handled = handled + 1;
}
```

每条连接依次经过接收、路由、发送、关闭。maximum 为 0 时持续服务，正数用于确定性测试；请求处理完毕才增加计数。当前代码把发送结果保存为 sent，但没有重试发送失败的响应。循环退出后，入口再关闭监听器并调用 app.stop()。

注意两个寿命：App 和监听器存在于循环外，单次请求的 socket、request、response 存在于循环内。这样下一次请求可以继续使用数据库和会话，但不会共享上一条请求的临时正文。

24.6.2 Request 与 readRequest：TCP 字节不等于完整 HTTP 请求

Request 保存解析后的方法、路径、查询串、正文、媒体类型、Authorization 以及解析状态；Response 只保存状态码、媒体类型和正文。把协议层结果集中成值对象后，业务函数就不必再次切分原始报文。

readRequest 首先循环调用 socket.receive(4096)，把数据追加到 raw，直到找到 \{}r\{}n\{}r\{}n。一次 TCP receive 可能只取得半个请求头，也可能同时带回部分正文，所以不能用“一次 receive 就读完一个请求”的假设实现 HTTP。

请求头结束后，函数拆分请求行，校验 HTTP/1.0 或 HTTP/1.1，分离 path 与 query，再逐行检查头部。它拒绝重复 Content-Length、非法 Host、不支持的 Transfer-Encoding 和 Expect；POST/PUT 缺少 Content-Length 时返回 411。随后继续接收，直到正文达到声明长度，再按该长度截取。

这些失败不会直接执行 SQL，而是写入 request.status。route 开头看到非零状态，就生成对应的错误响应。协议层与业务层因此只有一个明确交接点。

这仍是有边界的示例：请求头与正文有上限，连接逐条处理；accept 的等待时间不等于已接收连接具有完整的读超时。一个迟迟不发完正文的客户端仍可能阻塞后续请求，不能把这段循环描述成并发 HTTP 服务。

24.6.3 route：先处理公开入口，再查会话

route(App mut& app, Request const& request) 的参数说明了职责：它可能改变应用的会话状态，但只读取本次请求。代码的先后顺序很重要：

1. 请求解析失败：立即生成错误响应。

2. OPTIONS：直接生成预检响应。
3. GET /api/health：执行 SELECT 1，确认数据库实际可用。
4. /api/login：只允许 POST，并调用 app.login。
5. 非 /api/ 路径：返回 404，明确告诉调用方静态页面由另一个服务器提供。
6. 其余 API：提取 Bearer 令牌、查找会话，然后按资源分发。

核心的会话检查只有两行：

```
Session session = app.findSession(bearerToken(request.authorization));
if (!session.valid)
{
    return errorResponse(401, "未登录或会话已失效，请重新登录");
}
```

bearerToken 检查 Bearer 前缀，findSession 在 App.sessions 中查找令牌，找不到时返回 valid=false 的会话值。登录函数在核对用户名和口令之后创建会话，并把令牌、用户信息和可见模块树一起返回。注销则删除会话表里的对应记录，所以后续使用旧令牌的请求会失败。

issueToken 使用应用的随机状态生成 64 个十六进制字符。阅读代码时，应把“令牌字符串长度”与“密码学安全性”分开；这个练习项目不因此具备完整的生产认证方案。

24.6.4 dispatchStudents 与 allow：路由不直接等于业务授权

通过会话检查后，splitResource 把 /api/students 与后续资源部分分开，交给 dispatchStudents。它进一步同时判断资源尾段和 HTTP 方法。例如，根资源的 GET 是列表，POST 是新增；/{id} 的 PUT 是修改，DELETE 是删除；/query 只接受 POST。

下面是实际的列表与新增分支摘录：

```
if (request.method == "GET")
{
    if (!allow(session.level, "student", "list",&denial))
    {
        return denial;
    }
    return app.listStudents("", "");
}
if (request.method == "POST")
{
    if (!allow(session.level, "student", "add",&denial))
    {
        return denial;
    }
    return app.createStudent(&request);
}
```

同一个资源，动作不同，权限门槛也不同。allow 先通过 actionNeedLevel 查权限表；动作不存在属于服

务端配置错误，返回 500；当前等级不足则生成 403；通过后才调用业务方法。前端隐藏“新增”按钮只是界面提示，真正挡住直接 HTTP 请求的是这里。

班级、课程、成绩和用户模块采用相同的分发结构，但业务规则并不完全相同。例如，成绩有统计动作，用户模块还有重置口令与修改权限，不能只复制学生路由然后替换表名。

24.6.5 JSON、SQL 与结果封装：新增和查询各追一遍

新增学生时，`createStudent` 调用 `readJsonBody`，先确认媒体类型为 `application/json`，再解析正文。`textField` 读取字符串并 trim；`intField` 要求真正的 JSON 整数，类型不符或越界时给失败哨兵值。函数把需要的字段保存为独立的 String / 整数后，关闭 JSON 根对象，再校验姓名、性别和班级关系。

这个顺序同时展示了 Magel 的值语义和资源边界：临时 JSON 文档结束后，已经取得的值仍可用于 SQL；不能把借用文档内部对象的短命引用直接保存出去。阅读失败分支时，也要检查 `root.close()` 是否在应有位置执行。

SQL 值通过当前数据库连接的 `db.quote(...)` 转义。比如登录查询把姓名和口令作为值拼入固定 SQL，查询学生时也是在已知语句骨架上追加经过处理的条件。`quote` 处理的是值，不是任意表名和列名；启动参数里的数据库名称属于另一类配置输入，不能把它与浏览器传来的字段混为一谈。

列表查询可以从 `studentListSql` 开始读：

```
" FROM student st"
" LEFT JOIN sclass cl ON cl.sclass_id = st.sclass_id"
" LEFT JOIN selection se ON se.student_id = st.student_id"
```

LEFT JOIN 让“尚未分班”和“尚无成绩”的学生仍然出现在列表里。`COUNT(se.selection_id)` 统计实际成绩记录；换成 COUNT(*) 会把连接产生的空行也算进去。平均分使用 AVG、ROUND 和 IFNULL，`studentGroupSql` 补齐 GROUP BY 与排序，避免在 ONLY_FULL_GROUP_BY 模式下依赖不确定的分组行为。

执行结果由 `ResultSet` 承接，先检查 `rows.ok`，再读取行列。`studentRow` 把一行转成 JSON 对象，字符串经过 `jsonString` 处理；`listEnvelope` 最后补上 `items`、`count`、`total` 和 `limit`。因此页面既能显示当前返回多少行，也知道服务端列表上限。

24.6.6 sendResponse：统一协议出口，再回到浏览器

路由函数返回 `Response` 后，`sendResponse` 才负责写 HTTP 状态行、Content-Type、按 UTF-8 字节计算的 Content-Length、CORS 头和 `Connection: close`，最后追加正文。浏览器接到响应，按状态码决定显示数据还是错误信息，并用 `textContent` 把用户字段放进页面。

CORS 响应头让浏览器决定是否允许页面读取跨源响应，它不取代 Bearer 会话与后端权限。OPTIONS 不执行业务 SQL；真正的数据请求仍要重新走身份和动作检查。

现在可以拿一条操作自己追踪：登录管理员，点击“新增学生”，在浏览器网络面板中找到 POST `/api/students`，然后按 `readRequest → route → dispatchStudents → allow → createStudent → sendResponse` 对照源码。故意省略姓名、使用学生令牌或提交不存在的班级，分别观察字段校验、权限校验和关系校验在哪一步返回。

24.7 验收与交付

项目已经提供两类回归，使用独立测试库；在准备好 MySQL 的机器上运行：

```
python tests/magel_ipage_student_server.py --backends run
python tests/magel_ipage_student_server_browser.py --browser "C:/Program Files
    (x86)/Microsoft/Edge/Application/msedge.exe"
```

接口回归覆盖登录、角色权限、五模块业务、统计与 HTTP 边界；浏览器回归真正从不同源访问 API，检查菜单、增删改查和 HTML 注入防护。本章截图来自该真实浏览器回归，数据来自仓库夹具。

后端可通过 AOT 交付：

```
./magel.exe aot example/sql-magel/example/ipage-student-server/main.mgl --out build/student-server.exe
```

AOT 默认 checked 和 O3。前端目录仍单独部署，MySQL 服务与客户端依赖也需要准备；不能因为后端生成了可执行文件，就认为数据库与页面已经一并打包。

该示例沿用练习数据中的明文口令，会话没有到期与刷新机制，前端的登录尝试次数也不构成服务端限流。正式服务需进一步完善认证、会话、TLS、SQL 参数绑定、超时及并发；本章的目标是把现有应用的职责与请求链路讲清楚。

下一章进入另一个宿主：单片机上的 Magel 解释器。

第 25 章 单片机上的 Magel 解释器

桌面项目把程序交给操作系统运行，单片机需要自己提供启动、内存、文件和外设。仓库已经有一套 FK-STM32H743XIH6 板端工程：LCD 选择程序，FatFS 读取映像，按键控制运行，串口输出诊断。

本章按实际工程追踪一次“生成程序、加载执行、出错恢复”的过程。硬件与工具链的完整参数以板级 README、CMake 和源码为准。

25.1 单片机上解释的是什么

宿主机上的 Magel 编译器负责解析源码、检查类型与借用、优化四元式，最后导出 `.svmb`。板端读取字节码、数据区与布局信息，由共享 VM 执行；板端没有携带整套源码解析器，也不在运行时编译 C++。

```
桌面：.mgl → 解析与检查 → 优化四元式 → .svmb / .data.bin / .layout
板端：加载三份文件 → 配置内存与胶水 → VM 执行 → 清理本轮状态 → 返回菜单
```

源码入口	阅读重点
library/stackvm	桌面与板端共同的 VM 核心
source/stackvm_mcu	裁剪配置、静态池和宿主适配
板端 main.c	上电后的独立程序入口
magel_board.cpp	加载、菜单、执行与原生接口注册
programs/board.mgl	Magel 可调用的外设函数签名

板级 SDK 副本位于工程的 `sdk/`，不依赖原来的小车工程目录。共享 VM 不维护第二份实现，宏决定哪些能力编入固件。

25.2 先分清三类内存

真实 H743 工程从 SDRAM 分配如下三块区域：

区域	当前板级预算	存放内容
元数据	2 MiB	C++ 元数据分配、解码辅助与宿主对象
运行时 Arena	16 MiB	VM 容器、执行状态与运行时对象
字节池	4 MiB	程序分配、字符串与字节存储

总计 22 MiB，来自参考板的外部 SDRAM 配置，不代表只有少量 SRAM 的 MCU 也能直接使用。硬件容量变化时，需要同时核对 BSP 布局、链接脚本和三项预算。

板端关闭异常、RTTI、OS 线程、AOT 和大数，默认关闭反射，保留共享核心支持的执行优化。关闭 OS 线程不等于关闭协作式协程；具体标准库是否能用，还取决于模块门禁和宿主注册。

25.3 写一个可退出的 LCD 程序

在 programs/ 下创建练习源码，或使用 include 搜索路径找到 board.mgl：

```
%include "board.mgl"

void main() {
    lcdClear(0);
    lcdText(8, 8, "Hello Magel", 65535, 0);
    while (!boardExitRequested()) {
        if (keyClicked(2)) {
            lcdFill(8, 40, 100, 20, 2016);
        }
        boardDelay(10);
    }
}
```

boardExitRequested() 让程序协作响应 MODE/ENTER 退出。键号为 0=MODE、1=ENTER、2=UP、3=DOWN，颜色为 RGB565。boardDelay 的中断继续运行，程序也需要留出检查退出与输入的机会。

这是板端程序，需要相应胶水，不能直接在普通桌面 VM 中调用 LCD 函数。编译时指定板端关闭的反射配置：

```
./magel.exe svmb example/mcu/FK-STM32H743XIH6/programs/hello.mgl --no-reflection --out build/mcu-hello
```

上例命令使用仓库已有的 hello.mgl；若保存了自己的练习程序，替换源路径并添加对应 include 目录即可。三份输出必须使用同一个前缀。

25.4 从 Magel 签名到原生外设

board.mgl 声明函数类型，magel_board.cpp 的 RegisterBoard 使用原生注册表绑定名字、参数数量和回调，例如 lcdClear 对应一个参数。回调读取 NativeCallContext，检查坐标、颜色、文本和键号，再交给 BSP。

这里的分工很明确：Magel 负责程序逻辑，VM 负责执行与错误传播，胶水负责参数边界，BSP 负责具体硬件。新增外设时，应同时维护 Magel 签名、回调、注册参数数量与错误行为。

LCD 文本当前支持可打印 ASCII，不能把桌面字符串支持 UTF-8 等同于板上字体已经支持中文。串口为 USART1、PA9/PA10、115200 8N1；println 和 VM 诊断通过该宿主输出。

25.5 构建、加载与菜单

完整构建命令见板级构建说明。需要 CMake、Clang/LLVM、Make 和 Cortex-M7 硬浮点 Arm C/C++ 运行库；仓库提供运行库获取脚本。默认 SDK 来自本工程 sdk/，构建输出位于 build-local/board-sdk/。

程序加载使用 FatFS 的 0:/magel/。参考 BSP 中的 0: 对应外部 W25QXX 卷，不是自动识别任意 SD 卡。把每个程序的三份文件放在同一目录：

```
0:/magel/hello.svmb
0:/magel/hello.data.bin
0:/magel/hello.layout
```

单个部署文件上限为 1 MiB。缺文件或映像损坏时，显示失败并返回菜单，不悄悄运行另一份程序。菜单最多 32 项，包含内置测速、正常退出、故意 panic 三个入口。

默认上电自动运行一次 benchmark，完成后进入按键演示，再返回菜单。UP/DOWN 选择，ENTER 运行，MODE 重新扫描。若想上电直接显示菜单，构建时使用 `-DFK_AUTORUN_BENCHMARK=OFF`。



板端程序从菜单加载、执行到清理返回的生命周期流程

25.6 把五个源码入口串起来

前面的表格是定位地图，下面才是代码的调用关系。读这一套工程时，不要把“共享核心”“桌面 MCU 宿主”和“真实板级入口”当成三个不同的解释器：它们围绕同一个 `stackvm::Machine` 组合出不同的宿主。

25.6.1 library/stackvm：程序、数据与执行器分开

先看 stackvm.hpp 的几个公开类型。Program 保存加载后的程序与执行元信息，DataArea 保存数据及布局，NativeRegistry 按名字和参数数量登记宿主函数，Machine 才是执行状态与运行时服务的入口。RunResult 把成功状态、诊断、执行步数等结果交还宿主。

loadProgramBinary 把 .svmb 解码为 Program，DataArea::loadFromBytes 把数据字节和 layout 组合为运行时数据区。二者可以分别失败，所以板端 Execute 给它们安排了独立诊断和返回码，不等到第一条程序指令执行时才猜测“是不是文件读错了”。

Machine::run 使用共享的执行核心及其默认优化。源码解析、类型检查和四元式优化已经在桌面编译阶段完成；这里不会再次解释 struct、展开泛型或调用 AOT 编译器。想追具体指令的执行，再从 stackvm.cpp 包含的执行模块继续进入寄存器核心与执行准备逻辑。

这层不应直接知道 LCD 型号或 W25QXX 引脚。它通过字节堆、文件 API、输出下沉与原生注册表接收宿主能力，才能同时用于桌面和板端。

25.6.2 source/stackvm_mcu：把桌面服务替换为有预算的宿主

mcu_host.cpp 是可在桌面运行的 MCU 配置验证入口。它读取映像、安装内存池、挂接输出和原生胶水，再运行同一份字节码。它方便检查裁剪构建、程序结果和软重启，不能由此推断真实 LCD、串口或 SDRAM 已经验证。

把这一目录按职责拆开读会更清楚：

文件	要解决的具体问题
mcu_arenas.cpp/.hpp	安装元数据、运行时 Arena 与程序字节池，提供当前池与水位
mcu_metadata_heap.cpp	为 C++ 元数据分配接入有预算的存储
mcu_glue.cpp	注册板端基础转换、文本输出等原生函数，并挂接输出下沉
mcu_filesystem.hpp、mcu_file_api.cpp	把 VM 文件操作交给宿主文件系统 API
mcu_console.hpp	输入回调约定，供桌面 stdin 或板上串口实现

InstallRuntimeArena() 把 Memory:: 容器接到指定 Arena，因此应在构造使用这些容器的运行状态之前安装。板级代码先注入元数据与运行时池，安装分配器，再构造字节池的身份索引；顺序变化可能让长期对象先分配到错误的池。

mcuGlue::Register(machine, &registry) 把基础胶水注册到指定表，再通过 machine.setNative Registry 交给机器。输出经 setOutputSink 下沉到宿主，keepText=false 时不必把整轮输出积累在 RAM，串口可在运行过程中持续收到文本。

25.6.3 板端 main.c：先让硬件能支撑 C++ 宿主

真实板端的 main.c 先关闭中断，配置 MPU、Cache、HAL、系统时钟和 DWT，再初始化定时器、SDRAM、内存管理、串口与按键。打开中断后，输出启动标记，初始化 LCD 与 W25QXX，最后调用 Magel_BoardRun()。

这段顺序解释了为什么不能一上电就创建大容量 VM 容器：外部 SDRAM 与管理器此时还没有就绪。早期少量静态构造的分配由工程明确配置的启动堆承担，之后才切换到外部池。

主入口没有启动电机、舵机和小车任务。TIM6_DAC_IRQHandler 提供时基并驱动按键消抖；TIM7_IRQHandler 只确认参考工程创建的中断。读者修改固件时，应保留这些服务之间的边界，不能因为 SDK 中存在驱动就认为相关任务正在运行。

25.6.4 magel_board.cpp：菜单拥有宿主，Execute 拥有一轮程序

Magel_BoardRun 从 BSP 的外部堆申请三块池，注入元数据与运行时分配器，然后设置字节池、挂载 FatFS、扫描程序并绘制菜单。挂载只尝试读取已有卷，不在失败时格式化介质。

菜单循环维护 selected、run 和 lastStatus。UP/DOWN 改变选择，MODE 重扫，ENTER 把 run 设为真；真正运行发生在下一轮的 Execute(entries[selected])。默认启动测速与自检配置也通过这些状态进入执行路径，而不是再维护一套 VM 入口。

Execute 的前半段决定映像来源。内置程序直接引用 Flash 中的指针与长度，外部程序则按同一文件名前缀读取三份 FatFS 文件。ReadFile 检查 1 MiB 上限、实际读取长度和关闭结果；只要一份失败，就返回错误，不偷偷切回内置程序。

解码 Program、加载 DataArea 后，实际绑定与执行代码如下，摘自工程：

```
stackvm::NativeRegistry registry;
stackvm::Machine machine;
machine.setHeapAllocator(&mcuArenas::CurrentBytePool());
machine.setData(&data);
machine.setFileApi(&mcuFilesystem::CurrentApi());
mcuGlue::SetOutput(SerialWrite, nullptr, false);
mcuGlue::Register(machine, &registry);
RegisterBoard(registry);
mcuConsole::SetRead(SerialRead, nullptr);
const auto result = machine.run(program, {}, 10000000u);
```

这里先挂基础胶水，再追加板级接口，两者使用同一个注册表。注册表、数据区和字节池必须在 run 期间有效；registry 比 machine 更早构造，退出作用域时也会更晚析构。字节池是宿主长期提供的对象，不能拿临时池的地址交给机器。

Execute 对读文件、字节码解码、数据加载和执行失败分别返回 2、3、4、5，成功返回 0。菜单接到返回值后记录状态、输出三类池的峰值、清理按键事件并重新绘制界面，因此“运行结束”与“回到可操作菜单”是同一条可追踪路径。

程序、数据区、Machine 和本轮注册表都在 Execute 返回前析构。承载 C++ 静态对象的元数据池却不能直接整块重置，否则仍存在的静态对象会保存失效地址；源码特别保留了这一点。恢复一轮 VM 与清空全部 SDRAM 是不同操作。

25.6.5 board.mgl 与 RegisterBoard：名字和实参数量必须对齐

board.mgl 是 Magel 一侧的宿主函数声明，例如：

```
void lcdClear(int64 color);
bool boardExitRequested();
void boardDelay(int64 milliseconds);
```

它们没有 Magel 函数体，编译器按声明检查调用。执行时，RegisterBoard 把同名符号绑定到 C++ 回调：

```
registry.add("lcdClear", 1, LcdClear);
registry.add("boardExitRequested", 0, BoardExitRequested);
registry.add("boardDelay", 1, BoardDelay);
```

声明、名字、参数个数、参数含义和返回值必须同时一致。只在 board.mgl 增加声明不会凭空产生硬件能力；只增加 C++ 回调却没有注册，也不能按名字调用。

回调通过 NativeCallContext 读取参数并执行范围检查，再调用 BSP。比如 LCD 要检查颜色与坐标，按键要检查键号。板端禁用 C++ 异常，失败应写入原生调用的错误状态，让 VM 返回失败结果；不能在这里抛异常并期待菜单捕获。

最后沿一个调用自己追一遍：Magel 程序调用 lcdClear(0)，编译器产生宿主调用，VM 查 NativeRegistry 找到 LcdClear，回调取整数、检查范围，再调用 LCD 桥接函数。println 则走基础输出下沉到 SerialWrite。同一套程序逻辑由不同胶水连接到不同宿主，这正是共享核心能够上板的原因。

25.7 验证退出与错误恢复

依次验证：正常返回、主动退出、语言 panic、VM 指令预算超限、缺少数据文件。每轮应释放程序、数据区、VM 与注册表，并回到选择页；默认每轮预算为 10,000,000 条 VM 指令。

硬件 Fault、原生 C/C++ 越界或不可恢复的分配器耗尽不属于语言 panic，不能据此承诺任何错误都能恢复菜单。读代码时应区分 VM 的失败返回和 CPU 的硬件异常。

没有板子时，可先使用桌面上的独立 MCU 宿主回归：

```
python tests/mcu_vm.py --magel magel.exe
python tests/mcu_fk_controls.py
python example/mcu/FK-STM32H743XIH6/tools/verify_sdk.py
```

MCU 宿主回归使用其定义的测试预算，检查裁剪构建、异常符号与选定字节码执行；它不替代 LCD、按键、串口和 SDRAM 的实机测试。板级 README 保留真实硬件的原始测速与恢复记录，历史数据应保留原日期和配置。

做到这里，项目的主线就完整了：同一套语言和 VM 语义，桌面负责生成映像，板端提供受预算约束的宿主，用原生接口接入硬件，并在每轮程序结束后恢复可操作状态。

附录 A 常见错误速查

学一门语言，一半时间花在「看懂报错」上。这份附录把本书撰写过程中真跑出来的常见报错收在一起：每条都写清了它长什么样、你为什么踩到、怎么改。

> **提示**
>
> 这里的每一条消息都是 Magel 1.0.0 工具链的真实输出（中文本地化）。带 `<!-- fail -->` 标记的代码块会在本书的校验脚本里每次都被重跑一遍，所以它们不会随着工具链演进而失真。

A.1 报错的三种长相

Magel 的报错分三类，看开头几个字就能分清：

开头	是哪一类	什么时候出现
`magel：……`	编译期错误（语义检查）	运行之前，代码「含义不对」
`语法错误：文件第行第列：……`	语法错误	运行之前，代码「话都说不通」
`magel：运行期错误：……`	运行期错误	程序跑到那一句才炸

编译期错误和语法错误都不会生成程序，改好再跑就行。运行期错误说明程序编过了、跑起来了，但执行到某一步出了事（除零、越界、unwrap 空值……），后面还会跟着出错位置和一段 调用栈（外 → 内），告诉你是哪一行、哪个函数出的事。

一个真实例子，`1 / 0` 跑出来的样子：

```
int64 main()
{
    int64 a = 1;
    int64 b = 0;
    println("{}", a / b);
    return 0;
}
```

```
magel：运行期错误：整数除零
出错位置：hello.mgl:4（main）
调用栈（外 → 内）：
  main  hello.mgl:4 [thread 0 coroutine 0 ip 12]
```

从上往下读：先是「整数除零」，再是出错的行号，最后是一份调用栈——调用栈（外 → 内）意思是从最外层往里读，最后一行是炸掉的那一层。

诊断码。一部分编译期错误带一个 `[MGLxxxx]` 编号（比如下一节的 `[MGL1204]`）。编号是稳定的，遇到没见过的错误可以把编号拿去官方文档里搜。

A.2 编译期错误速查

这是你会遇到最多的一类。按「你可能写下的东西」排序：

现象	真实消息	怎么办
用没声明的变量	`magel: 引用了没声明的变量：y`	检查拼写；变量必须先声明再用
调没定义的函数	`magel: 调用了没定义的函数：nosuch`	名字拼错，或者忘了 `import`
同一个块里重名	`magel: 同一个块里不能重复声明：x（要遮蔽外层变量就在内层块里声明）`	换名字；想遮蔽就用内层块
给 const 赋值	`magel: 不可变变量（const）不能再赋值：LIMIT`	去掉 `const`，或不该在这里改
类型对不上（赋值/返回/传参）	`magel: 不能隐式转换：String -> int64（请显式强转）`	见下面 A.2.1
整数字面量超出目标范围	`magel: 整数字面量 200 超出 int8 的取值范围（-128..127）`	换类型，或换成范围内的值（见 A.2.2）
`if` 条件不是布尔	`magel: if 条件必须是 bool，现在给的是 int64（数值条件请显式写成 (bool) 值或比较表达式）`	写比较表达式，别拿数字当布尔
实参个数/类型对不上	`magel: 调用函数 f 找不到匹配的重载：(int64, int64, int64)；没有参数类型能对上的重载（候选：f）`	数实参个数；看类型是否精确对上
接口没实现完	`magel: struct Circle 实现了接口 Shape，但没有完全实现：area() -> int64（接口要求同名、同返回类型、同形参类型；一个都不能少）`	把接口里的方法补全，签名一字不差
`match` 漏了分支	`magel: enum match 必须覆盖所有变体或提供 _ 兜底分支；缺少：Color::Green`	补上缺的变体，或加 `_ =>` 兜底
自定义泛型名撞了内置	`magel: generics: 重复的泛型声明 Pair`	换个名字（内置有 `Pair`，改叫 `Duo` / `Entry` 之类）
找不到模块	`magel: 模块：找不到模块 no_such_module（需要 no_such_module.mgl）`	检查 `import` 的名字与文件是否在同目录
引用声明没初始化	`magel: 引用声明必须初始化，缺省值请使用 Option<T mut&>`	引用必须当场绑定；可能为空就用 `Option<T mut&>`
只读引用被写入	`magel: 只读引用（T const&）不能写入元素`	改用 `T mut&`
悬垂引用（返回局部引用）	`magel: [MGL1204] 返回值中的引用不能超过局部来源的生命周期（int64 mut&，所属类型，来源 local）`	别把指向局部变量的引用交出去
引用已被释放还在用	`magel: [MGL1205] 借用来源已释放，不能继续访问：表达式`	检查 `Drop` / 作用域，别在对象没了以后还用它

A.2.1 「不能隐式转换」——最高频的一条

九成的类型报错都是这一句。先记住第 3 章 3.8 节的规矩：Magel 只放行「装得下」的方向。下面这几种会自动放行（不用签字）：

方向	例子
同符号整型变宽	int8 → int64、int32 → int64、uint8 → uint32
无符号 → 更宽的有符号	uint8 → int32、uint32 → int64
整型 → 浮点	int64 → float64、int32 → float32
浮点变宽	float32 → float64

反方向一律要显式强转（目标类型）值。最常见的四种：

```
int64 main()
{
    int64 big = 70000;
    int32 small = big;
    // 缩窄：位宽变小
    println("{}", small);
    return 0;
}
```

```
magel: 不能隐式转换：int64 -> int32（请显式强转）
```

- 缩窄（位宽变小）：int64 → int32、float64 → float32、uint8 → int8 → 写 (int32)big
- 有符号 → 无符号：int32 → uint32、int64 → uint64（目标更宽也不行，负数塞不进去）→ 写 (uint32) x
- 跨大类：float64 → int64、char/bool ↔ int64 → 写 (int64)f
- 浮点字面量：float32 f = 1.5; 里的 1.5 是 float64，往更窄的 float32 放也要签字 → (float32)1.5

> **提示**
> + 号在谁身上，谁就有错。读这条报错只要看箭头两边：左边是「你给的」，右边是「它想要的」，改左边（强转）或者右边（换类型）都行。

A.2.2 「整数字面量超出取值范围」

这是「字面量落进槽位」时独立的一道检查——十进制字面量的值装不进目标整型就报错：

```
int64 main()
{
    uint8 b = 256;
    // uint8 只到 255
    return 0;
}
```

```
magel: 整数字面量 256 超出 uint8 的取值范围（0..255）
```

规矩（第 3 章 3.10 节）：

- `uint8 b = 255;` ✓（在 0..255 里）、`int16 s = -300;` ✓（在 -32768..32767 里）；
- `uint8 b = 256;` ✗、`int8 x = 200;` ✗ ——不再静默回绕；
- `0x` / `b` 前缀的字面量按位模式放行（`int8 x = 0xFF;` 得到 -1），不做范围检查；
- 显式强转不走这条检查：`(uint8)256` 得到 0，是你签字的结果。

A.2.3 引用相关的错，多在「生命周期」上

引用一共就两类错，认清就能改：

第一类：引用没绑定就声明。Magel 没有空引用：

```
int64 main()
{
    int64 mut& r = 5;
    // 引用要从对象身上取地址，不能绑字面量
    return 0;
}
```

```
magel: 不能隐式转换：int64 -> int64 mut&（请显式强转）
```

改成绑定一个确实存在的对象：`int64 x = 5; int64 mut& r = &x;`。

第二类：引用活得比目标久。编译期就会拦：

```
int64 main()
{
    int64 mut& r;
    println("{}", r);
    return 0;
}
```

```
magel: 引用声明必须初始化，缺省值请使用 Option<T mut&>
```

把指向局部变量的引用返回出去，报的是 [MGL1204]：

```
magel: [MGL1204] 返回值中的引用不能超过局部来源的生命周期（int64 mut&，所属类型 ，来源 local）
```

规矩只有一条：来源必须比引用活得久。来源是参数、是调用方给的，就安全；来源是本函数的局部变量，就不行。详细展开见第 7 章。

A.3 语法错误速查

语法错误长这样：`语法错误：文件第行第列：期望……，实际是……`，并且会画出出错位置和一个小波浪线 `~~~~`。消息里的记号名（条件 `(if)`、分号 `(;)` 之类）都是中文，照着它说的补就行，行列号也是准的。

现象	真实消息	怎么办
语句少写分号	语法错误：x.mgl 第 3 行第 5 列：期望条件 (if) / 自增 (++) / 自减 (--) / 加号 (+) 等 44 个之一，实际是返回 (return)	看行号，上一句末尾补 ;

类型声明的尾部分号是可选的：struct、enum、interface 在 } 结束，可以再写 ;，也可以不写。不要把合法的省略分号误当成错误；普通声明、赋值、返回和 do ... while 的分号仍然要保留。

```
struct Point
{
    int64 x;
    int64 y;
}

int64 main()
{
    Point point;
    println("{}", point.x);
    return 0;
}
```

```
0
```

其它几条词法/语法坑（第 3 章 3.10 节实测过）：

你可能想写	实际会怎样	改成
0b1010	词法错误（前缀是 b）	b1010
1_000	语法错误（下划线分隔不认）	1000
1.0F / 1U	后缀必须小写	1.0f / 1u

A.4 运行期错误速查

运行期错误都带 magel：运行期错误：前缀，后面跟具体原因、出错位置和调用栈。它们不是「未定义行为」，是明确被检查到的错误——出问题的那一下，程序会停下来告诉你，而不是悄悄读到垃圾数据。

现象	真实消息（首行）
整数除以零	`magel: 运行期错误: 整数除零`
定长数组越界	`magel: 运行期错误: 数组下标越界: fixed 的下标 5, 长度 3`
List 越界	`magel: 运行期错误: vec 下标越界: 5（长度 1）`
展开空的 Option	`magel: 运行期错误: panic: called Option.unwrap() on None`
展开 Err 的 Result	`magel: 运行期错误: panic: called Result.unwrap() on Err`
expect 落空	`magel: 运行期错误: panic: <你写的消息 >`
assert 失败	`magel: 运行期错误: panic: <你写的消息 >`
主动 panic	`magel: 运行期错误: panic: <你写的消息 >`

unwrap 空 Option，一次看清楚它长什么样：

```
int64 main()
{
    Option<int64> none = None;
    println("{}", none.unwrap());
    return 0;
}
```

```
magel: 运行期错误: panic: called Option.unwrap() on None
```

> **提示**
>
> 注意区分两种「空」。`Option<int64> none;`（默认构造）不是空——它的 `isNone()` 是 0，里面装的是默认值 0。真正的空要显式写 `Option<int64> none = None;`，这时 `isNone()` 才是 1。第 16 章讲过这个区别。所以 unwrap 报错的是「`= None` 那种空」，不是随便一个 Option 变量。

越界的两条消息措辞不同（一个是数组、一个是 List），别以为是两种错：

```
magel: 运行期错误: 数组下标越界: fixed 的下标 5, 长度 3      <- 定长数组，带变量名
magel: 运行期错误: vec 下标越界: 5（长度 1）                  <- List
```

A.5 工具链与工程错误

现象	真实消息	怎么办
用了注解/反射但工具链不支持	`magel: x.mgl 第 1 行第 1 列: 注解与反射功能未开启，请使用支持该能力的工具链及 --reflection`	用带反射的构建，运行时加 `--reflection`（第 20 章）
调了不存在的方法	`magel: 方法只能调在 struct 上: fooBar`	方法名拼错，或那个类型上真没有这个方法

现象	真实消息	怎么办
嵌入模式用了被禁的能力	（编译期门禁）	见第 22 章的 [embedded] 一节

方法只能调在 struct 上：xxx 这条措辞有误导性：它不是说「你不能在 struct 上调方法」，而是说「你调的这个方法没找到」。看到它，先回去核对方法名。

反射那条同理——不是你的代码错，是工具链没开那个功能。本书第 20 章的所有例子都得靠一个开了反射的工具链来跑，默认构建会直接拒绝。

A.6 排查五步法

卡住了，按这个顺序走，八成能自己解决：

1. 看开头几个字，定类别：语法错误 → 语法；magel： → 编译期；magel：运行期错误： → 运行期。三类改法完全不同。
2. 语法/编译期：看行号，往上一行找。报错行常常只是「被发现」的地方，真正的问题往往在上一行（典型的：少分号）。
3. 类型报错：读箭头。不能隐式转换：A -> B 就是「你给了 A，它要 B」。要么强转左边，要么把右边换成 A。
4. 引用报错：问「来源是谁」。来源是本函数的局部变量吗？是就错在这；来源是参数、是更长命的对象吗？那就是别的地方。
5. 还是不懂：拿消息关键词去查。消息里的中文（比如「只读引用」）和诊断码（[MGL1204]）都能拿去文档或代码里搜——这套消息是刻意写清楚给人看的。

A.7 下一步

错误看得懂，剩下就是写。翻到附录 B，按能力域认识一遍标准库；或者回到第 23 章，拿一个完整项目把学到的东西串起来。

附录 B 标准库速查

Magel 的「标准库」不是一堆函数，而是一组模块：除核心 `std::lang` 自动加载外，其余都得先 `import`。这份附录按能力域把它们列一遍，让你知道「想干这件事，该去 `import` 谁」；每个模块的完整签名与错误边界，去库函数 API 参考查。

> **提示**
>
> 模块名与方法对齐 Magel 1.0.0。终端中可以直接用 `magel man` 查模块，用 `magel man 模块::符号` 查签名、说明和示例；命令说明见 doc/man。

B.1 先看一个例子

```
import std::math;
import std::text;
import std::iter;

int64 main()
{
    println("sqrt={}", std::math::sqrt(144.0));
    println("upper={}", std::text::Text::toUpper("magel"));
    int64 sum = 0;
    foreach (int64 v in std::iter::range(1, 5))
    {
        sum += v;
    }
    println("sum={}", sum);
    return 0;
}
```

```
sqrt=12
upper=MAGEL
sum=10
```

三行分别来自三个模块：数学、文本、迭代。注意 `range(1, 5)` 是左闭右开（拿到 1、2、3、4），所以 `sum` 是 10——这是第 17 章的规矩，标准库里凡带「起点、终点」的地方都一样。

B.2 核心模块 `std::lang`（不用 import）

这几样任何程序里都能直接用：

能力	用法
输出	`println(fmt, …)`、`format(fmt, …)`
断言	`assert(cond)`
取长度	`len(x)`（字符串 / 容器）
字符串	`std::lang::String`
容器	`List<T>`、`Map<K,V>`、`Set<T>`
类型擦除盒子	`std::lang::Any`（第 13 章的运行期多态）
数学常数	`std::lang::Math::PI`、`std::lang::Math::E`
大数	`BigInteger`、`BigDecimal`
输入	`input(variable)`
整行输入	`std::lang::readLine()`，保留空格，不含 LF/CRLF
多行输入	`std::lang::readLines(int32 N)`，LF 连接，EOF 提前结束

提示

早年间它们不在 `lang` 里也能用（靠 `source/std/lang.mgl` 注入正文）。现在除了上面这些，其它能力一律要显式 `import`——这是第 18 章「模块与工程」讲过的变化。

B.3 按能力域速查

B.3.1 数学与数值

模块	能干什么
`std::math`	`sin cos sqrt` 等常用函数（常数见 `std::lang::Math`）
`std::numerics`	标量数值方法：导数、辛普森积分、二分、RK4、牛顿迭代
`std::numtheory`	整数数论：`gcd lcm` 取模运算，以及确定性的 `isPrime`（Miller-Rabin）
`std::complex`	复数 `Complex<T>`，加减乘除都是一等公民
`std::linalg`	稠密向量与矩阵：`luDecompose symmetricEigen svd jordan2 diagonalize2`
`std::random`	确定性随机数（LCG）与熵播种；句柄型状态，实现 `Drop`
`std::signal`	DFT / FFT 与逆变换；基 2 的 FFT 只接受 2 的幂长度

B.3.2 文本与数据格式

模块	能干什么
`std::text`	`Text::find split join replace trim toUpper padStart ……`
`std::json`	`parse` → `Result<JsonValue, String>`；再 `isInt isObject len asInt asString` 取值
`std::toml`	与 json 同构的 TOML 文档访问
`std::regex`	正则；模式缓存在宿主侧，Magel 侧只拿一个整数句柄

模块	能干什么
std::parse	字符串 → 整数，失败给 ParseError
std::binary	定宽解码、字节序处理与边界检查
std::path	路径拼接、取文件名、取扩展名、规范化
std::compress	文本压缩 / 解压；压缩结果是二进制 List<uint8>

B.3.3 并发与协作

模块	能干什么
std::coroutine	共享协程
std::thread	隔离线程 IsolatedThread
std::message	跨协程 / 跨线程的消息
std::concurrency	显式共享槽 Shared<T>（普通全局变量在线程间仍是隔离的）
std::event	中断事件与软件定时器；start() 后按周期在指令边界触发

> **提示**
>
> SharedCoroutine / IsolatedThread / Message 是三个独立模块，不能合并——它们各自有同名的宿主函数 create / wait / receive。

B.3.4 系统与环境

模块	能干什么
std::io	文件读写；错误类型 IoError（OpenFailed ReadFailed WriteFailed SeekFailed Closed）
std::net	TCP 与极简 HTTP 客户端；套接字在宿主胶水里，Magel 侧只拿 int32 句柄
std::sql	连 MySQL / SQLite / PostgreSQL / SQL Server；SQL 交给数据库引擎执行
std::process	进程与系统调用；命令管道由宿主管柄池持有
std::clock	单调时钟 steady_clock，只用来做差值，别当挂钟用
std::memory	Span<T>（指针 + 长度视图）；它的迭代器给的是元素指针，能直接写
std::watchdog	软重启看门狗（水位与超时两个维度都默认关闭），桌面与板端共用一份实现

B.3.5 迭代

模块	能干什么
std::iter	range、map / filter / collect 惰性链，以及 IterNext<T> 接口

> **提示**
> std::probe 是测试用的桩模块，不是给应用用的。

B.4 三条容易踩的约定

一、句柄型资源要显式关。std::json、std::regex、std::sql 这类持有宿主资源的值，用 close() 关，或者让它实现 Closeable、由作用域替你关（第 8 章）。重复关闭返回 false，不会崩。

二、压缩结果是二进制，不是文本。std::compress 的输出含 \{}0 字节，类型是 std::lang::List<uint8>；别拿它当字符串打印，解压出来的才是 std::lang::String。

三、前端嵌入式配置固定禁用部分模块。std::net、std::process、std::sql、std::io、std::thread、std::coroutine、std::concurrency 在 [embedded] 目标下 import 就会编译报错——不是你的代码写错了，是那一端不提供这个能力。见第 22 章。

B.5 细节去哪儿查

你想查	去哪
某个函数的确切签名、默认值、返回与错误	库函数 API 参考 与全部函数索引
按能力域通读一遍标准库	07 标准库概览
标准库源码（每个模块头部都有能力说明）	source/std/ 下的 source/std/<名字 >.mgl
宿主侧绑定（std::* 的真正实现）	source/glue/ 下的 source/glue/std_*.hpp
可直接运行的标准库样例	example/samples/

> **模块名怎么读**
> std::json 对应源码 source/std/json.mgl、胶水 source/glue/std_json.hpp，三处名字一对就找到了。

B.6 下一步

标准库认完了，前面二十几章也读完了——接下来该自己写点东西了。

- 想先复习报错怎么读：翻回附录 A；
- 想把整本书的东西拼成一个完整程序：第 23 章综合项目；
- 想把这门语言接到你自己的 C/C++ 代码上：第 21 章与 C 互操作。